



Requirement & Design Specification
Crowndine - Restaurant Management System(RMS)

Subject: SWD392

Version: 1.0

Record of Changes

Version	Date	In Charge	Change Description
1.0	July 2026	Project Team	Initial version

* A - Added M - Modified D - Deleted

Table of Contents

Record of Changes	i
I Requirement Specification	1
I.1 Problem Description	1
I.2 Major Features	1
I.2.1 Features for Customers	1
I.2.2 Features for Staff	2
I.2.3 Features for Admin	3
I.3 System Context	4
I.4 Non-Functional Requirements	5
I.5 Functional Requirements	6
I.5.1 Customer Portal & Management	6
I.5.2 Staff Portal & Restaurant Operations	7
I.5.3 Administrative Portal	8
I.5.4 Use Case Diagrams	9
I.5.5 Use Case Descriptions	14
I.5.6 Activity Diagrams	51
II Analysis Models	57
II.1 Interaction Diagrams	57
II.2 State Diagram	63
II.3 Communication Diagram	65
III Design Specification	67
III.1 Integrated Communication Diagrams	67
III.2 System High-Level Design	67
III.3 Component & Package Diagram	69
III.4 Detailed Design	72
III.5 Database Design	80
IV Implementation	83
IV.1 Map architecture to the structure of the project	83
IV.1.1 Backend Subsystem (Spring Boot 3.5 Application)	83
IV.1.2 Frontend Subsystem (React 19 SPA)	85
IV.1.3 Project Directory Structure Overview	85
IV.2 Mapping Class Diagrams & Interaction Diagrams to Code	86
IV.3 Applying Alternative Architecture Patterns	94
IV.3.1 1. Microservices Architecture	94
IV.3.2 2. Event-Driven Architecture (EDA)	94
IV.3.3 3. Serverless (Function-as-a-Service - FaaS)	95
IV.3.4 Trade-off Analysis Matrix	95
V Service-Oriented Architecture (SOA)	96
V.1 Applying the Service-Oriented Architecture	96
V.2 Applying Service Discovery Pattern in the Service-Oriented Architecture	98

I Requirement Specification

I.1 Problem Description

Crowndine is a restaurant management system designed to support daily restaurant operations, including customer ordering, staff coordination, menu management, reservation handling, and order tracking. The system aims to reduce manual effort, improve service accuracy, and provide a centralized platform for restaurant workflows.

I.2 Major Features

The CrownDine system provides separate feature groups for customers, staff, and administrators. These features are adapted from the implemented business logic in both the Backend and Frontend, including reservation workflows, online payment, restaurant operations, management dashboards, and AI chatbot integrations.

I.2.1 Features for Customers

F01: Viewing Menu & Product Details

- Browse food items and drinks grouped by categories.
- Browse food combo packages.
- View detailed information of items and combos, including descriptions, prices, images, and customer feedback.
- Search items and combos by name and filter them by categories.
- Add items to the favorites list, view favorite items, and manage the favorites list.

F02: Smart Table Reservation

- Use a step-by-step reservation wizard.
- Select booking date, dining time slot, and guest count.
- View an interactive table map showing table locations, floor layouts, table status, shape, and seating capacity.
- Select preferred tables for the reservation.

F03: Pre-order Cart Management

- Add food items and combos to the reservation order during booking.
- View the food reservation cart.
- Edit cart items by changing quantities or removing items.

F04: Booking Placement & Payment

- Review reservation details and total price before checkout.
- Apply discount vouchers.
- Make online payments through PayOS integration using QR Code or bank transfer.
- Receive real-time payment success or failure redirection and booking confirmation.

- Cancel pending or upcoming bookings.

F05: Customer Authentication

- Register a new account with email verification via OTP.
- Log in using username and password or Google OAuth2.
- Securely log out, refresh login tokens, and update passwords.

F06: Profile & Loyalty Program

- View and update customer personal profile details.
- Request email update with OTP verification for enhanced security.
- Upload profile avatar images processed and hosted through Cloudinary.
- View reservation history, order details, and statuses.
- Track accumulated loyalty membership points.
- Exchange loyalty points for discount vouchers.

F07: Customer AI Booking Assistant

- Chat in real time with AI chatbot for booking assistance.
- Ask questions about the menu, dishes, prices, and opening hours.
- Automatically parse relative time expressions into actual calendar dates
- View available table recommendations matching guest counts.
- Confirm table selection and receive direct routing links to step 3 for pre-order menu or step 4 for payment and checkout.

I.2.2 Features for Staff**F08: Table Reservation Processing**

- View the full list of customer reservations and filter by date ranges and reservation statuses, including Pending, Confirmed, Checked-in, Completed, and Cancelled.
- Look up booking details and search reservations.
- Perform customer check-in when guests arrive at the restaurant.
- Reassign or update tables for an active reservation.
- Cancel reservation bookings for customers with required cancellation reasons.

F09: Cashier & Walk-in Dine-in Orders

- Create walk-in dine-in orders directly for a table.
- Retrieve customer profiles by phone number to link them to orders for point accumulation.
- Append items, modify quantities, or remove items in active dine-in orders.
- Apply vouchers to active orders.
- Merge tables or split bills.
- Generate PayOS checkout QR codes for bills, process payments, and track transactions.
- Print invoice receipts and close completed tables by setting table status to Free.

F10: Kitchen Display System & Order Processing

- Use a real-time Kitchen Display System (KDS) for kitchen staff to view pending and preparation order queues.
- Group identical ordered dishes into batch preparations.
- Update item status in real time from Pending to Preparing to Ready using WebSockets.
- Automatically notify floor staff through toast notifications when dishes are marked as Ready for serving.

F11: Shift Schedule & Attendance Tracking

- View assigned weekly work schedules.
- View personal attendance history.
- Perform check-in and check-out to log work hours.

I.2.3 Features for Admin**F12: Restaurant Layout & Table Design**

- Add, edit, and delete floors and areas.
- Add, edit, and delete restaurant tables.
- Configure the visual layout by editing table names, coordinates, width, height, shape, and seat capacity to build an interactive floor plan.

F13: Menu & Category Management

- Add, edit, delete, and search food categories.
- Add, edit, delete, and search food items and combo packages.
- Manage item pricing, upload dish images through Cloudinary integration, write descriptions, and update combo item lists.

F14: Promotion & Voucher Management

- Create, edit, and delete discount vouchers.
- Configure voucher types, including percentage discounts and fixed amounts, minimum order values, expiry dates, usage limits, and descriptions.
- Manually assign vouchers to specific users.
- Manage point exchange rates and point costs for vouchers.

F15: Staff Account & Role Assignment

- Create new staff accounts.
- Search staff profiles by name and view detailed staff information.
- Update staff profiles, edit status as Active or Blocked, and assign system roles and authorities such as Admin, Staff, Kitchen, and Cashier.

F16: Employee Shift Scheduling

- Configure daily shifts, such as Morning, Afternoon, and Evening.
- Assign weekly work schedules to staff members.
- Update or reassign scheduled shifts, change work schedule status, and delete schedules.

F17: Attendance Auditing & Payroll Summary

- View weekly schedules and clock-in statuses for all staff.
- Track detailed clock-in logs.
- View monthly attendance summaries.

F18: Sales Analytics Dashboard & Excel Export

- View interactive dashboard sales graphs, including revenue by hour or day, customer counts, and growth rates.
- Export comprehensive sales reports to Excel spreadsheets.

F19: Admin AI Analytics Assistant

- Chat with Gemini AI to ask questions about sales, revenue reports for today, week, or month, and growth trends.
- Request lists of top-selling combos based on transaction data using tool-call integration.
- Receive operation performance analysis and predictive insights.

I.3 System Context

The system interacts with customers, restaurant staff, administrators, payment services, notification services, and the restaurant database. External services may be integrated depending on implementation scope.

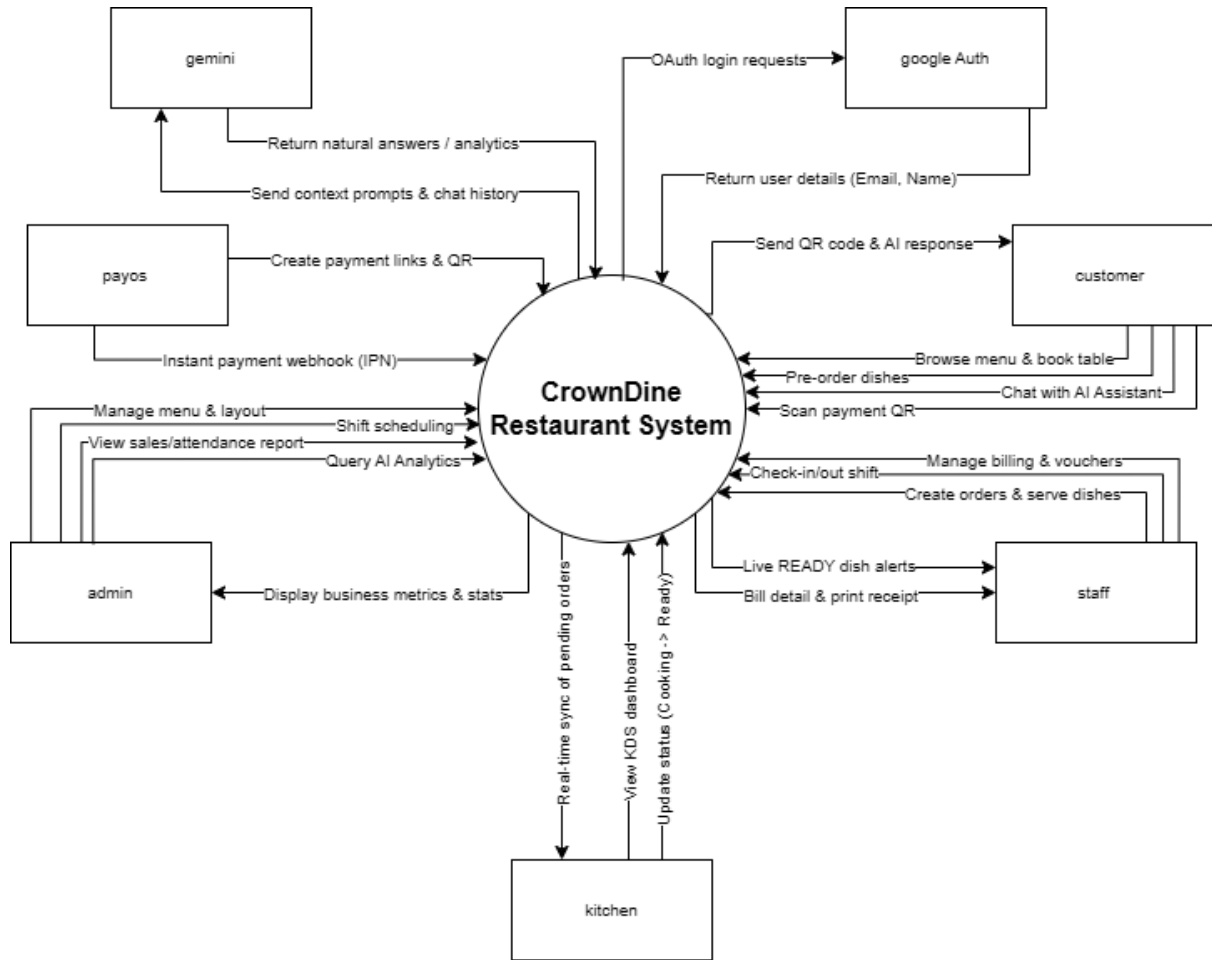


Figure 1: Crowndine System Context Diagram

I.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes and operating constraints that the Crowndine Restaurant Management System must satisfy. These requirements ensure that the system is not only functionally correct but also practical, secure, reliable, and maintainable in a real restaurant environment.

- Usability:** The system shall provide an intuitive, seamless experience for all roles. For customers, the 4-step booking journey, interactive Table Map, and vibrant digital menu must be highly accessible. For staff and kitchen personnel, the Calendar View and KDS interfaces must be visually clear to manage real-time operations without requiring extensive technical training.
- Reliability:** CrownDine must guarantee absolute accuracy in preserving reservations, order batching sequences, and financial records processed via PayOS (QR Code and Bank Transfer). The system must consistently maintain state synchronization across all modules and reliably execute the automated cancellation and 80% refund policy without any data loss or manual intervention.
- Performance:** The platform shall process core actions—such as interactive table selection, cart updates, and payment authorizations—with minimal latency. The Kitchen Display System (KDS) must efficiently utilize WebSockets to ensure instant, real-time status

updates under heavy loads, while the integrated Gemini AI chatbot must deliver prompt analytical insights.

- **Security:** All user data and transactions shall be strictly protected. The system must implement robust authentication utilizing JWT and Google OAuth2. Furthermore, a granular role-based access control (RBAC) mechanism must precisely isolate permissions among Admin, Staff, Kitchen, and Customer roles to prevent unauthorized system modifications.
- **Maintainability:** The software architecture shall be highly modular, enabling developers to seamlessly implement updates to the WebSockets infrastructure, PayOS integrations, and AI features without system downtime. This ensures easy future expansion and allows administrators to flexibly configure business rules via the admin panel rather than deep code changes.

I.5 Functional Requirements

This section defines the functional requirements of the CrownDine Restaurant Management System. These requirements specify the detailed capabilities that the system must provide for each user role, structured by functional areas.

I.5.1 Customer Portal & Management

- **FR-C-01 Authentication & Account Management**
 - The system shall allow users to register a customer account and verify it using a 6-digit OTP sent via email.
 - The system shall allow customers to log in using their username and password, or via Google OAuth2.
 - The system shall allow customers to securely log out, update their account passwords, and refresh login tokens.
 - The system shall allow customers to manage profile details, including name, phone, and address; request email changes verified via OTP; and upload profile avatars hosted on Cloudinary.
- **FR-C-02 Loyalty Program & Vouchers**
 - The system shall track customer loyalty points accumulated from completed dine-in or reservation orders.
 - The system shall allow customers to exchange loyalty points for discount vouchers.
- **FR-C-03 Viewing Menu & Products**
 - The system shall display menu items, drinks, and combo packages grouped by categories.
 - The system shall display detailed information for menu items, including ingredients, descriptions, prices, images, and customer ratings or reviews.
 - The system shall allow customers to search for items or combos by name and filter by category.
 - The system shall allow customers to add items to, or remove items from, their favorites list.

- **FR-C-04 Smart Table Reservation**

- The system shall guide customers through a step-by-step reservation wizard for selecting date, time slot, and guest count.
- The system shall display a 2D interactive floor plan layout showing table locations, status, shapes, and seat capacities.
- The system shall allow customers to select preferred tables and hold-lock them for 15 minutes during checkout.

- **FR-C-05 Pre-order Cart & Payment**

- The system shall allow customers to add food items or combos to their booking pre-order cart and modify quantities.
- The system shall calculate the required deposit and allow customers to apply discount vouchers.
- The system shall integrate with the PayOS payment gateway to generate checkout links or bank transfer QR codes.
- The system shall handle payment webhook notifications to update reservation status to **CONFIRMED**.
- The system shall allow customers to view booking history and cancel upcoming bookings subject to policy.

- **FR-C-06 Customer AI Assistant**

- The system shall provide a real-time Gemini AI chatbot for customers to ask questions about menu items, opening hours, or table availability.
- The chatbot shall parse relative time expressions into calendar dates and recommend tables.
- The chatbot shall generate quick navigation links to pre-fill reservation details based on the chat context.

I.5.2 Staff Portal & Restaurant Operations

- **FR-S-01 Reservation Processing**

- The system shall allow staff to view and filter customer reservations by date range and status, including Pending, Confirmed, Checked-in, Completed, and Cancelled.
- The system shall allow staff to check in customers, reassign tables for active reservations, and cancel bookings with cancellation reasons.

- **FR-S-02 Walk-in & Dine-in Orders**

- The system shall allow staff to create dine-in orders directly for tables.
- The system shall allow staff to associate dine-in orders with customer profiles via phone number for loyalty point accumulation.
- The system shall allow staff to append dishes, adjust quantities, apply vouchers, and split bills or merge tables.
- The system shall generate PayOS QR codes for cashier checkout, print physical invoice receipts, and free up tables in the database.

- **FR-S-03 Kitchen Display System (KDS)**

- The system shall provide a real-time KDS interface for kitchen staff to view pending and preparation order queues.
- The system shall allow kitchen staff to group identical items into batch preparations.
- The system shall synchronize preparation statuses from Pending to Preparing to Ready in real time using WebSockets.
- The system shall send instant toast notifications to floor staff when a dish is marked as Ready.

- **FR-S-04 Shift & Attendance**

- The system shall allow staff to view weekly work schedules and personal attendance logs.
- The system shall allow staff to clock in and clock out to record actual work hours.

I.5.3 Administrative Portal

- **FR-A-01 Restaurant Layout Design**

- The system shall allow administrators to create, update, and delete floors, areas, and tables.
- The system shall allow administrators to configure table layout details, including names, 2D coordinates, dimensions, shape, and capacity.

- **FR-A-02 Menu Management**

- The system shall allow administrators to manage menu categories, food items, and combo packages.
- The system shall support uploading menu item images to Cloudinary, adjusting prices, and editing description details.

- **FR-A-03 Vouchers & Promotions**

- The system shall allow administrators to manage discount vouchers, including types, rates, minimum values, expiry dates, and usage limits.
- The system shall allow administrators to manually assign vouchers to specific users and configure point exchange rates.

- **FR-A-04 Staff & Roles Management**

- The system shall allow administrators to create staff accounts, manage roles, and toggle account statuses as Active or Blocked.

- **FR-A-05 Employee Shift Scheduling & Auditing**

- The system shall allow administrators to configure shift templates, such as Morning, Afternoon, and Evening, and assign weekly schedules to staff.
- The system shall allow administrators to review weekly attendance logs, audit clock-in statuses, and view monthly payroll or attendance summaries.

- **FR-A-06 Analytics & AI Assistant**

- The system shall provide an interactive sales dashboard displaying revenue graphs by hour or day and growth trends.
- The system shall support exporting detailed sales reports to Excel spreadsheets.
- The system shall provide a Gemini AI chat interface for administrators to query revenue metrics and top-selling combos using Tool Calling integration.

I.5.4 Use Case Diagrams

Based on the described user requirements and detailed analysis, the CrownDine Restaurant Management System is deployed on a web platform with role-based access for authentication, customers, staff, and administrators. The following use case diagrams present the main interactions between each actor group and the system, as well as selected detailed use cases for customer workflows.

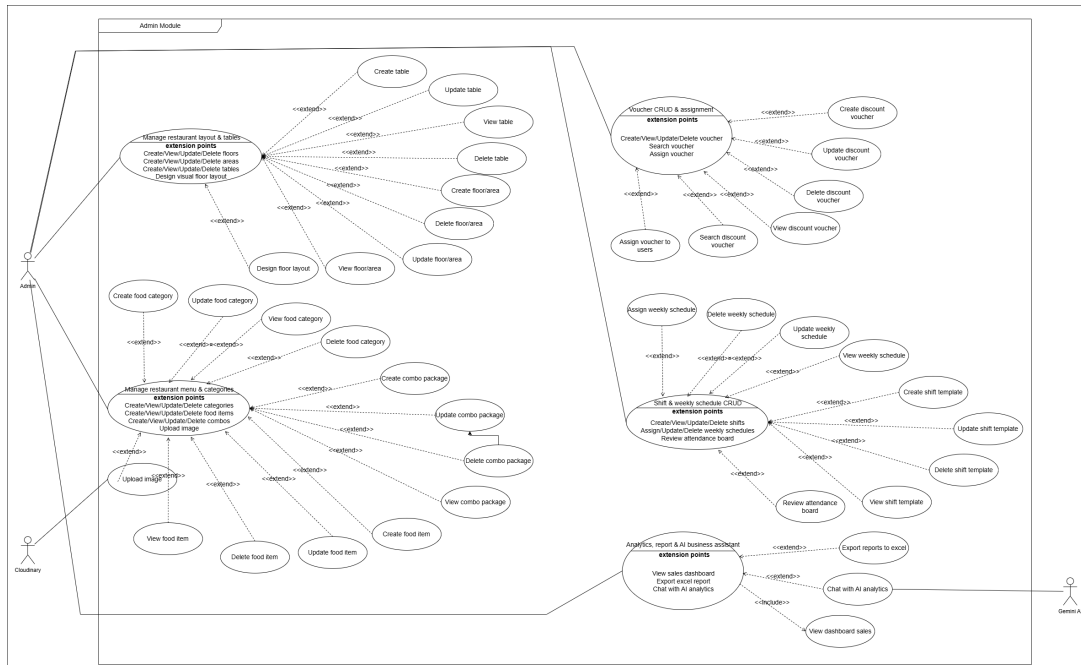


Figure 2: Overview Use Case Diagram for Authentication

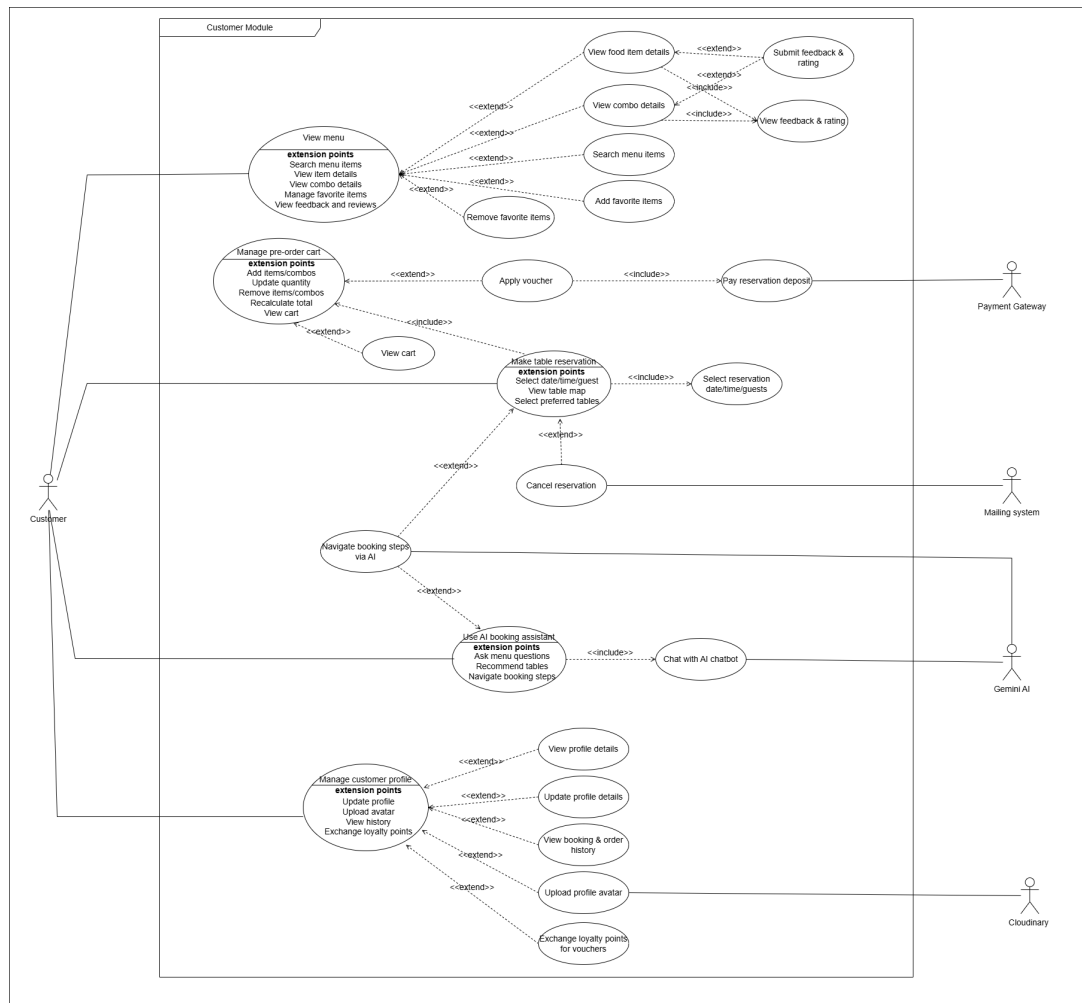


Figure 3: Overview Use Case Diagram for Customer Site

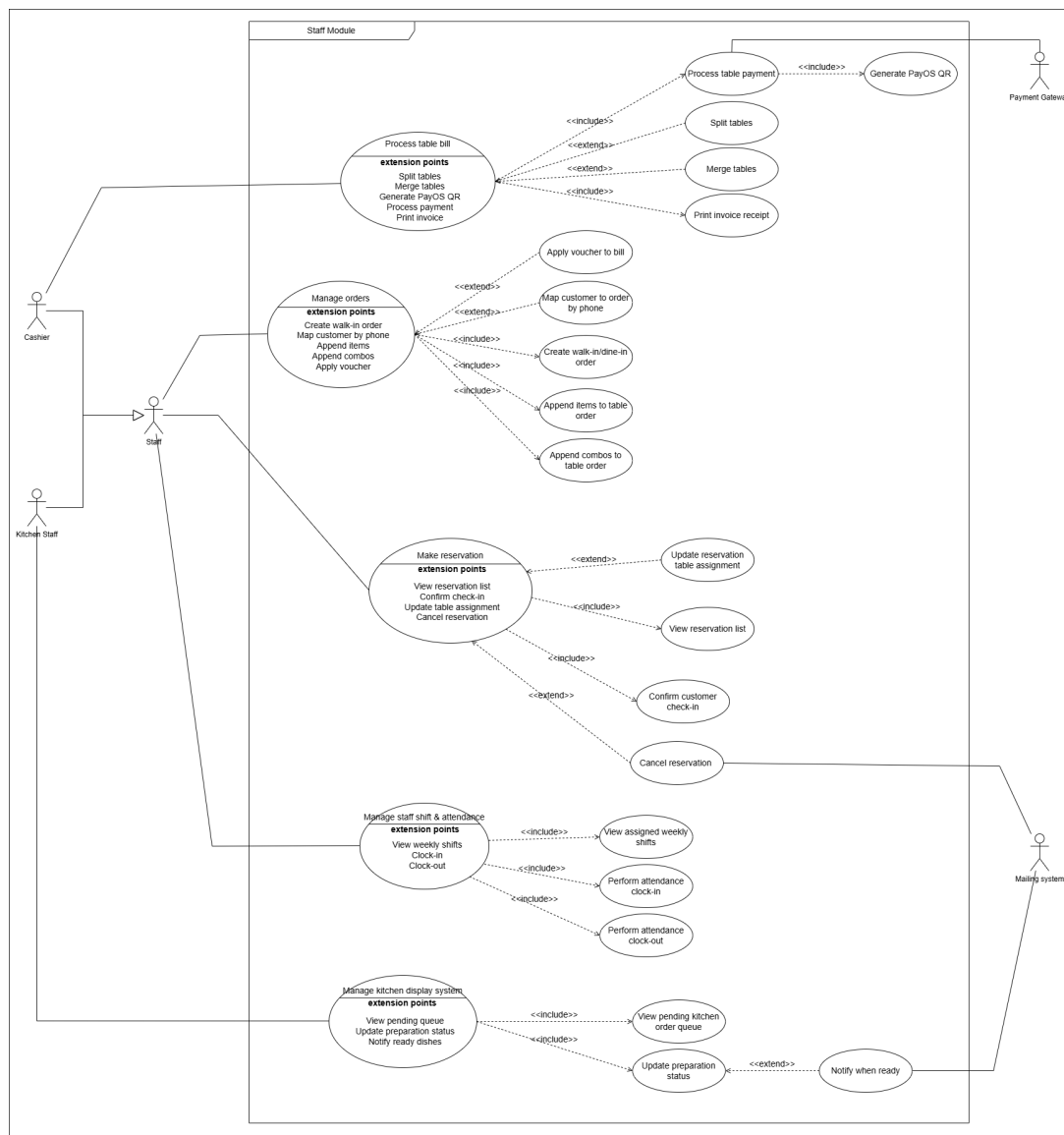


Figure 4: Overview Use Case Diagram for Staff Site

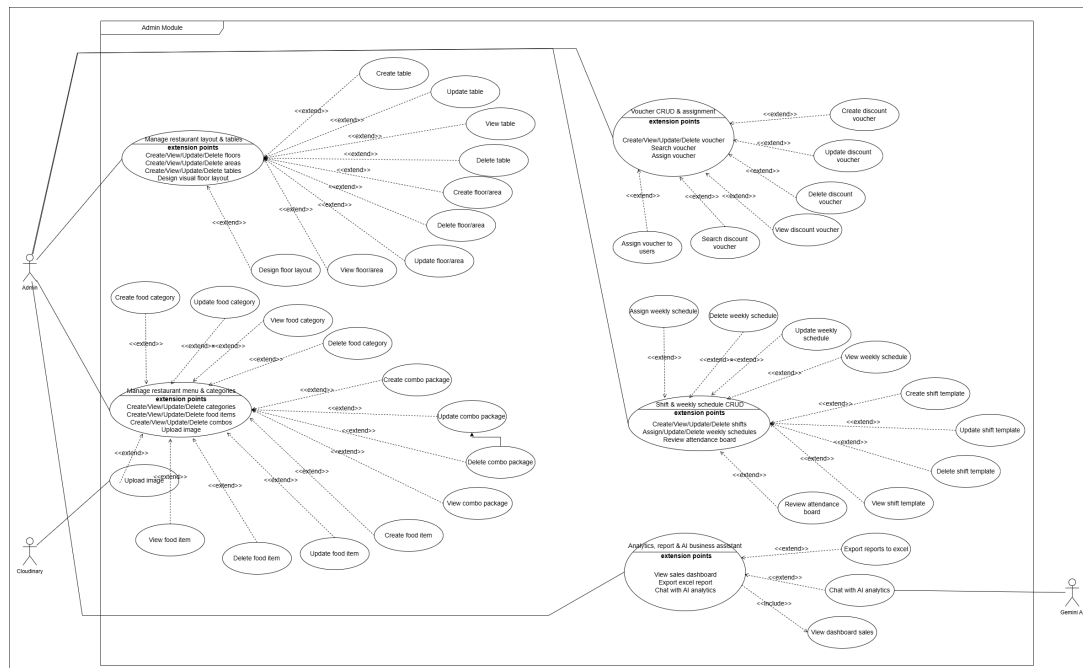


Figure 5: Overview Use Case Diagram for Administration Site

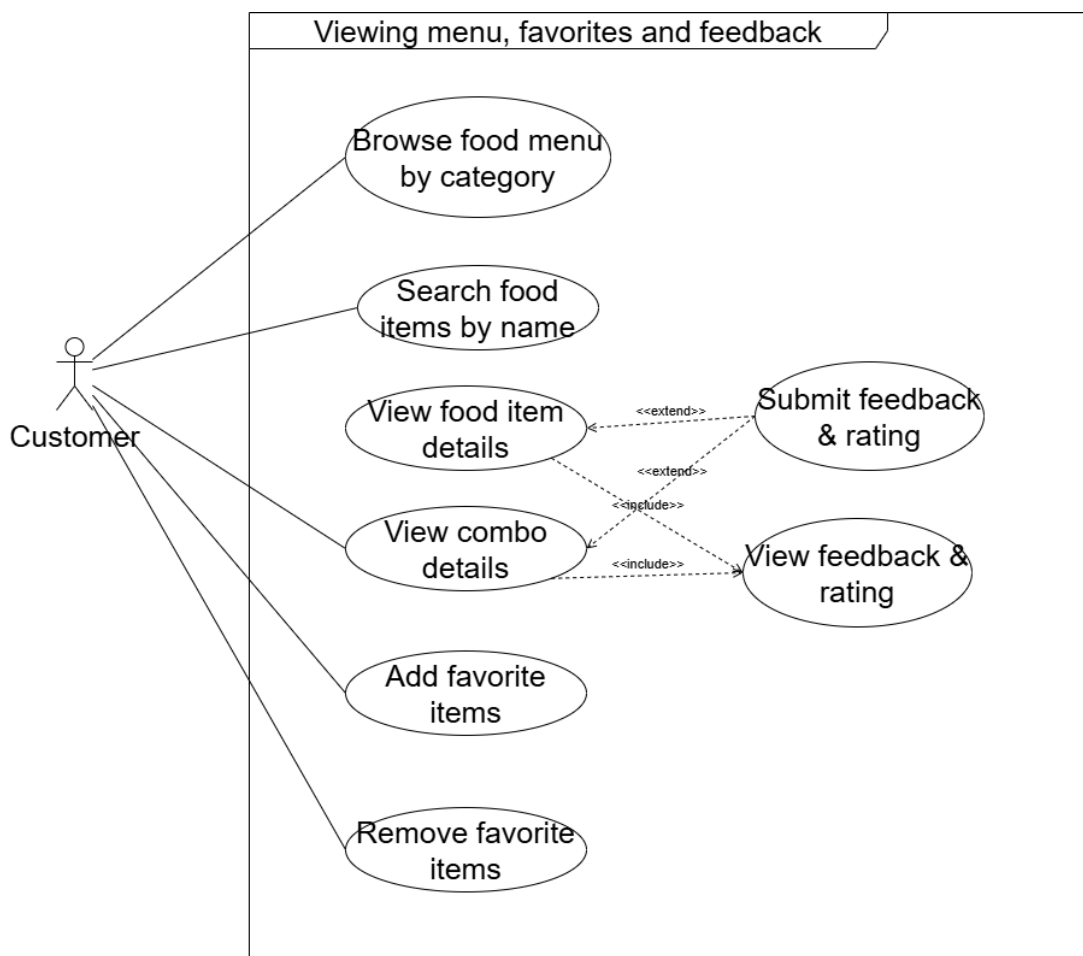


Figure 6: Detailed Use Case Diagram for Viewing Menu

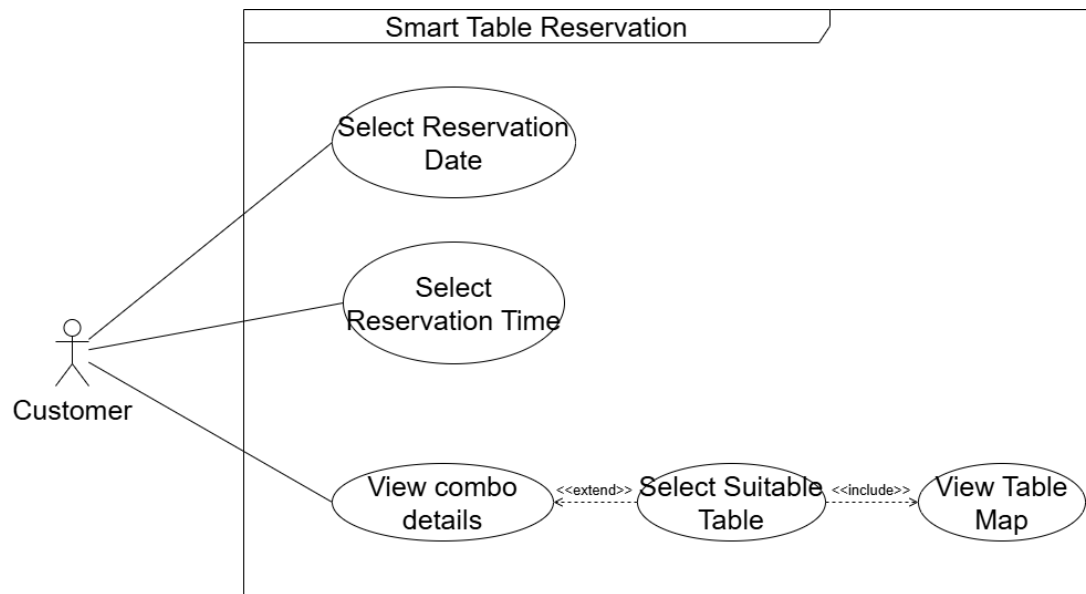


Figure 7: Detailed Use Case Diagram for Table Reservation

I.5.5 Use Case Descriptions

This section specifies the detailed use cases of the CrownDine Intelligent Restaurant Management System. The use cases are grouped by customer, staff, kitchen, cashier, and admin workflows.

UC-01: Browse food menu by categories

ID and Name	UC-01 Browse food menu by categories
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customers to view the electronic menu grouped by categories such as Starters, Main Course, and Drinks.
Trigger	Customer selects the Menu section on the website.
Preconditions	None.
Postconditions	POST-1. Menu items are displayed correctly according to the chosen category.
Normal Flow	1. Customer navigates to the Menu page. 2. System retrieves active categories and corresponding food items from the database. 3. System displays category list and items under the default category. 4. Customer clicks different categories to filter the food list. 5. System refreshes menu items according to the selected category.
Alternative Flows	None.
Exceptions	None.
Priority	High
Business Rules	Only active categories and items are displayed to customers.

UC-02: View food item and combo details

ID and Name	UC-02 View food item and combo details
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customers to view detailed information of a dish or combo, including description, ingredients, price, image, and reviews.
Trigger	Customer clicks on a food item or combo card.
Preconditions	None.
Postconditions	POST-1. Detailed item/combo page is displayed.
Normal Flow	1. Customer selects a specific item or combo from the menu. 2. System queries detailed attributes of the selected item or combo sub-items. 3. System displays photos, details, price, and customer reviews.
Alternative Flows	None.
Exceptions	Item not found: system shows an error message and redirects back to Menu page.
Priority	High
Business Rules	Pricing shown must match the active base price configured by Admin.

UC-03: Search menu items by name

ID and Name	UC-03 Search menu items by name
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customers to search for food items or combos by entering keywords or names.
Trigger	Customer types in the search bar on the Menu page.
Preconditions	None.
Postconditions	POST-1. Menu list is filtered by the search query.
Normal Flow	1. Customer enters keywords into the search bar. 2. System queries database for items matching the search query using case-insensitive substring match. 3. System displays matching items or combos to the user.
Alternative Flows	None.
Exceptions	No matches found: system displays 'No items found matching your search'.
Priority	Medium
Business Rules	Search query must be sanitized to prevent SQL injection. Inactive menu items are excluded from search results.

UC-04: Manage favorite items

ID and Name	UC-04 Manage favorite items
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows logged-in customers to save food items to or remove them from their favorites list.
Trigger	Customer clicks the favorite heart icon on a food item card.
Preconditions	PRE-1. Customer is logged into the system.
Postconditions	POST-1. The favorite status of the item is updated in the database.
Normal Flow	1. Customer clicks the heart icon of an item. 2. System checks current status: if not in favorites, it adds item ID; if already in favorites, it removes it. 3. System updates the heart icon styling in UI.
Alternative Flows	None.
Exceptions	Customer is not logged in: system prompts customer to log in.
Priority	Low
Business Rules	Favorites are linked to customer account and persistent across sessions. Maximum of 50 favorite items per user.

UC-05: Submit feedback and rating on items

ID and Name	UC-05 Submit feedback and rating on items
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customers to submit reviews and star ratings on specific menu items they have ordered.
Trigger	Customer clicks Write Feedback on an item in their history.
Preconditions	PRE-1. Customer is logged in. PRE-2. Customer has ordered the item in the past.
Postconditions	POST-1. Feedback is saved and average rating is updated.
Normal Flow	1. Customer enters rating from 1 to 5 stars and review text, then clicks Submit. 2. System validates content and saves feedback in the database. 3. System recalculates average rating score of the item.
Alternative Flows	Customer can edit or update their existing feedback for the item.
Exceptions	None.
Priority	Medium
Business Rules	Customer must have ordered the item to be allowed to submit a review.

UC-06: View item feedbacks and reviews

ID and Name	UC-06 View item feedbacks and reviews
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customers to view reviews and ratings left by other diners on specific menu items.
Trigger	Customer opens the reviews section on a food detail page.
Preconditions	None.
Postconditions	POST-1. List of feedbacks, ratings, and comments are displayed.
Normal Flow	1. Customer views detail of an item. 2. System retrieves reviews, ratings, and comments associated with the item ID. 3. System displays review statistics and detailed review list.
Alternative Flows	None.
Exceptions	None.
Priority	Medium
Business Rules	Only approved or active reviews are displayed publicly. Reviews are sorted chronologically by default.

UC-07: Select reservation date, time, and guests

ID and Name	UC-07 Select reservation date, time, and guests
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Customer specifies date, time, and headcount to search for available seating slots.
Trigger	Customer clicks Book Table button.
Preconditions	PRE-1. Customer is logged in.
Postconditions	POST-1. Seating slot checking criteria is set.
Normal Flow	1. Customer opens Reservation Wizard. 2. Customer selects Date, Time Slot, and number of guests. 3. System checks opening hours from 09:00 to 22:00 and ensures date is not in the past. 4. System queries tables database to check availability.
Alternative Flows	None.
Exceptions	Selected date/time is in the past or outside business hours: system shows validation error.
Priority	High
Business Rules	Booking must be made within business hours from 09:00 to 22:00. Bookings cannot be made in the past.

UC-08: View table map and select preferred tables

ID and Name	UC-08 View table map and select preferred tables
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customer to view a 2D floor plan layout and select preferred tables to book.
Trigger	Customer advances to step 2 of booking.
Preconditions	PRE-1. Customer selected valid booking date, time, and guest count.
Postconditions	POST-1. Tables are selected and added to booking selection.
Normal Flow	1. System loads restaurant floor layouts. 2. System queries reservations matching selected date and time slot. 3. System renders table shapes, labels, and availability colors. 4. Customer clicks a green table matching their preference. 5. System marks selected table and highlights it.
Alternative Flows	None.
Exceptions	Table capacity is too small: system suggests a larger table.
Priority	High
Business Rules	Real-time table status synchronization is used. Guests must not exceed table capacity plus allowed tolerance.

UC-09: Manage reservation pre-order cart

ID and Name	UC-09 Manage reservation pre-order cart
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customer to add, edit quantity, or remove pre-ordered dishes in reservation cart.
Trigger	Customer interacts with pre-order cart interface.
Preconditions	PRE-1. Customer has selected booking tables.
Postconditions	POST-1. Pre-order items in reservation cart are updated.
Normal Flow	1. Customer browses items and adds them to booking cart. 2. System verifies item availability and updates quantities in session cart. 3. System recalculates total cart price.
Alternative Flows	None.
Exceptions	Item out of stock: system alerts customer that item cannot be ordered.
Priority	Medium
Business Rules	Quantity must be positive. Pre-order items can only be edited or removed before booking deposit is paid.

UC-10: Apply discount voucher to reservation

ID and Name	UC-10 Apply discount voucher to reservation
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customer to input a voucher code at reservation checkout to get a discount.
Trigger	Customer inputs code and clicks Apply.
Preconditions	PRE-1. Customer has active reservation order at step 4.
Postconditions	POST-1. Discount is applied to booking subtotal and final pay price is updated.
Normal Flow	1. Customer inputs voucher code. 2. System validates code by expiry date, active status, user constraints, and minimum order limit. 3. System calculates percentage or fixed discount and subtracts it from total bill.
Alternative Flows	None.
Exceptions	Voucher invalid, expired, or used: system displays error message.
Priority	High
Business Rules	Only one voucher can be applied per reservation order. Total bill must meet minimum spend requirements.

UC-11: Pay reservation deposit via PayOS

ID and Name	UC-11 Pay reservation deposit via PayOS
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	PayOS Payment Gateway
Description	Customer pays a booking deposit through PayOS gateway integration.
Trigger	Customer clicks Pay Now button.
Preconditions	PRE-1. Reservation details and table selection are completed.
Postconditions	POST-1. Reservation is registered with PENDING_PAYMENT status and PayOS redirect URL is generated.
Normal Flow	1. Customer clicks payment button. 2. System creates reservation with PENDING_PAYMENT status. 3. System communicates with PayOS API to generate checkout URL and QR code. 4. System redirects customer to PayOS checkout page. 5. Customer completes payment. 6. PayOS sends webhook and backend marks reservation CONFIRMED. 7. System redirects customer back to CrownDine success page.
Alternative Flows	Customer cancels payment: PayOS redirects to failure URL and system updates reservation status to FAILED.
Exceptions	PayOS API timeout: system displays payment service unavailable message.
Priority	High
Business Rules	Reservation hold lock on selected tables is released if payment is not completed within 15 minutes.

UC-12: Cancel pending/upcoming reservation

ID and Name	UC-12 Cancel pending/upcoming reservation
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customers to cancel their upcoming reservation from profile history.
Trigger	Customer clicks Cancel booking on an active booking card.
Preconditions	PRE-1. Booking has CONFIRMED status.
Postconditions	POST-1. Booking status is updated to CANCELLED and tables are freed.
Normal Flow	1. Customer selects booking cancellation. 2. System checks cancellation policy. 3. System updates reservation status to CANCELLED. 4. System sends email alert notifying successful cancellation.
Alternative Flows	None.
Exceptions	Late cancellation less than 2 hours before booking: system denies cancellation and directs user to support.
Priority	High
Business Rules	Cancellations are only allowed up to 2 hours before scheduled time slot.

UC-13: Register customer account

ID and Name	UC-13 Register customer account
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows new customers to create a CrownDine account.
Trigger	Customer submits registration form.
Preconditions	None.
Postconditions	POST-1. Temporary inactive account is created and OTP email is sent.
Normal Flow	1. Customer enters username, email, phone, password, and name. 2. System checks if username or email already exists. 3. System creates inactive user, hashes password, generates OTP, and sends email.
Alternative Flows	None.
Exceptions	Username or email already exists: system displays registration error.
Priority	High
Business Rules	Passwords must meet minimum security requirements.

UC-14: Verify registration via email OTP

ID and Name	UC-14 Verify registration via email OTP
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customers to verify email address using 6-digit OTP.
Trigger	Customer inputs OTP and clicks Verify.
Preconditions	PRE-1. Account registration initiated.
Postconditions	POST-1. Account status is activated.
Normal Flow	1. Customer checks email and enters OTP. 2. System validates OTP and expiry. 3. System sets customer user status to ACTIVE.
Alternative Flows	None.
Exceptions	Incorrect or expired OTP: system displays validation failure message.
Priority	High
Business Rules	OTP code is valid for exactly 5 minutes before expiring.

UC-15: Log in to the system

ID and Name	UC-15 Log in to the system
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer, Staff, Admin
Secondary Actors	Google Auth Service
Description	Allows registered users to log in to access their dashboards.
Trigger	User submits login credentials or clicks Google Login.
Preconditions	PRE-1. Account must be active.
Postconditions	POST-1. JWT access token is returned and user is redirected based on role.
Normal Flow	1. User enters credentials or uses Google Sign-In. 2. System validates user record in database. 3. System generates JWT tokens and maps authorities. 4. System redirects Admin to admin dashboard, Staff to staff dashboard, and Customer to Home.
Alternative Flows	None.
Exceptions	Invalid credentials or blocked account: login request is denied.
Priority	High
Business Rules	Access tokens expire in 1 hour; refresh tokens expire in 7 days. Failed logins lock account temporarily after 5 attempts.

UC-16: Log out of the system

ID and Name	UC-16 Log out of the system
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer, Staff, Admin
Secondary Actors	None
Description	Logs the user out and clears session tokens.
Trigger	User clicks Log Out button.
Preconditions	PRE-1. User is logged in.
Postconditions	POST-1. Refresh token is deleted and client storage is cleared.
Normal Flow	1. User clicks Log Out. 2. System deletes JWT refresh token record. 3. Client clears storage and redirects to Login or Home page.
Alternative Flows	None.
Exceptions	None.
Priority	High
Business Rules	Logged-out tokens are blacklisted to prevent reuse before expiration.

UC-17: View and update customer profile details

ID and Name	UC-17 View and update customer profile details
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customers to view and update personal name, phone number, and address.
Trigger	Customer edits profile form and clicks Save.
Preconditions	PRE-1. Customer is logged in.
Postconditions	POST-1. User details are updated in database.
Normal Flow	1. Customer opens profile. 2. Customer edits fields such as name and phone. 3. Customer clicks Save Changes. 4. System validates inputs and updates database record.
Alternative Flows	None.
Exceptions	Invalid phone format: system shows validation warning.
Priority	Medium
Business Rules	Phone number must consist of valid digits. Users can edit profile only while logged in.

UC-18: Update email address with OTP confirmation

ID and Name	UC-18 Update email address with OTP confirmation
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows customers to update email address by verifying it with OTP.
Trigger	Customer initiates email change in profile settings.
Preconditions	PRE-1. Customer is logged in.
Postconditions	POST-1. User email is changed after OTP verification.
Normal Flow	1. Customer inputs new email and clicks Send OTP. 2. System sends verification email with OTP. 3. Customer inputs received OTP and clicks Verify. 4. System validates OTP and updates email in database.
Alternative Flows	None.
Exceptions	Incorrect OTP: system rejects update and prompts user to check code.
Priority	Medium
Business Rules	New email must not already exist. OTP is single-use and invalid after verification attempt.

UC-19: Upload profile avatar

ID and Name	UC-19 Upload profile avatar
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	Cloudinary Service
Description	Allows customers to upload an image file as profile picture.
Trigger	Customer selects avatar photo and clicks Upload.
Preconditions	PRE-1. Customer is logged in.
Postconditions	POST-1. Avatar is uploaded and URL is updated in user record.
Normal Flow	1. Customer clicks Change Avatar and selects image file. 2. System uploads file to Cloudinary. 3. Cloudinary returns image URL. 4. System updates avatar URL in user record. 5. System displays new avatar in UI.
Alternative Flows	None.
Exceptions	File too large or invalid format: system shows warning.
Priority	Low
Business Rules	Supported formats: JPG, PNG, GIF. Max file size: 2MB.

UC-20: Change account password

ID and Name	UC-20 Change account password
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer, Staff, Admin
Secondary Actors	None
Description	Allows logged-in users to update account passwords.
Trigger	User submits Change Password form.
Preconditions	PRE-1. User is logged in.
Postconditions	POST-1. Password is updated in database.
Normal Flow	1. User enters current password, new password, and confirmation. 2. System validates current password. 3. System hashes new password and updates database record.
Alternative Flows	None.
Exceptions	Incorrect current password: password update is rejected.
Priority	Medium
Business Rules	New password must differ from current password and must be hashed using BCrypt.

UC-21: View booking and order history

ID and Name	UC-21 View booking and order history
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows Customer to perform the CrownDine workflow: View booking and order history.
Trigger	Customer selects or submits the View booking and order history action.
Preconditions	PRE-1. Customer is logged in and authorized.
Postconditions	POST-1. View booking and order history workflow is completed and system state is updated.
Normal Flow	1. Customer opens the View booking and order history function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-22: Exchange loyalty points for vouchers

ID and Name	UC-22 Exchange loyalty points for vouchers
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows Customer to perform the CrownDine workflow: Exchange loyalty points for vouchers.
Trigger	Customer selects or submits the Exchange loyalty points for vouchers action.
Preconditions	PRE-1. Customer is logged in and authorized.
Postconditions	POST-1. Exchange loyalty points for vouchers workflow is completed and system state is updated.
Normal Flow	1. Customer opens the Exchange loyalty points for vouchers function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-23: Chat with AI booking chatbot

ID and Name	UC-23 Chat with AI booking chatbot
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	Gemini AI API
Description	Allows Customer to perform the CrownDine workflow: Chat with AI booking chatbot.
Trigger	Customer selects or submits the Chat with AI booking chatbot action.
Preconditions	PRE-1. Customer is logged in and authorized.
Postconditions	POST-1. Chat with AI booking chatbot workflow is completed and system state is updated.
Normal Flow	1. Customer opens the Chat with AI booking chatbot function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-24: Navigate booking steps via AI links

ID and Name	UC-24 Navigate booking steps via AI links
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Customer
Secondary Actors	None
Description	Allows Customer to perform the CrownDine workflow: Navigate booking steps via AI links.
Trigger	Customer selects or submits the Navigate booking steps via AI links action.
Preconditions	PRE-1. Customer is logged in and authorized.
Postconditions	POST-1. Navigate booking steps via AI links workflow is completed and system state is updated.
Normal Flow	1. Customer opens the Navigate booking steps via AI links function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-25: View reservation list

ID and Name	UC-25 View reservation list
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Staff
Secondary Actors	None
Description	Allows Staff to perform the CrownDine workflow: View reservation list.
Trigger	Staff selects or submits the View reservation list action.
Preconditions	PRE-1. Staff is logged in and authorized.
Postconditions	POST-1. View reservation list workflow is completed and system state is updated.
Normal Flow	1. Staff opens the View reservation list function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-26: Confirm customer check-in

ID and Name	UC-26 Confirm customer check-in
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Staff
Secondary Actors	None
Description	Allows Staff to perform the CrownDine workflow: Confirm customer check-in.
Trigger	Staff selects or submits the Confirm customer check-in action.
Preconditions	PRE-1. Staff is logged in and authorized.
Postconditions	POST-1. Confirm customer check-in workflow is completed and system state is updated.
Normal Flow	1. Staff opens the Confirm customer check-in function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-27: Update reservation table assignment

ID and Name	UC-27 Update reservation table assignment
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Staff
Secondary Actors	None
Description	Allows Staff to perform the CrownDine workflow: Update reservation table assignment.
Trigger	Staff selects or submits the Update reservation table assignment action.
Preconditions	PRE-1. Staff is logged in and authorized.
Postconditions	POST-1. Update reservation table assignment workflow is completed and system state is updated.
Normal Flow	1. Staff opens the Update reservation table assignment function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-28: Cancel reservation by staff

ID and Name	UC-28 Cancel reservation by staff
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Staff
Secondary Actors	None
Description	Allows Staff to perform the CrownDine workflow: Cancel reservation by staff.
Trigger	Staff selects or submits the Cancel reservation by staff action.
Preconditions	PRE-1. Staff is logged in and authorized.
Postconditions	POST-1. Cancel reservation by staff workflow is completed and system state is updated.
Normal Flow	1. Staff opens the Cancel reservation by staff function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-29: Create walk-in dine-in order

ID and Name	UC-29 Create walk-in dine-in order
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Staff (Cashier)
Secondary Actors	None
Description	Allows Staff (Cashier) to perform the CrownDine workflow: Create walk-in dine-in order.
Trigger	Staff (Cashier) selects or submits the Create walk-in dine-in order action.
Preconditions	PRE-1. Staff (Cashier) is logged in and authorized.
Postconditions	POST-1. Create walk-in dine-in order workflow is completed and system state is updated.
Normal Flow	1. Staff (Cashier) opens the Create walk-in dine-in order function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-30: Map customer to order by phone number

ID and Name	UC-30 Map customer to order by phone number
Created By	Lê Trần Bảo Phúc
Date Created	09/06/2026
Primary Actor	Staff (Cashier)
Secondary Actors	None
Description	Allows Staff (Cashier) to perform the CrownDine workflow: Map customer to order by phone number.
Trigger	Staff (Cashier) selects or submits the Map customer to order by phone number action.
Preconditions	PRE-1. Staff (Cashier) is logged in and authorized.
Postconditions	POST-1. Map customer to order by phone number workflow is completed and system state is updated.
Normal Flow	1. Staff (Cashier) opens the Map customer to order by phone number function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-31: Append items/combo to table order

ID and Name	UC-31 Append items/combos to table order
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Staff (Cashier)
Secondary Actors	None
Description	Allows Staff (Cashier) to perform the CrownDine workflow: Append items/combos to table order.
Trigger	Staff (Cashier) selects or submits the Append items/combos to table order action.
Preconditions	PRE-1. Staff (Cashier) is logged in and authorized.
Postconditions	POST-1. Append items/combos to table order workflow is completed and system state is updated.
Normal Flow	1. Staff (Cashier) opens the Append items/combos to table order function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-32: Apply discount voucher to table bill

ID and Name	UC-32 Apply discount voucher to table bill
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Staff (Cashier)
Secondary Actors	None
Description	Allows Staff (Cashier) to perform the CrownDine workflow: Apply discount voucher to table bill.
Trigger	Staff (Cashier) selects or submits the Apply discount voucher to table bill action.
Preconditions	PRE-1. Staff (Cashier) is logged in and authorized.
Postconditions	POST-1. Apply discount voucher to table bill workflow is completed and system state is updated.
Normal Flow	1. Staff (Cashier) opens the Apply discount voucher to table bill function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-33: Process table payment via PayOS or Cash

ID and Name	UC-33 Process table payment via PayOS or Cash
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Staff (Cashier)
Secondary Actors	PayOS Gateway
Description	Allows Staff (Cashier) to perform the CrownDine workflow: Process table payment via PayOS or Cash.
Trigger	Staff (Cashier) selects or submits the Process table payment via PayOS or Cash action.
Preconditions	PRE-1. Staff (Cashier) is logged in and authorized.
Postconditions	POST-1. Process table payment via PayOS or Cash workflow is completed and system state is updated.
Normal Flow	1. Staff (Cashier) opens the Process table payment via PayOS or Cash function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-34: Split bill / Merge tables

ID and Name	UC-34 Split bill / Merge tables
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Staff (Cashier)
Secondary Actors	None
Description	Allows Staff (Cashier) to perform the CrownDine workflow: Split bill / Merge tables.
Trigger	Staff (Cashier) selects or submits the Split bill / Merge tables action.
Preconditions	PRE-1. Staff (Cashier) is logged in and authorized.
Postconditions	POST-1. Split bill / Merge tables workflow is completed and system state is updated.
Normal Flow	1. Staff (Cashier) opens the Split bill / Merge tables function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	Medium
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-35: Print invoice receipt

ID and Name	UC-35 Print invoice receipt
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Staff (Cashier)
Secondary Actors	None
Description	Allows Staff (Cashier) to perform the CrownDine workflow: Print invoice receipt.
Trigger	Staff (Cashier) selects or submits the Print invoice receipt action.
Preconditions	PRE-1. Staff (Cashier) is logged in and authorized.
Postconditions	POST-1. Print invoice receipt workflow is completed and system state is updated.
Normal Flow	1. Staff (Cashier) opens the Print invoice receipt function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-36: View pending kitchen order queue

ID and Name	UC-36 View pending kitchen order queue
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Kitchen Staff
Secondary Actors	None
Description	Allows Kitchen Staff to perform the CrownDine workflow: View pending kitchen order queue.
Trigger	Kitchen Staff selects or submits the View pending kitchen order queue action.
Preconditions	PRE-1. Kitchen Staff is logged in and authorized.
Postconditions	POST-1. View pending kitchen order queue workflow is completed and system state is updated.
Normal Flow	1. Kitchen Staff opens the View pending kitchen order queue function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-37: Group items into batches and Update preparation status

ID and Name	UC-37 Group items into batches and Update preparation status
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Kitchen Staff
Secondary Actors	None
Description	Allows Kitchen Staff to perform the CrownDine workflow: Group items into batches and Update preparation status.
Trigger	Kitchen Staff selects or submits the Group items into batches and Update preparation status action.
Preconditions	PRE-1. Kitchen Staff is logged in and authorized.
Postconditions	POST-1. Group items into batches and Update preparation status workflow is completed and system state is updated.
Normal Flow	1. Kitchen Staff opens the Group items into batches and Update preparation status function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-38: View assigned weekly shifts schedule

ID and Name	UC-38 View assigned weekly shifts schedule
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Staff
Secondary Actors	None
Description	Allows Staff to perform the CrownDine workflow: View assigned weekly shifts schedule.
Trigger	Staff selects or submits the View assigned weekly shifts schedule action.
Preconditions	PRE-1. Staff is logged in and authorized.
Postconditions	POST-1. View assigned weekly shifts schedule workflow is completed and system state is updated.
Normal Flow	1. Staff opens the View assigned weekly shifts schedule function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-39: Perform shifts attendance clock-in/out

ID and Name	UC-39 Perform shifts attendance clock-in/out
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Staff
Secondary Actors	None
Description	Allows Staff to perform the CrownDine workflow: Perform shifts attendance clock-in/out.
Trigger	Staff selects or submits the Perform shifts attendance clock-in/out action.
Preconditions	PRE-1. Staff is logged in and authorized.
Postconditions	POST-1. Perform shifts attendance clock-in/out workflow is completed and system state is updated.
Normal Flow	1. Staff opens the Perform shifts attendance clock-in/out function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-40: Manage floors and areas

ID and Name	UC-40 Manage floors and areas
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Manage floors and areas.
Trigger	Admin selects or submits the Manage floors and areas action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Manage floors and areas workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Manage floors and areas function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	Medium
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-41: Design floor layout and table configuration

ID and Name	UC-41 Design floor layout and table configuration
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Design floor layout and table configuration.
Trigger	Admin selects or submits the Design floor layout and table configuration action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Design floor layout and table configuration workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Design floor layout and table configuration function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-42: Manage food categories

ID and Name	UC-42 Manage food categories
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Manage food categories.
Trigger	Admin selects or submits the Manage food categories action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Manage food categories workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Manage food categories function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-43: Manage food items

ID and Name	UC-43 Manage food items
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	Cloudinary
Description	Allows Admin to perform the CrownDine workflow: Manage food items.
Trigger	Admin selects or submits the Manage food items action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Manage food items workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Manage food items function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-44: Manage combo packages

ID and Name	UC-44 Manage combo packages
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	Cloudinary
Description	Allows Admin to perform the CrownDine workflow: Manage combo packages.
Trigger	Admin selects or submits the Manage combo packages action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Manage combo packages workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Manage combo packages function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-45: Create discount vouchers

ID and Name	UC-45 Create discount vouchers
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Create discount vouchers.
Trigger	Admin selects or submits the Create discount vouchers action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Create discount vouchers workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Create discount vouchers function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-46: Edit/Delete discount vouchers

ID and Name	UC-46 Edit/Delete discount vouchers
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Edit/Delete discount vouchers.
Trigger	Admin selects or submits the Edit/Delete discount vouchers action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Edit/Delete discount vouchers workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Edit/Delete discount vouchers function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	Medium
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-47: Assign vouchers to users

ID and Name	UC-47 Assign vouchers to users
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Assign vouchers to users.
Trigger	Admin selects or submits the Assign vouchers to users action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Assign vouchers to users workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Assign vouchers to users function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	Medium
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-48: Create and manage staff accounts

ID and Name	UC-48 Create and manage staff accounts
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Create and manage staff accounts.
Trigger	Admin selects or submits the Create and manage staff accounts action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Create and manage staff accounts workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Create and manage staff accounts function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-49: Assign/revoke staff permissions and status

ID and Name	UC-49 Assign/revoke staff permissions and status
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Assign/revoke staff permissions and status.
Trigger	Admin selects or submits the Assign/revoke staff permissions and status action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Assign/revoke staff permissions and status workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Assign/revoke staff permissions and status function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-50: Configure shifts template

ID and Name	UC-50 Configure shifts template
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Configure shifts template.
Trigger	Admin selects or submits the Configure shifts template action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Configure shifts template workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Configure shifts template function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	Medium
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-51: Assign weekly work schedule to staff

ID and Name	UC-51 Assign weekly work schedule to staff
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Assign weekly work schedule to staff.
Trigger	Admin selects or submits the Assign weekly work schedule to staff action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Assign weekly work schedule to staff workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Assign weekly work schedule to staff function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-52: Review weekly attendance board

ID and Name	UC-52 Review weekly attendance board
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Review weekly attendance board.
Trigger	Admin selects or submits the Review weekly attendance board action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Review weekly attendance board workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Review weekly attendance board function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	Medium
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-53: View interactive dashboard sales graphs

ID and Name	UC-53 View interactive dashboard sales graphs
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: View interactive dashboard sales graphs.
Trigger	Admin selects or submits the View interactive dashboard sales graphs action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. View interactive dashboard sales graphs workflow is completed and system state is updated.
Normal Flow	1. Admin opens the View interactive dashboard sales graphs function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-54: Export reports to Excel

ID and Name	UC-54 Export reports to Excel
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	None
Description	Allows Admin to perform the CrownDine workflow: Export reports to Excel.
Trigger	Admin selects or submits the Export reports to Excel action.
Preconditions	PRE-1. Admin is logged in and authorized.
Postconditions	POST-1. Export reports to Excel workflow is completed and system state is updated.
Normal Flow	1. Admin opens the Export reports to Excel function. 2. System validates role, input data, and related business constraints. 3. System performs the requested workflow and updates database records when required. 4. System refreshes the interface and shows the result to the actor.
Alternative Flows	None.
Exceptions	Invalid data, unavailable records, or access denial: system displays an error message.
Priority	High
Business Rules	Only authorized actors can perform this use case. All changes are validated and persisted consistently.

UC-55: Chat with Gemini AI for sales metrics and business health reports

ID and Name	UC-55 Chat with Gemini AI for sales metrics and business health reports
Created By	Lê Trần Bảo Phúc
Date Created	10/06/2026
Primary Actor	Admin
Secondary Actors	Gemini AI API
Description	Allows Admin to interact with Gemini AI chatbot to query real-time revenue stats, top-selling combos, and business suggestions.
Trigger	Admin submits analytical queries to AI Chatbot.
Preconditions	PRE-1. Admin is logged in.
Postconditions	POST-1. Streaming text response from Gemini AI is displayed.
Normal Flow	1. Admin inputs analytical query about revenue and top-selling combos. 2. System routes query to AI Admin Chat Service. 3. AI detects and triggers secured tools such as getRevenueReport and getTopSellingCombos. 4. AI compiles analytical reply and streams response to admin chatbot window.
Alternative Flows	None.
Exceptions	Rate limits or API failure: system shows alert.
Priority	High
Business Rules	Tool calling functions are secured and only accessible if user role is ADMIN.

I.5.6 Activity Diagrams

This section presents the main activity diagrams of the Crowndine Restaurant Management System in the required order.

REGISTRATION & LOGIN FLOW (Authentication & Authorization)

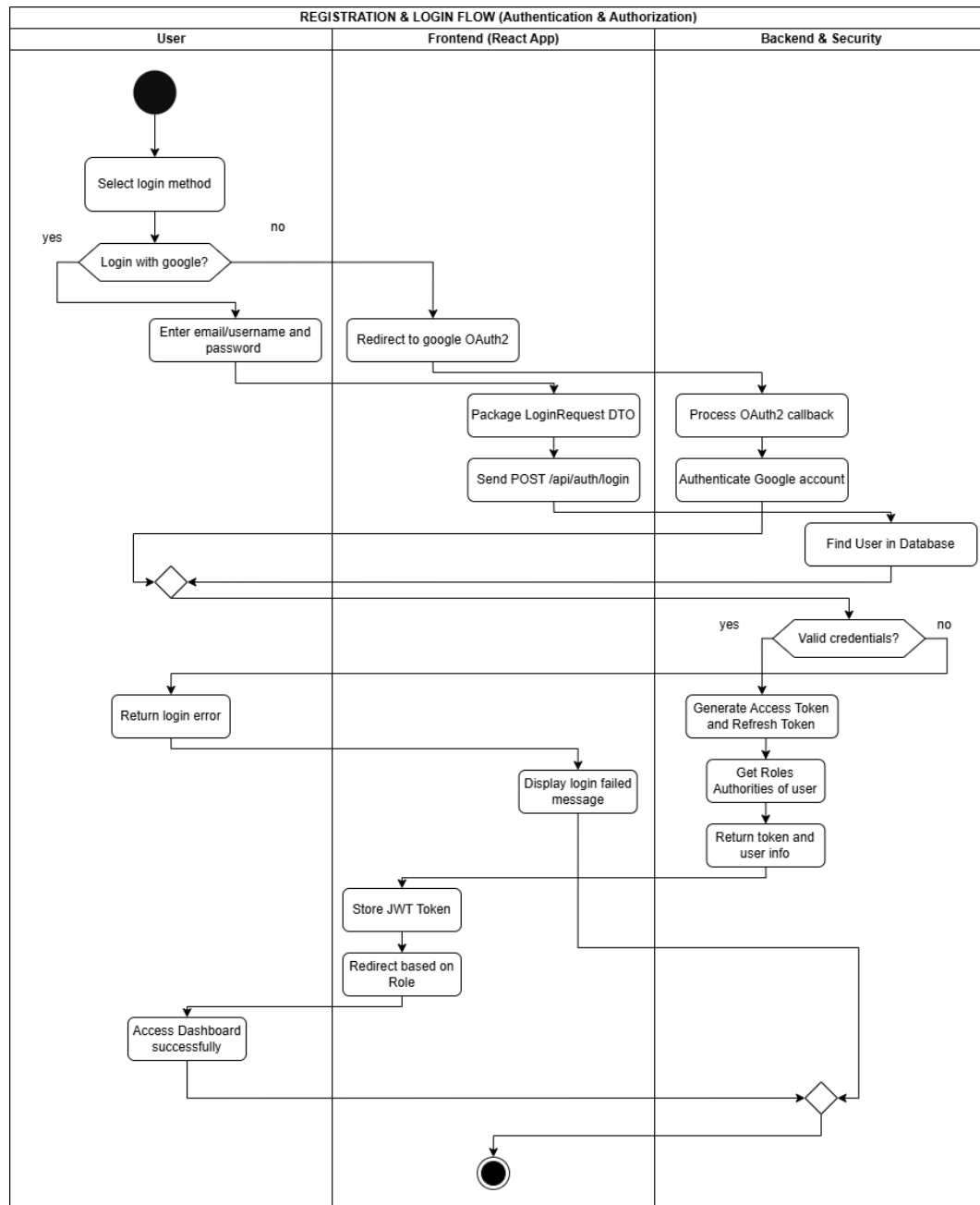


Figure 8: Registration and Login Flow

ONLINE RESERVATION & PAYOS PAYMENT FLOW (Online Reservation & PayOS Payment)

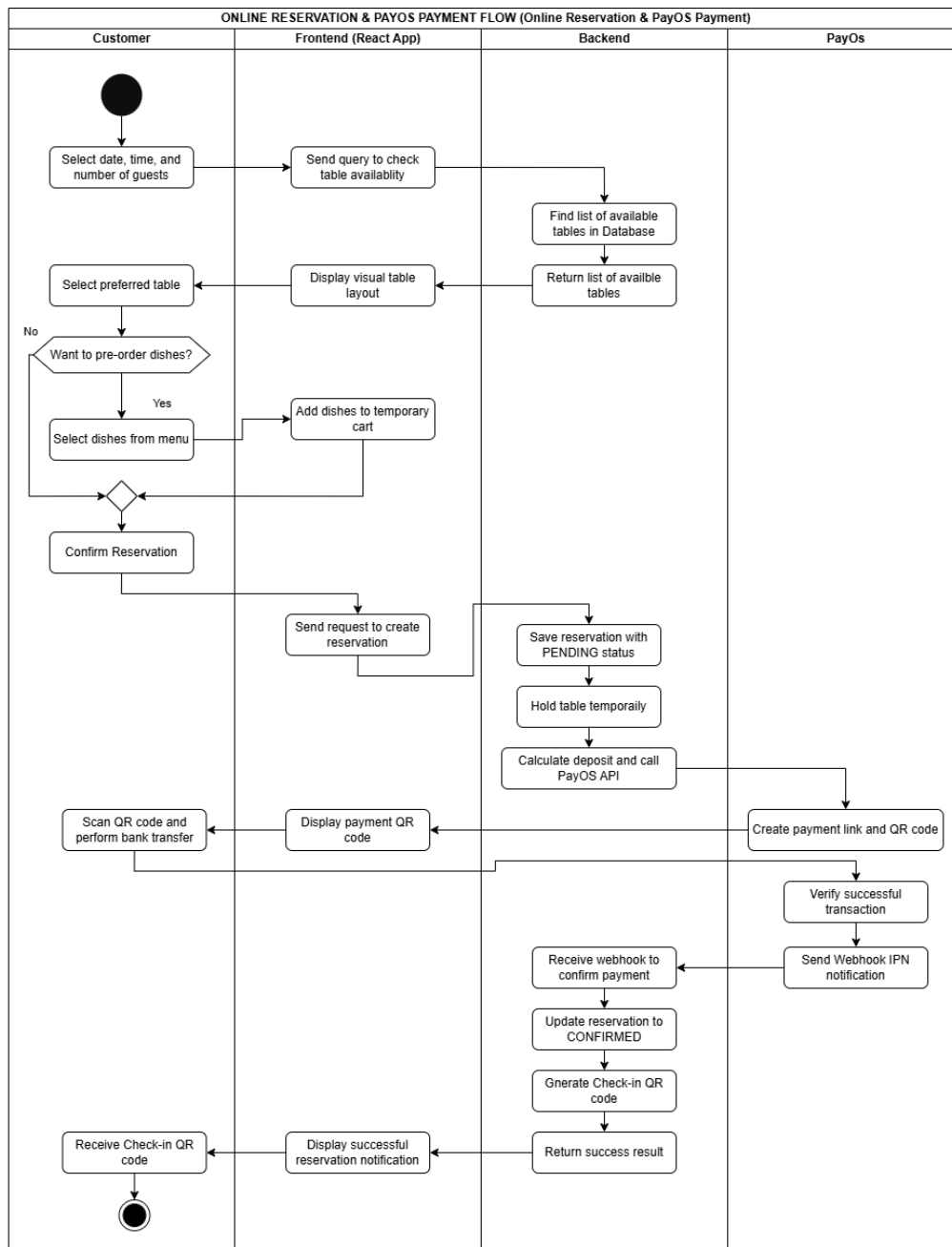


Figure 9: Online Reservation and PayOS Payment Flow

ORDER & REAL-TIME KITCHEN COORDINATION FLOW (Order & WebSocket KDS)

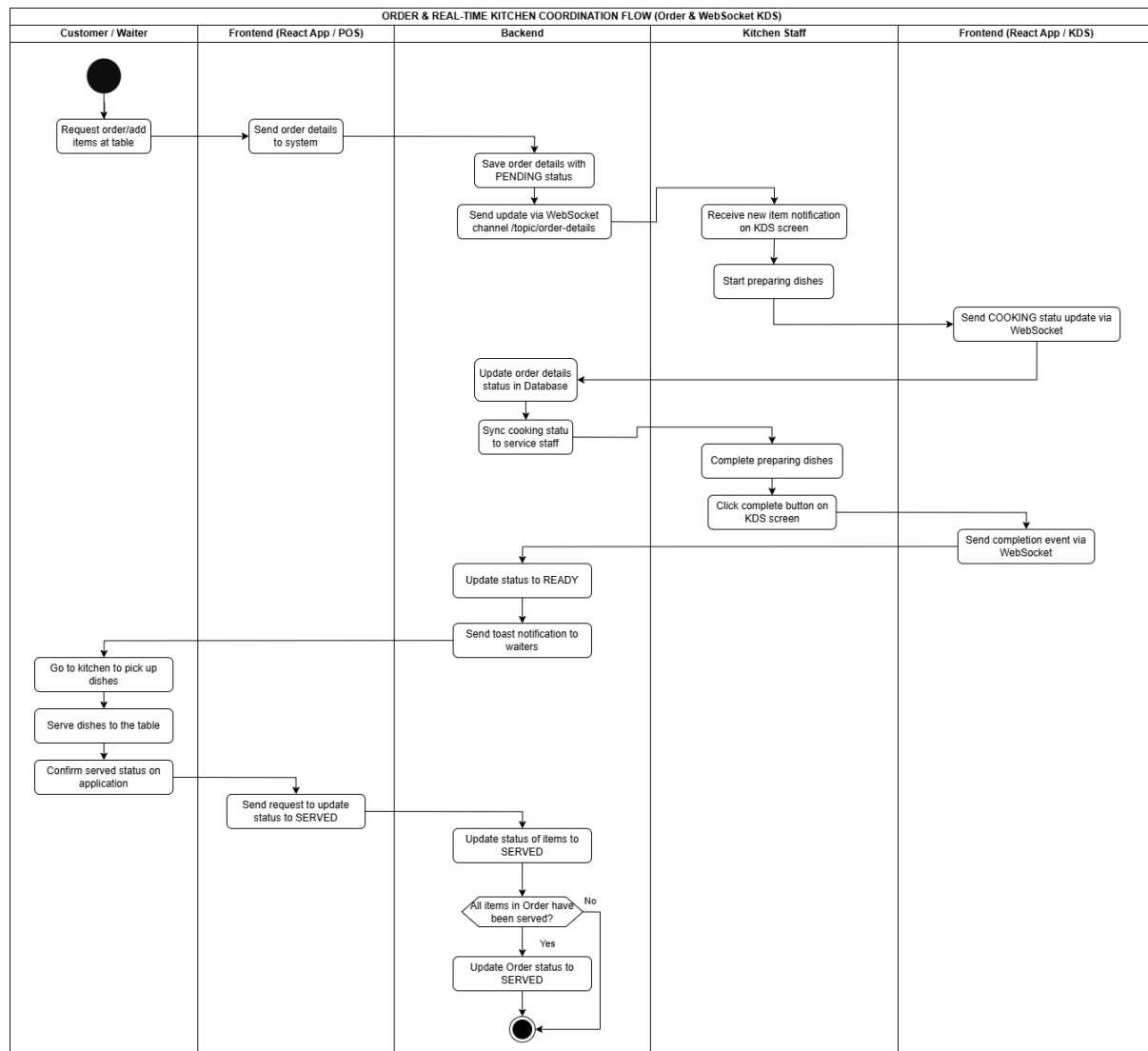


Figure 10: Order and Real-Time Kitchen Coordination Flow

BOOKING VIA AI CHATBOT ASSISTANT FLOW (AI Chatbot Booking Assistant)

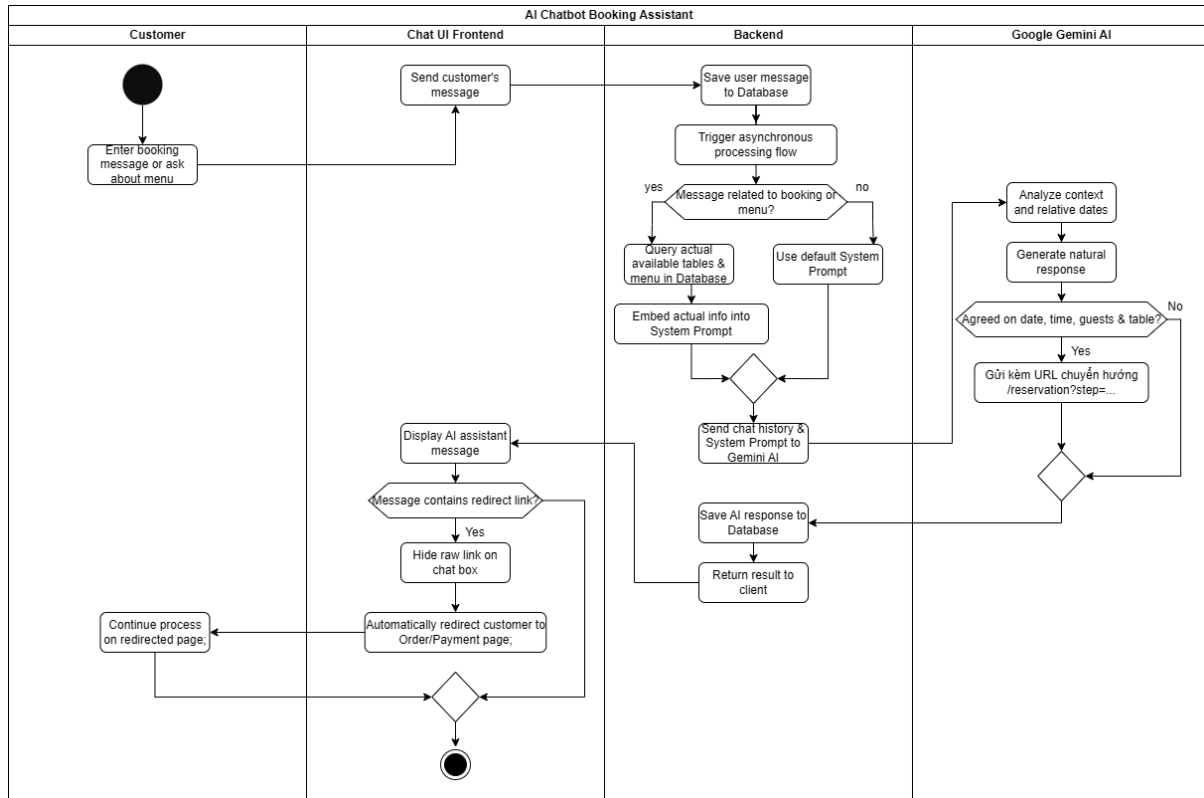


Figure 11: Booking via AI Chatbot Assistant Flow

VOUCHER APPLICATION & BILLING FLOW (Voucher Application & Billing)

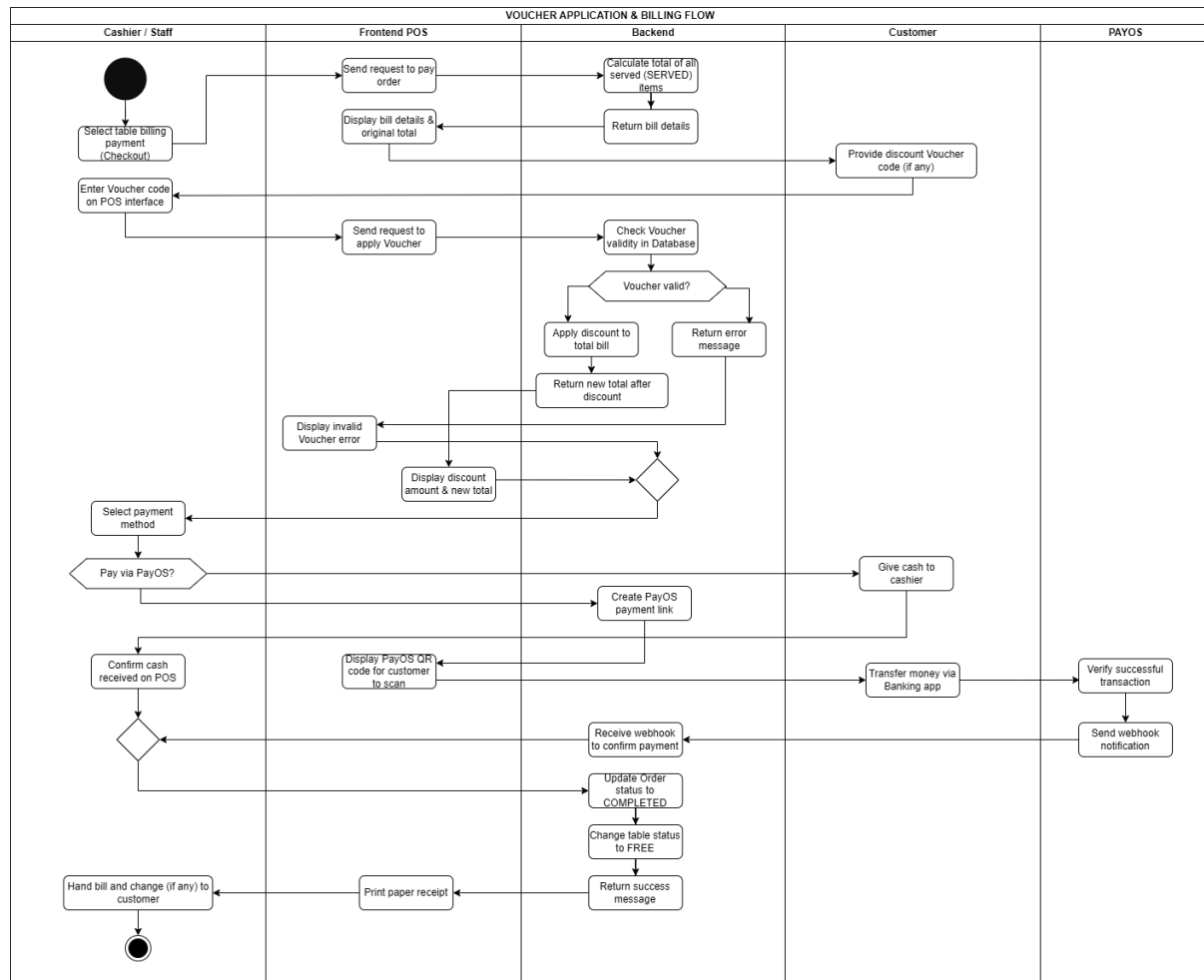


Figure 12: Voucher Application and Billing Flow

STAFF SHIFT & ATTENDANCE FLOW (Staff Shift & Attendance)

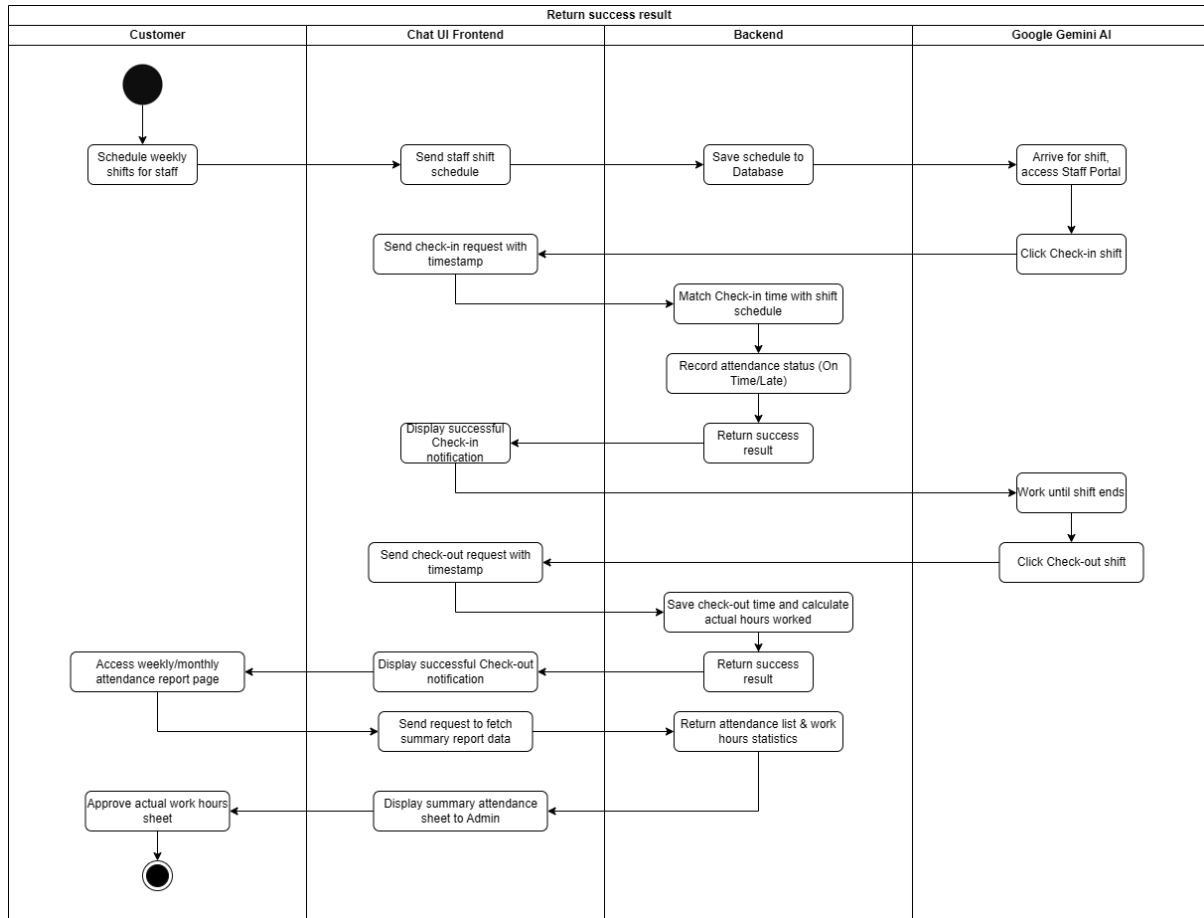


Figure 13: Staff Shift and Attendance Flow

II Analysis Models

II.1 Interaction Diagrams

This section presents the main sequence diagrams for core CrownDine workflows. The diagrams are organized according to their corresponding use cases.

Customer Account Registration & Verification via Email OTP (UC-13 & UC-14)

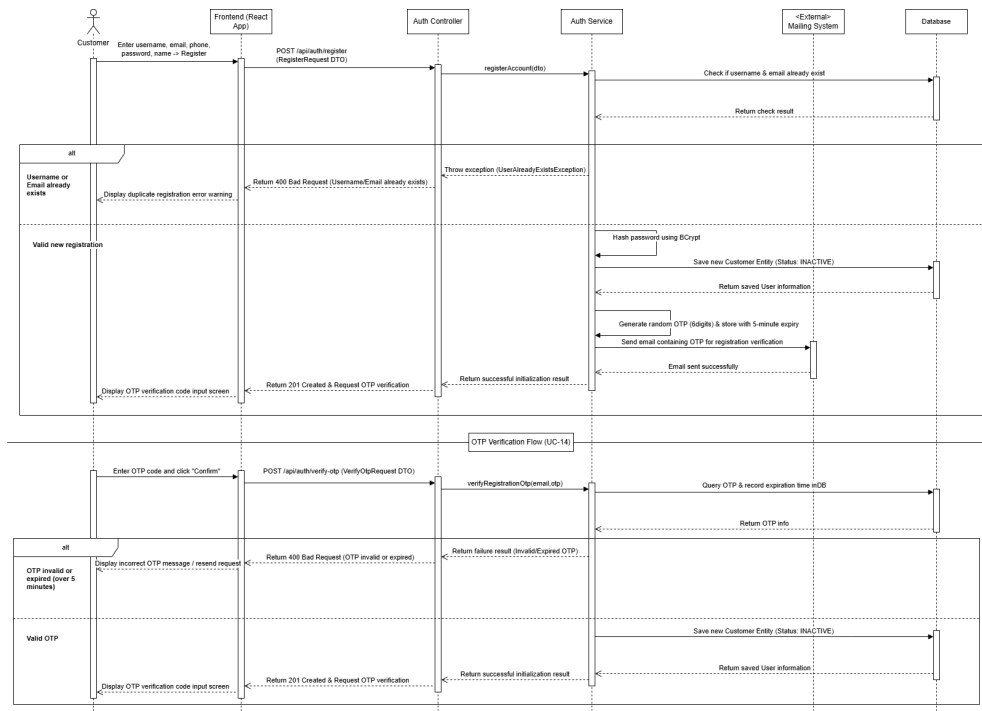


Figure 14: Customer Account Registration and Email OTP Verification Sequence Diagram

System Login – Regular & Via Google (UC-15)

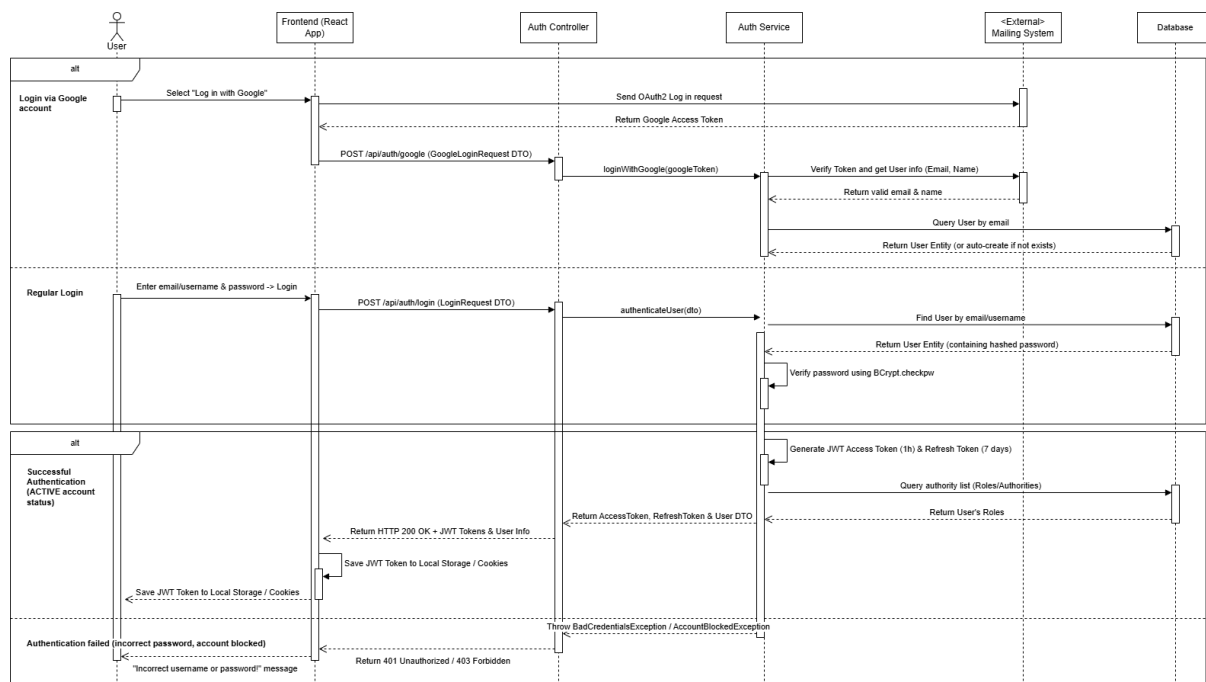


Figure 15: System Login Regular and Google OAuth Sequence Diagram

Online Table Booking & Deposit Payment via PayOS (UC-11)

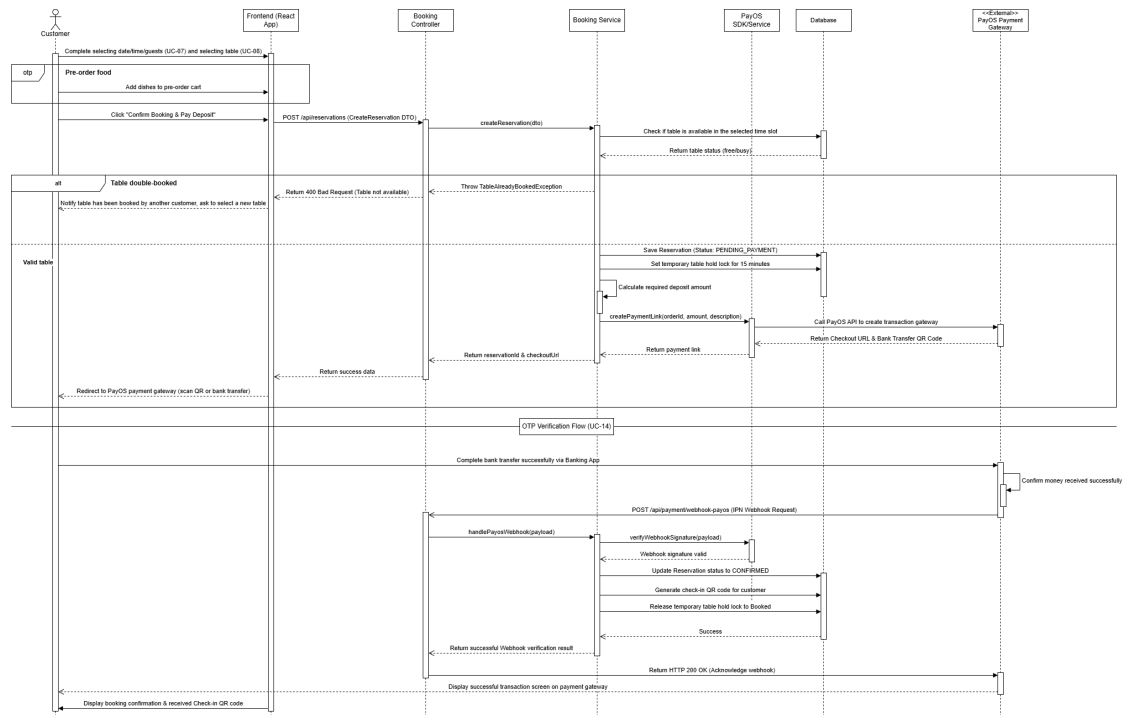


Figure 16: Online Table Booking and PayOS Deposit Payment Sequence Diagram

Table Booking & Menu Consultation via AI Chatbot Gemini (UC-23)

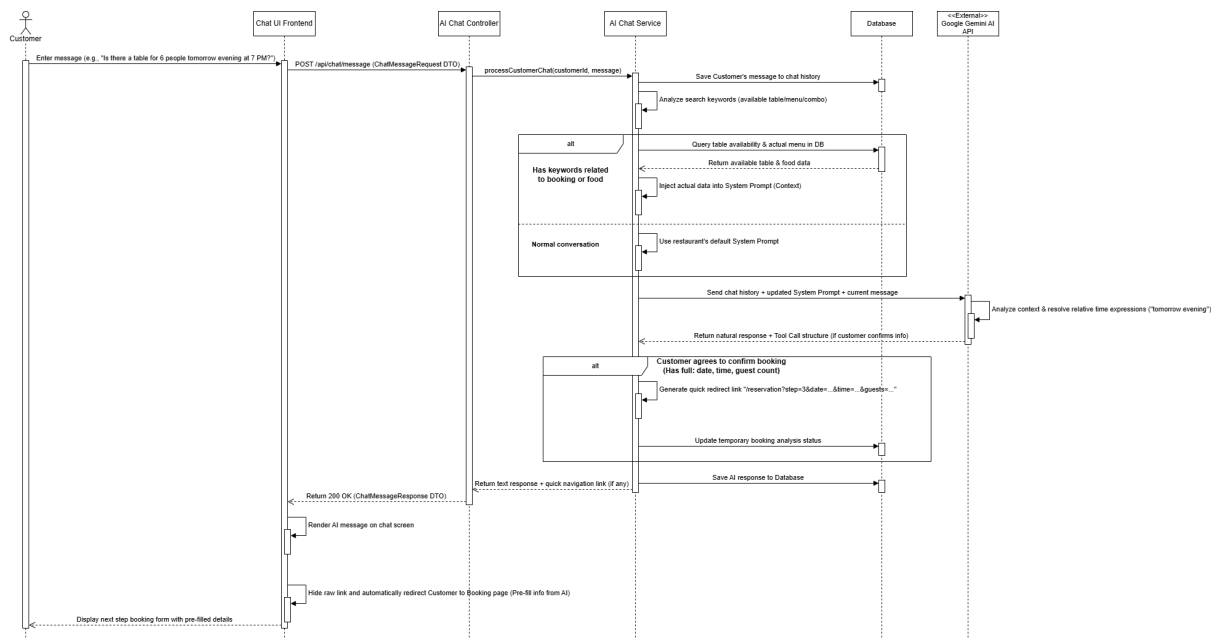


Figure 17: Table Booking and Menu Consultation via Gemini AI Chatbot Sequence Diagram

Add Dishes/Combos Directly at Dine-in Table (UC-31)

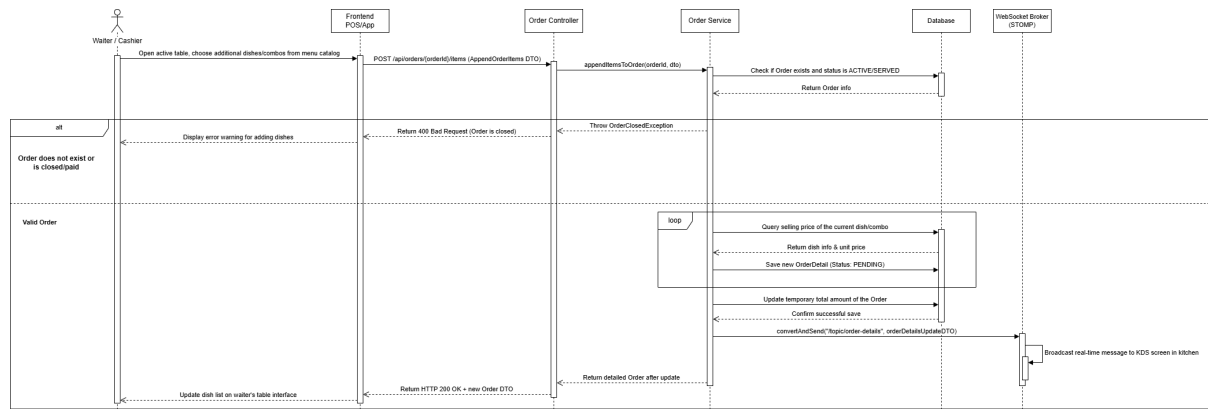


Figure 18: Add Dishes and Combos at Dine-in Table Sequence Diagram

Kitchen Cooking Process & Status Synchronization via WebSockets (UC-37)

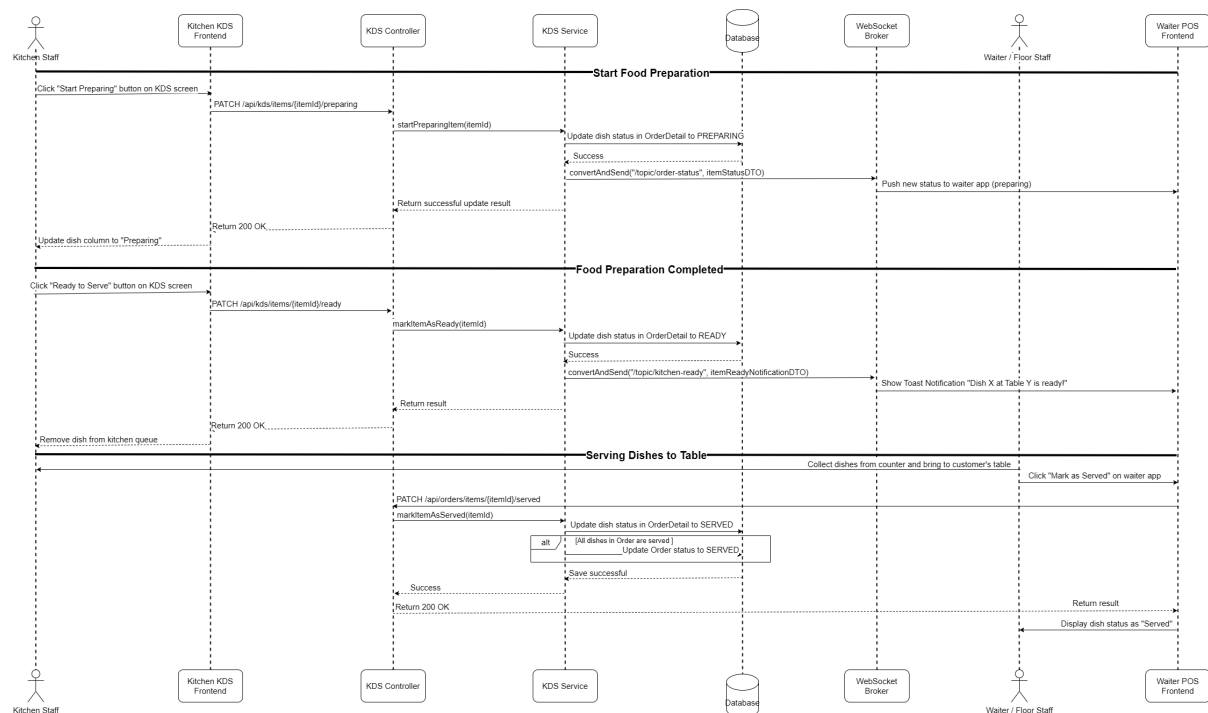


Figure 19: Kitchen Cooking Process and WebSocket Status Synchronization Sequence Diagram

Counter Bill Payment, Voucher Application & Bill Printing (UC-33 & UC-32)

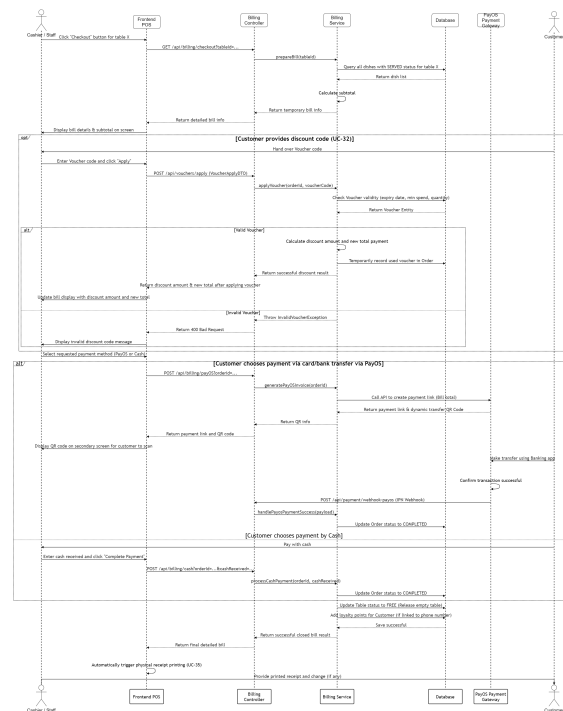


Figure 20: Counter Bill Payment, Voucher Application, and Bill Printing Sequence Diagram

Staff Check-in/Check-out Attendance (UC-39)

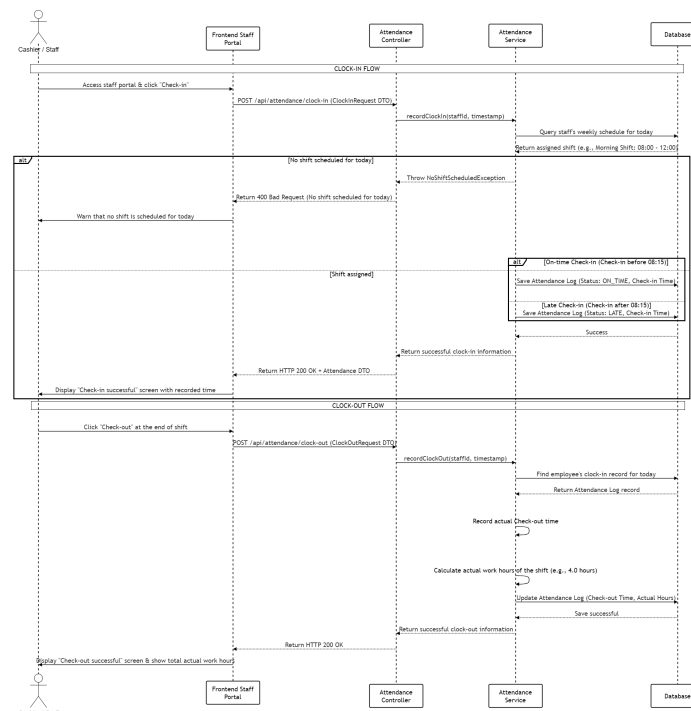


Figure 21: Staff Check-in and Check-out Attendance Sequence Diagram

Admin Menu Item Add/Edit Integrated with Cloudinary (UC-43)

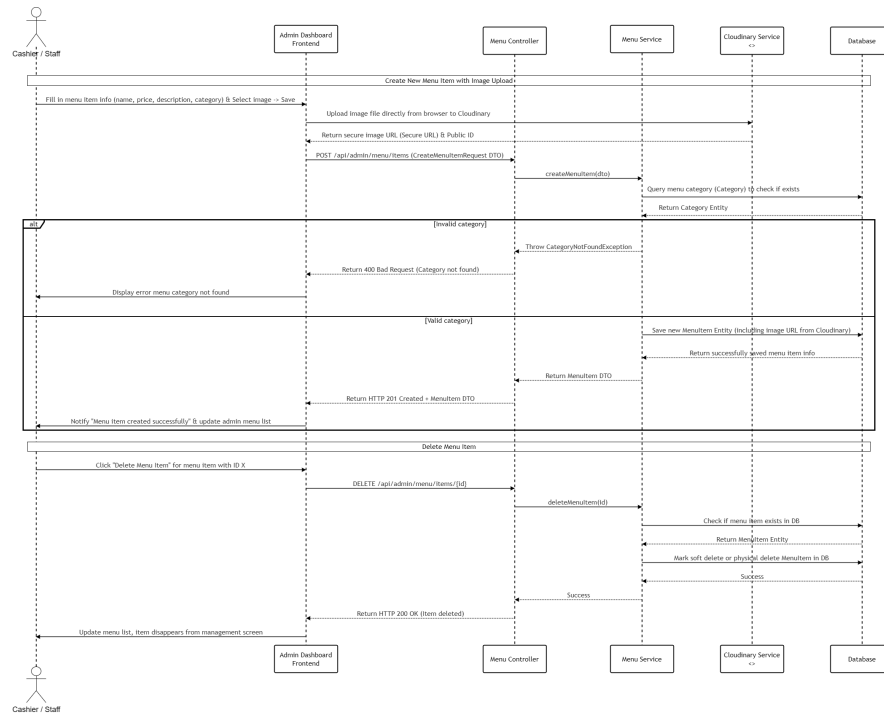


Figure 22: Admin Menu Item Add and Edit with Cloudinary Sequence Diagram

Admin Real-time Revenue Query via Gemini AI Tool Calling (UC-55)

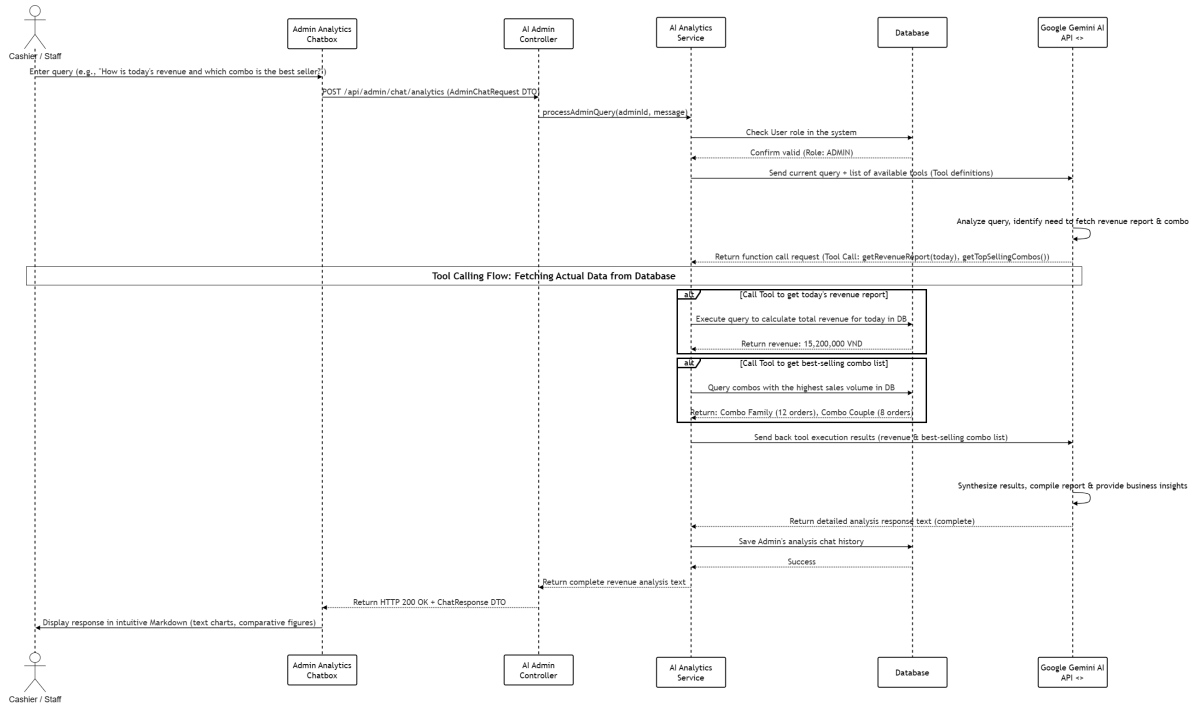


Figure 23: Admin Real-time Revenue Query via Gemini AI Tool Calling Sequence Diagram

II.2 State Diagram

This section presents the state diagrams for key CrownDine workflows, including booking placement and payment, reservation processing, dine-in ordering, and kitchen order processing.

F04 Booking Placement & Payment State Diagram

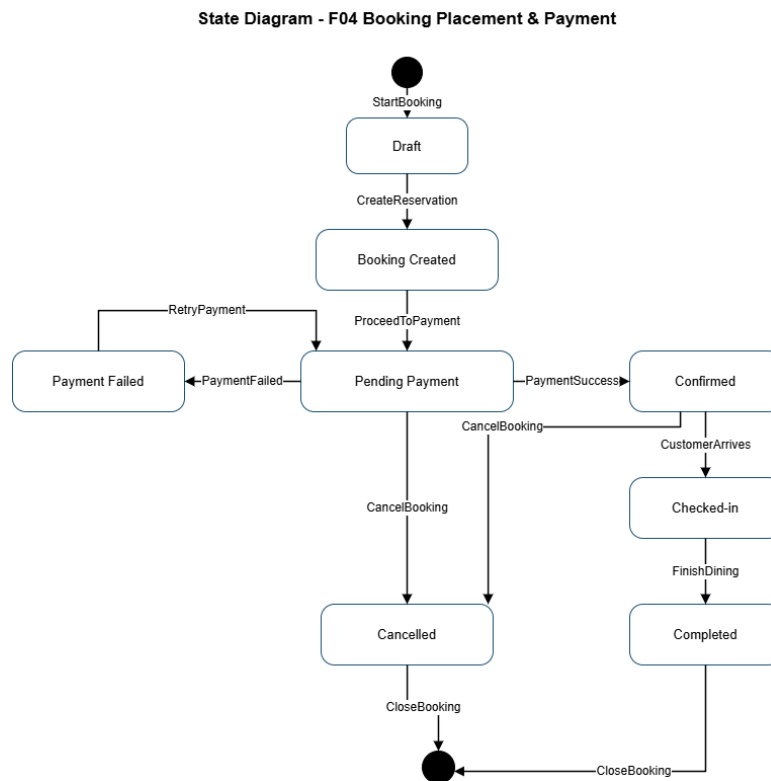


Figure 24: Booking Placement and Payment State Diagram

F08 Reservation Processing State Diagram

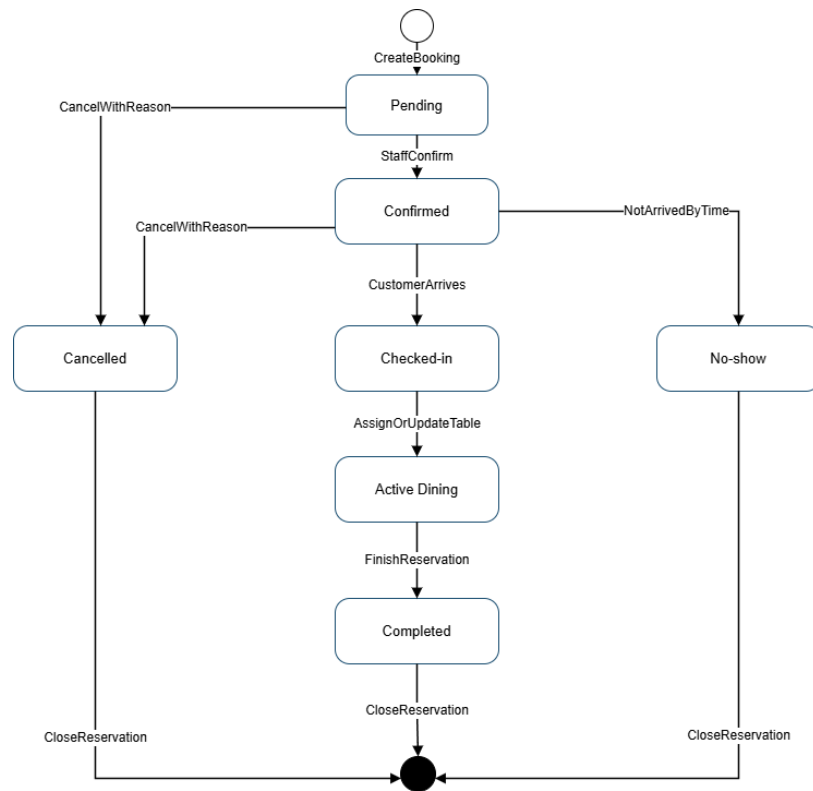


Figure 25: Reservation Processing State Diagram

F09 Walk-in and Dine-in Order State Diagram

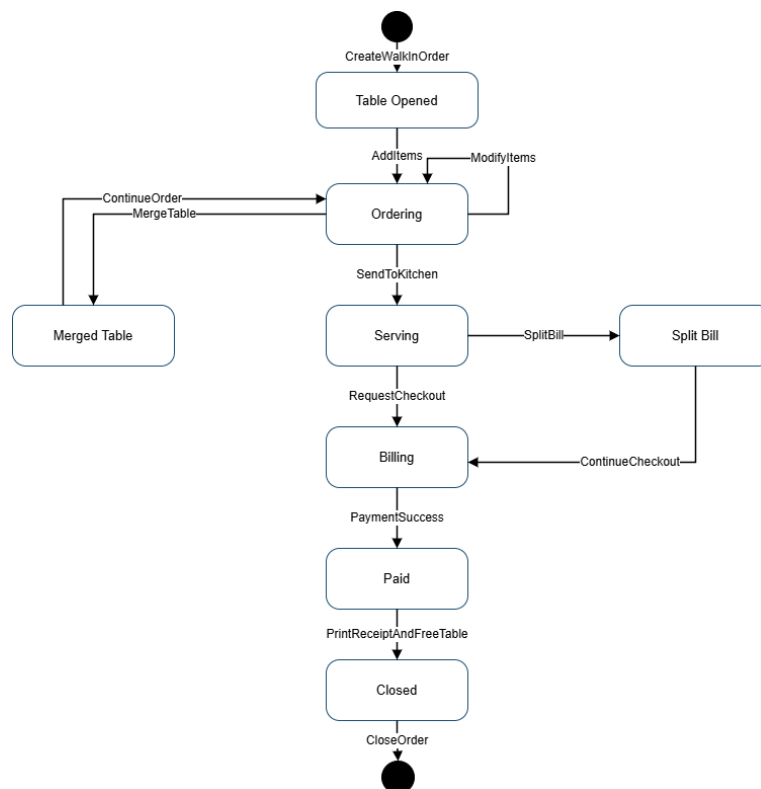


Figure 26: Walk-in and Dine-in Order State Diagram

F10 Kitchen Display System and Order Processing State Diagram

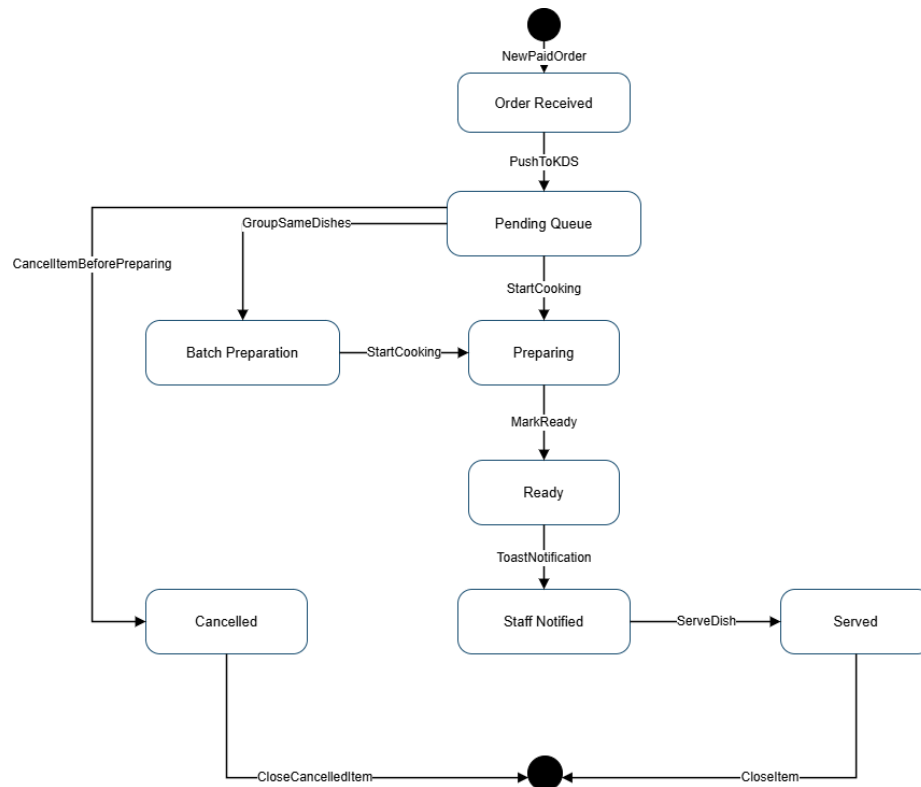


Figure 27: Kitchen Display System and Order Processing State Diagram

II.3 Communication Diagram

This section presents communication diagrams for the main booking and dine-in ordering workflows.

F04 Booking Placement & Payment Communication Diagram

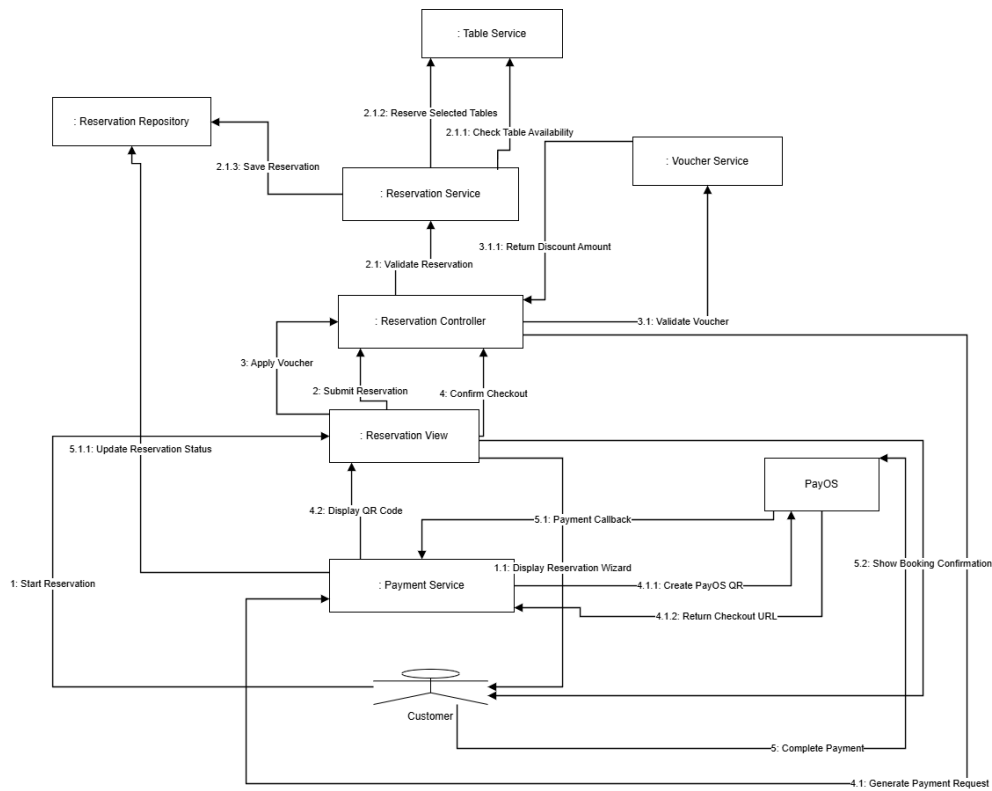


Figure 28: Booking Placement and Payment Communication Diagram

F09 Walk-in Dine-in Order Communication Diagram

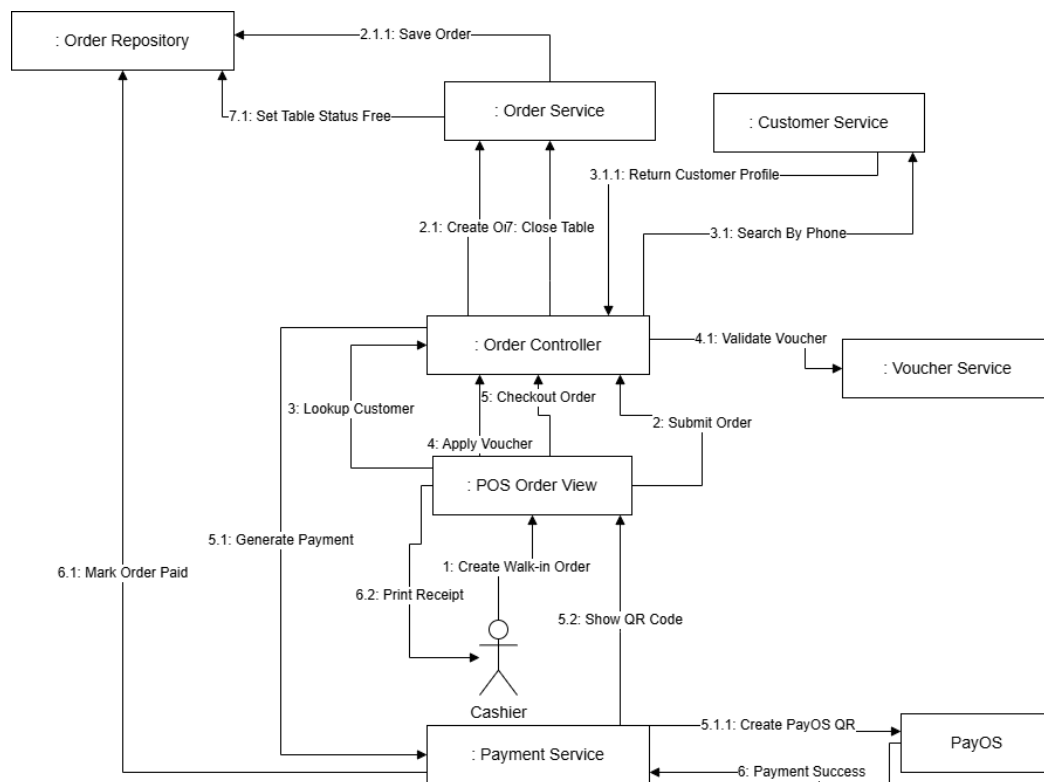


Figure 29: Walk-in Dine-in Order Communication Diagram

III Design Specification

III.1 Integrated Communication Diagrams

The integrated communication diagram describes how the main CrownDine components collaborate to process user requests. Customers, staff, kitchen users, administrators, the frontend application, backend services, database, PayOS payment gateway, Cloudinary storage, WebSocket channel, and Gemini AI service exchange messages through clearly separated responsibilities.

The diagram emphasizes the following communication paths:

- Customer-facing requests are initiated from the web interface and routed to backend services for authentication, reservation, ordering, payment, and profile workflows.
- Staff and kitchen operations communicate with order and reservation services, while WebSocket messages synchronize real-time kitchen and serving statuses.
- Administrative workflows communicate with menu, staff, voucher, reporting, Cloudinary, and AI analytics services.
- External integrations such as PayOS, Cloudinary, and Gemini AI are accessed through backend service boundaries to keep business logic centralized and secure.

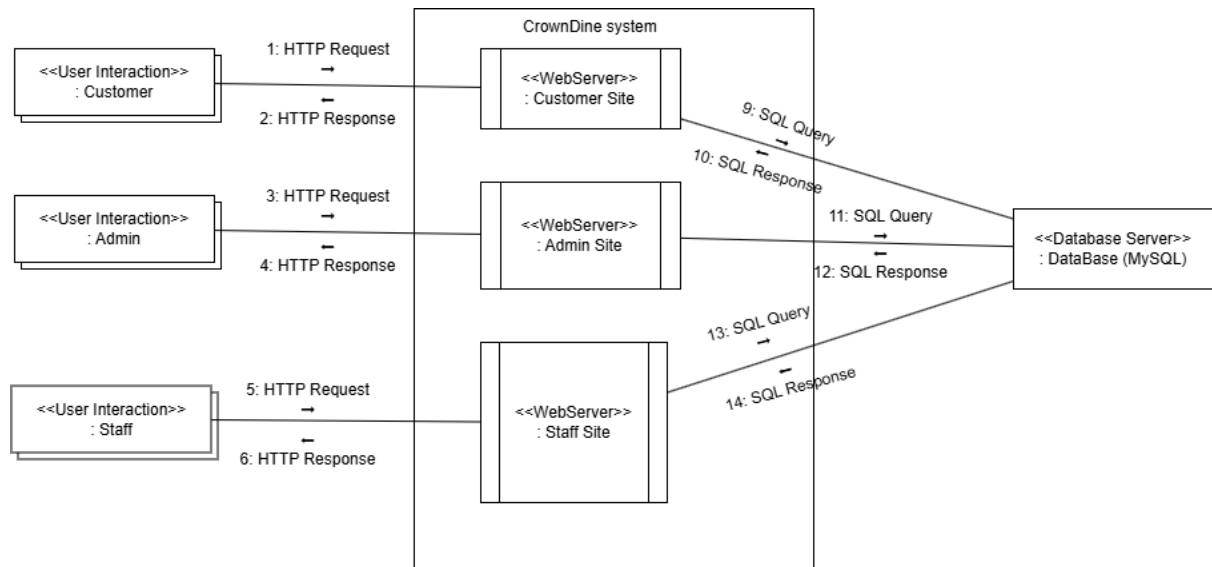


Figure 30: Communication Diagram for the CrownDine System Architecture

III.2 System High-Level Design

The CrownDine system follows a web-based layered architecture. The frontend layer provides role-based user interfaces for customers, staff, kitchen users, and administrators. The backend layer exposes APIs and coordinates domain services such as authentication, booking, menu management, ordering, payment, attendance, reporting, notification, and AI assistance. The persistence and integration layers support database storage and external services.

At a high level, the architecture separates responsibilities as follows:

- **Presentation layer:** React-based interfaces support customer booking, staff operations, kitchen display, and administrative management.
- **Application service layer:** Backend controllers and services validate requests, enforce role-based access control, and coordinate business workflows.
- **Domain layer:** Core modules manage reservations, tables, orders, menu items, vouchers, payments, shifts, attendance, and reporting logic.
- **Integration layer:** PayOS handles payment checkout and callbacks, Cloudinary stores uploaded images, WebSockets synchronize real-time status changes, and Gemini AI supports chatbot and analytics features.
- **Data layer:** The database persists users, roles, tables, reservations, orders, payments, menu data, vouchers, attendance records, and reports.

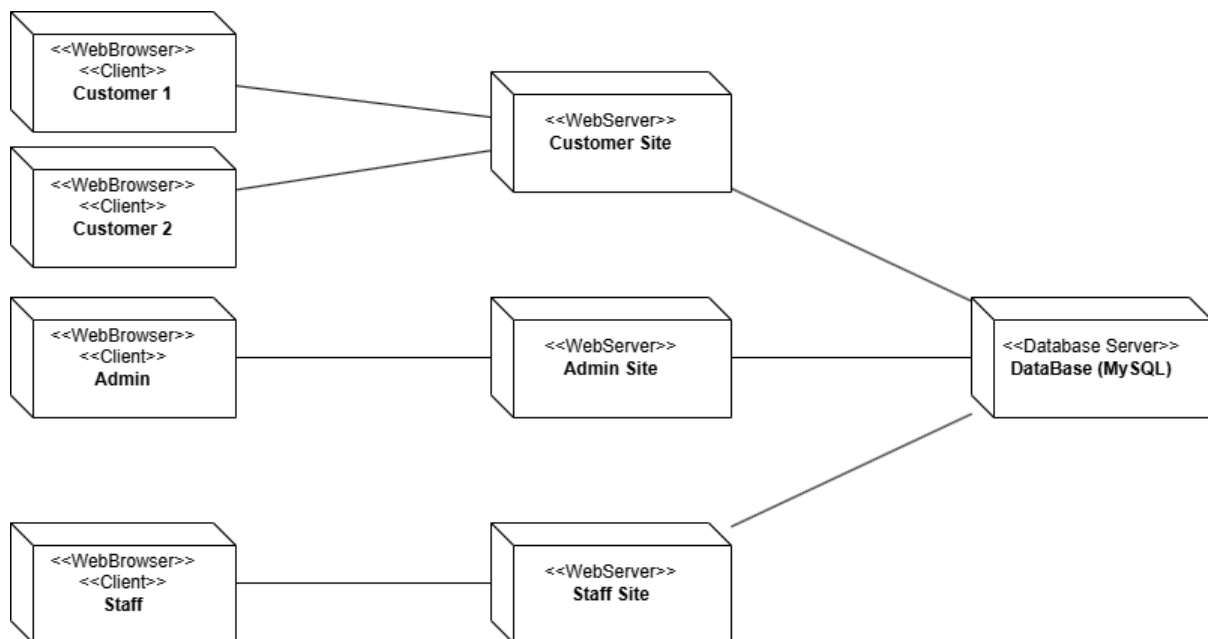


Figure 31: CrownDine System High-Level Design

The class diagram refines the high-level architecture by showing the major domain entities and their relationships. It highlights how user accounts, roles, tables, reservations, orders, order details, payments, menu items, vouchers, shifts, attendance records, and feedback are organized to support the functional requirements.

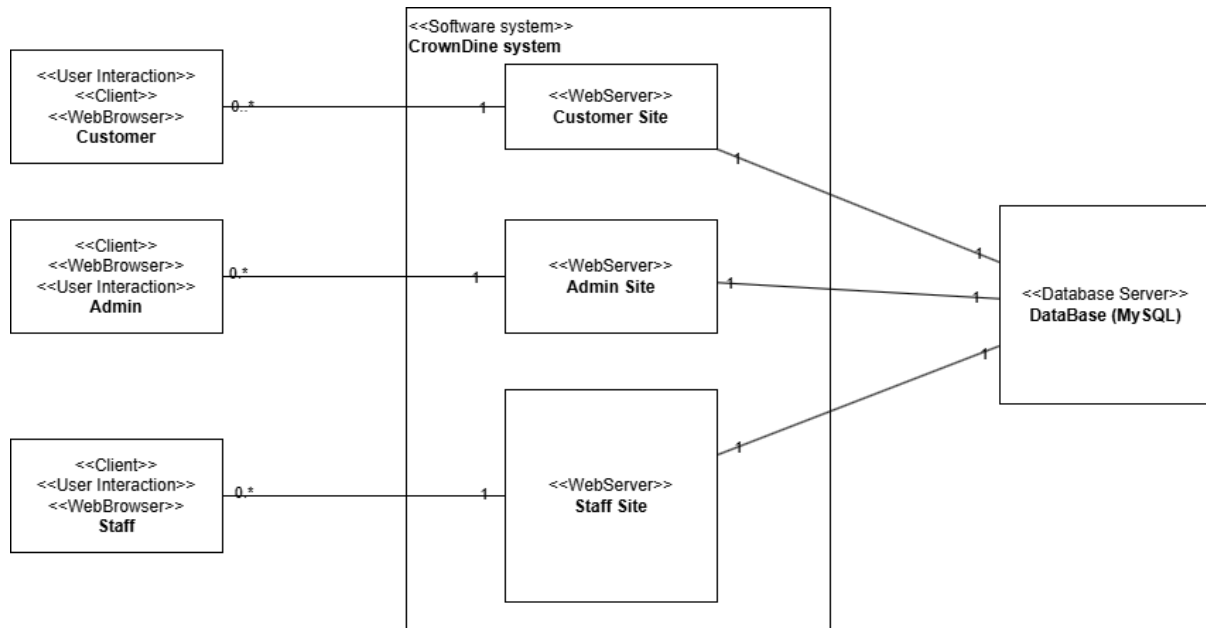


Figure 32: Class Diagram for the CrownDine System Architecture

III.3 Component & Package Diagram

This section presents the component diagrams of the CrownDine Restaurant Management System. These diagrams describe how the system is divided into deployable and maintainable components across the frontend, backend, real-time ordering, AI analytics, payment, storage, and database layers. The component view clarifies the responsibilities of each module, the dependencies between modules, and the communication boundaries between internal services and external platforms such as PayOS, Cloudinary, Gemini AI, and WebSocket infrastructure.

The overall architecture diagram provides a system-wide view of how client interfaces communicate with backend APIs and supporting services. The backend application diagram details the main service modules that handle authentication, reservation, menu, order, payment, voucher, attendance, reporting, and integration workflows. The frontend diagram shows how role-based user interfaces are organized for customers, staff, kitchen users, and administrators. The KDS and real-time ordering diagram focuses on the interaction between dine-in orders, kitchen preparation status, and live WebSocket updates. Finally, the AI chatbot and analytics integration diagram explains how Gemini AI supports customer booking assistance and administrative revenue analysis through backend-controlled tool calling and data access.

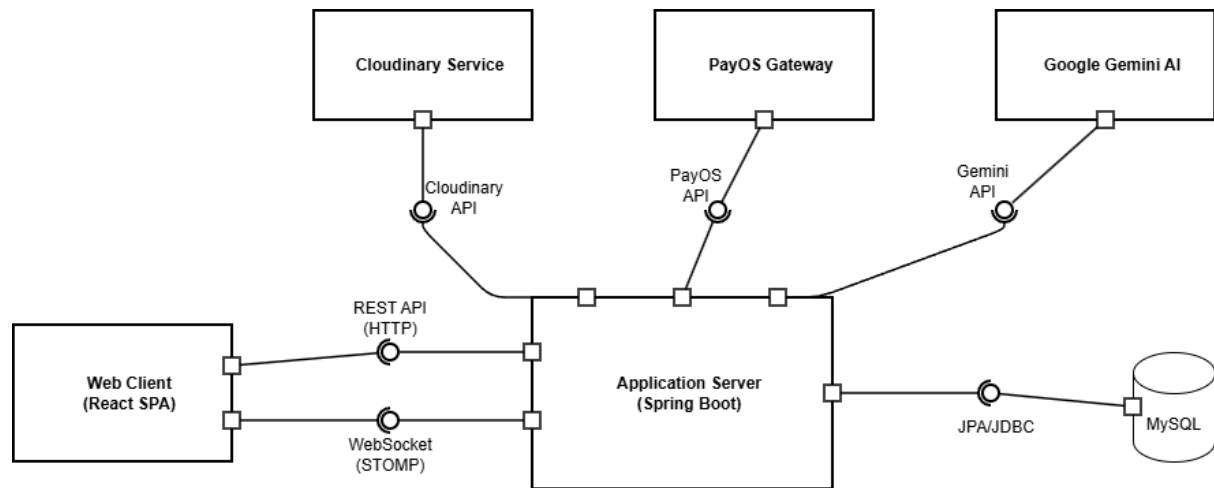


Figure 33: Overall System Architecture Component Diagram

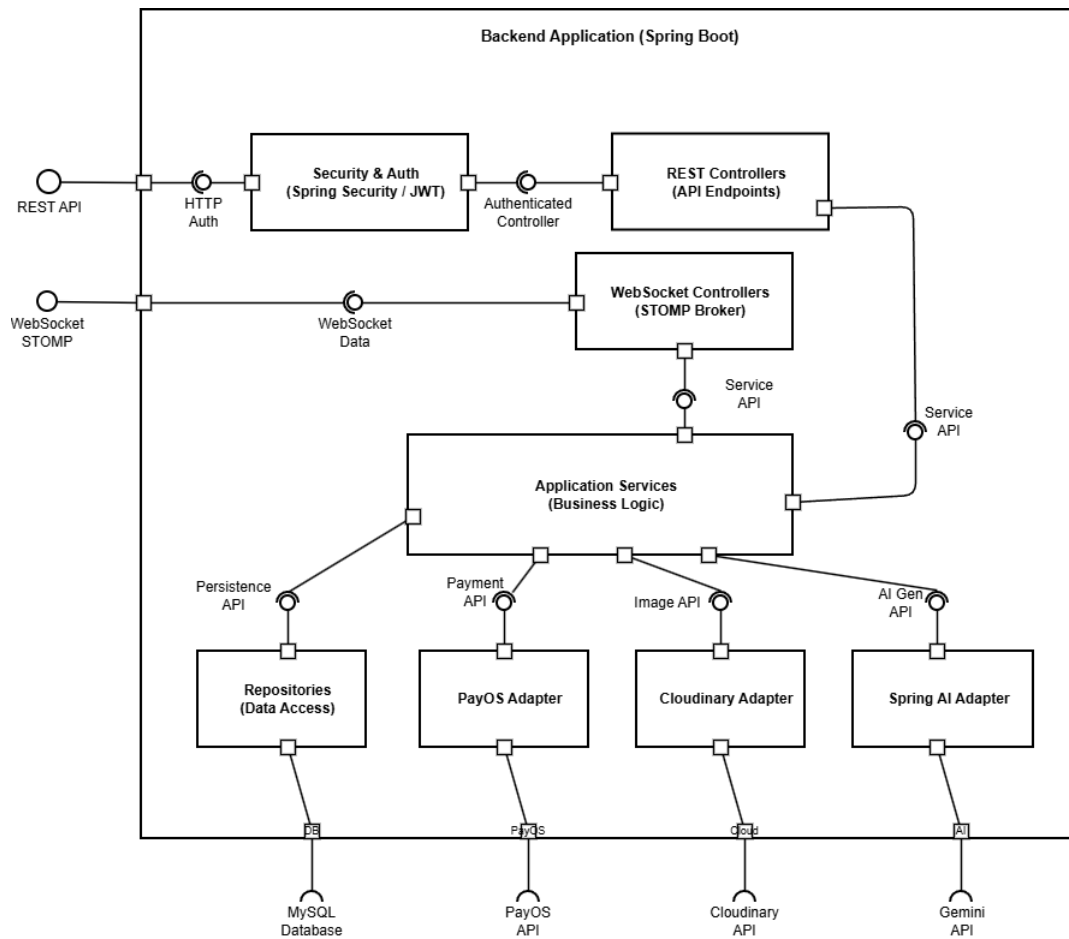


Figure 34: Backend Application Component Diagram

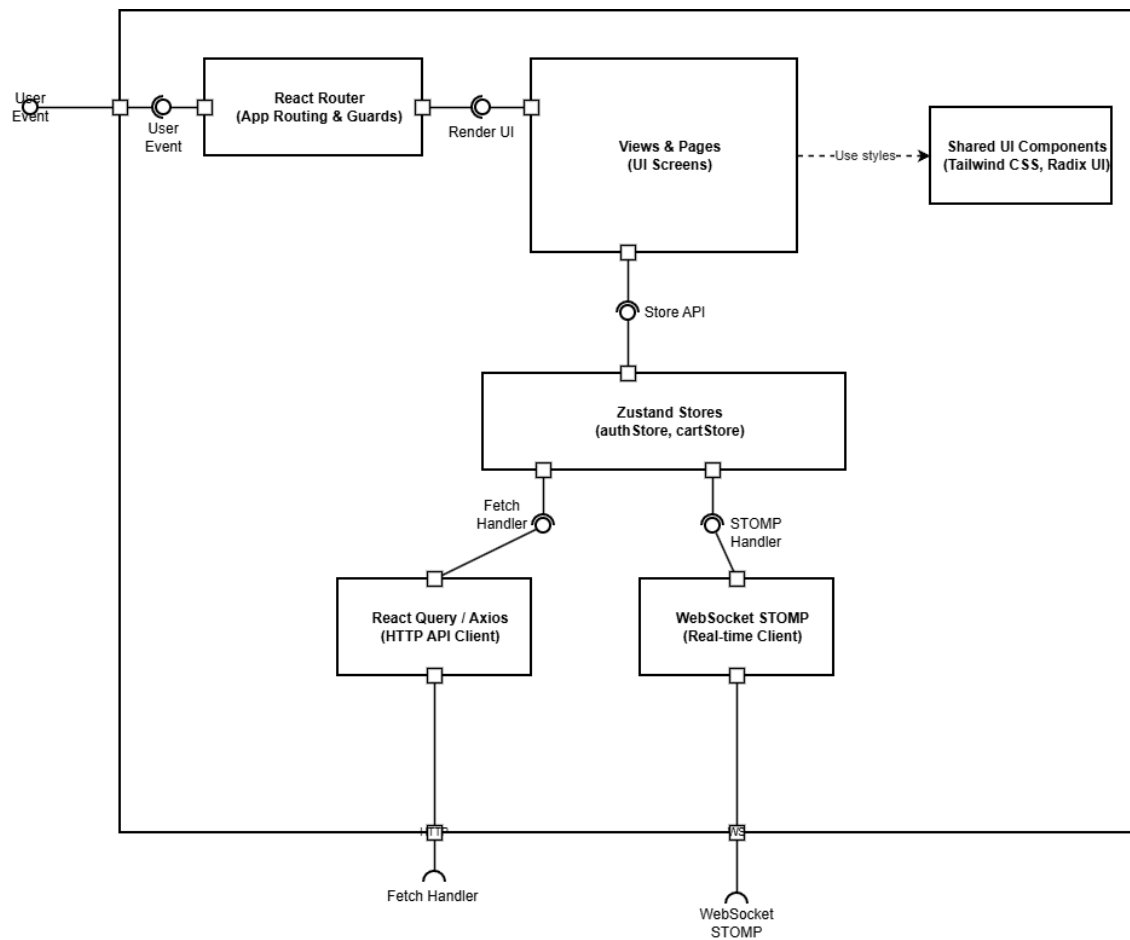


Figure 35: Frontend Application Component Diagram

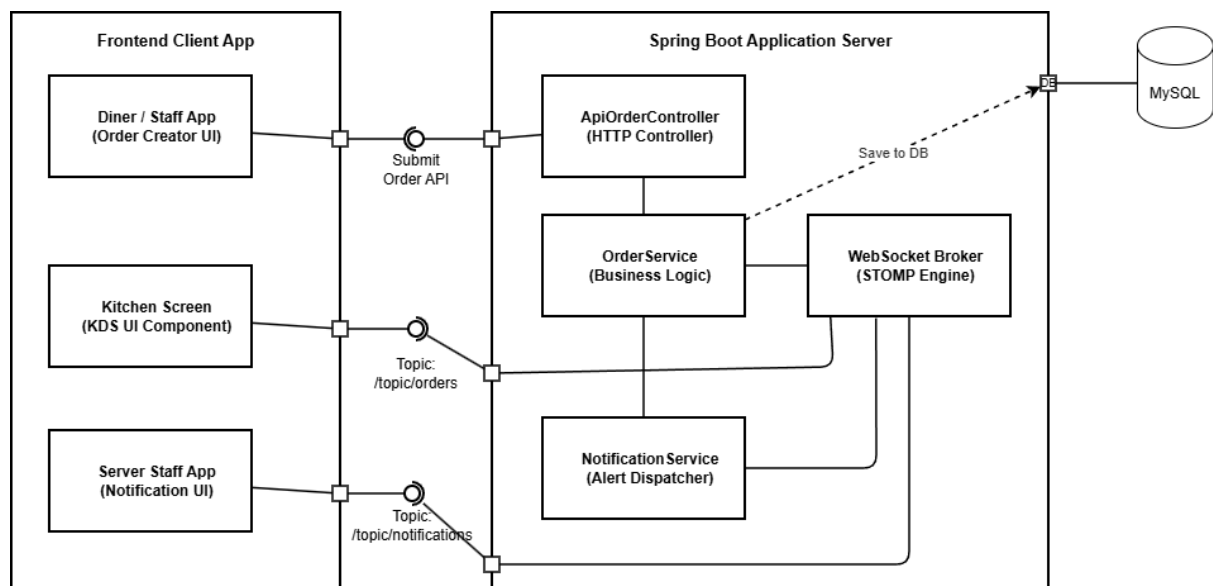


Figure 36: KDS and Real-time Ordering Component Diagram

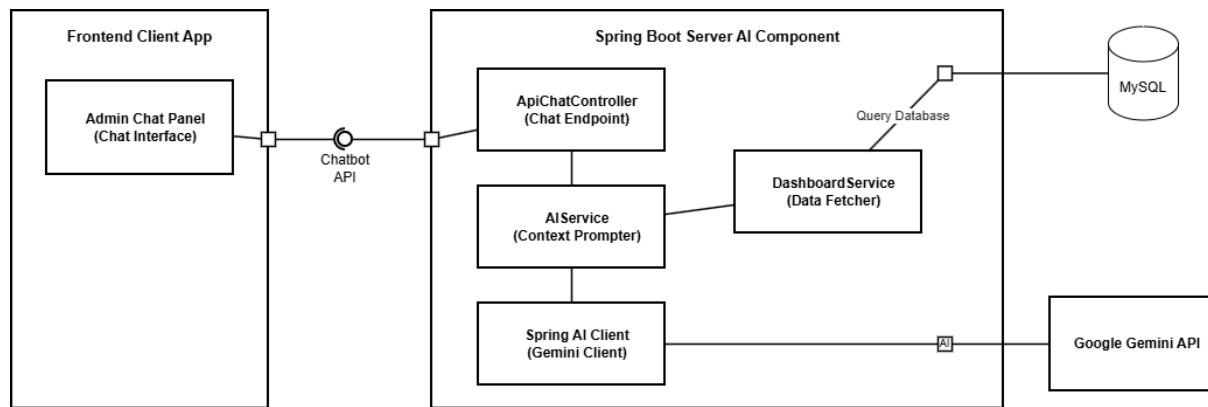


Figure 37: AI Chatbot and Data Analytics Integration Component Diagram

III.4 Detailed Design

The detailed design section describes the internal module-level structure of the CrownDine Restaurant Management System. Each diagram focuses on one functional module and shows the main classes, responsibilities, relationships, and data flow needed to implement the corresponding business capability. Together, these diagrams provide a bridge between the high-level architecture and the actual source code modules.

The detailed design is organized into nine modules: menu management, table and floor management, reservation management, order and payment management, authentication and users, staff schedule management, marketing and promotion, customer engagement, and communication. These modules align with the functional requirements and support separation of concerns across the backend services, frontend features, persistence layer, and external integrations.

Menu Management Detailed Design This module defines the detailed design for menu categories, food items, combo packages, pricing, image upload, availability status, and menu search/filter operations.

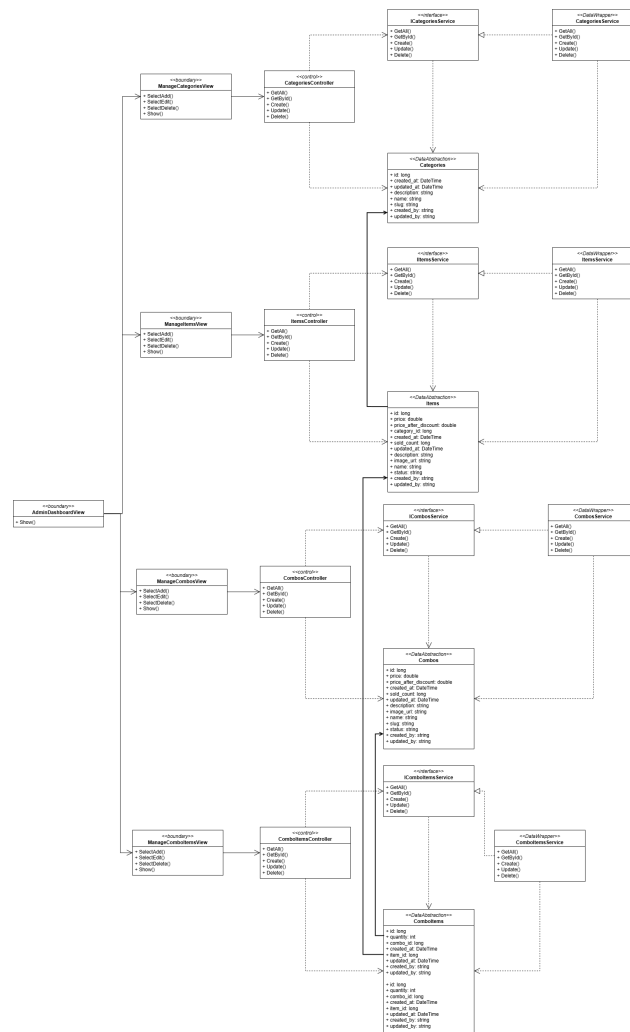


Figure 38: Menu Management Detailed Design Diagram

Table and Floor Management Detailed Design This module defines how restaurant floors, areas, and tables are created, configured, positioned, and updated to support the interactive table map and reservation workflows.

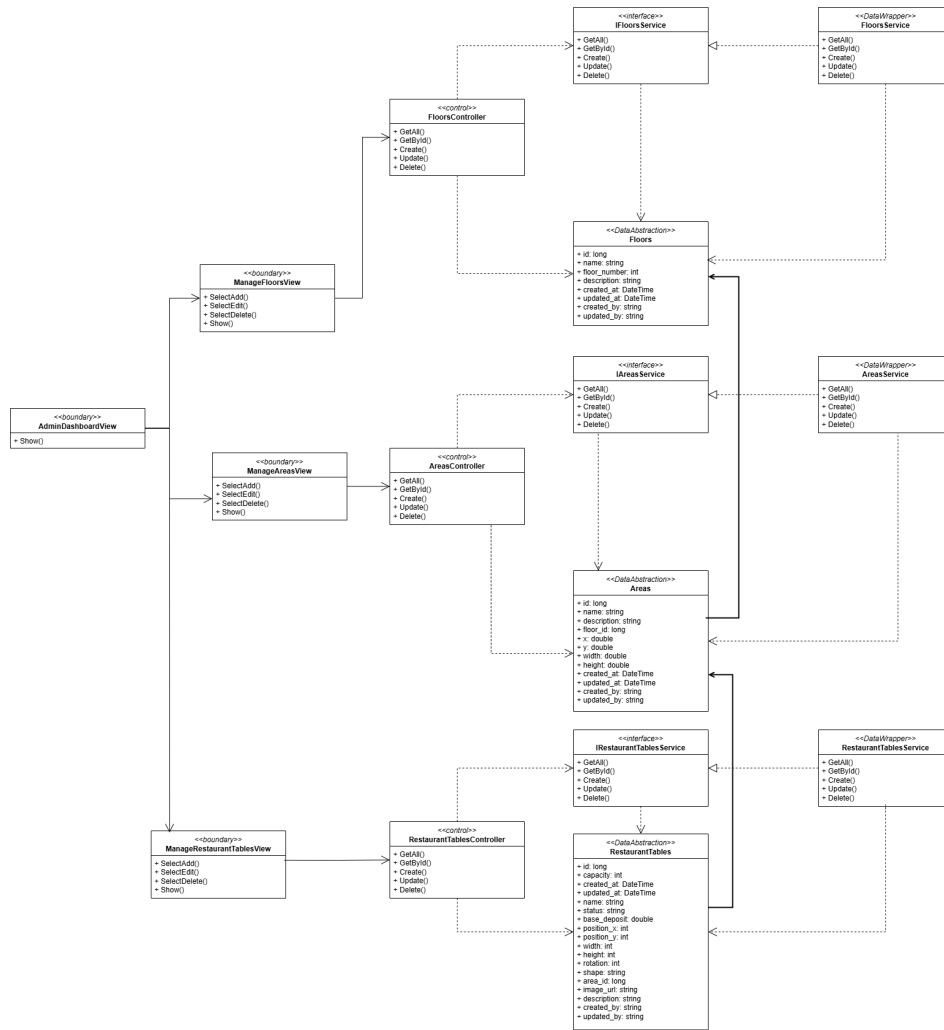


Figure 39: Table and Floor Management Detailed Design Diagram

Reservation Management Detailed Design This module describes the reservation lifecycle, including table selection, booking creation, deposit payment, status transitions, cancellation handling, and staff check-in operations.

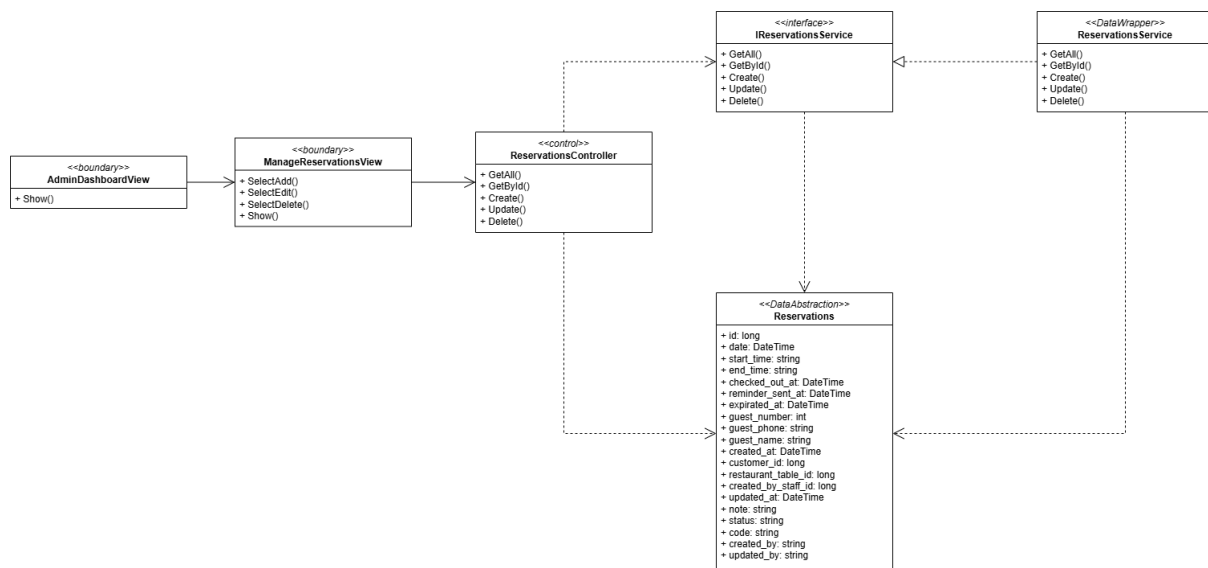


Figure 40: Reservation Management Detailed Design Diagram

Order and Payment Management Detailed Design This module describes dine-in ordering, order detail updates, voucher application, bill calculation, PayOS payment integration, invoice printing, and order completion.

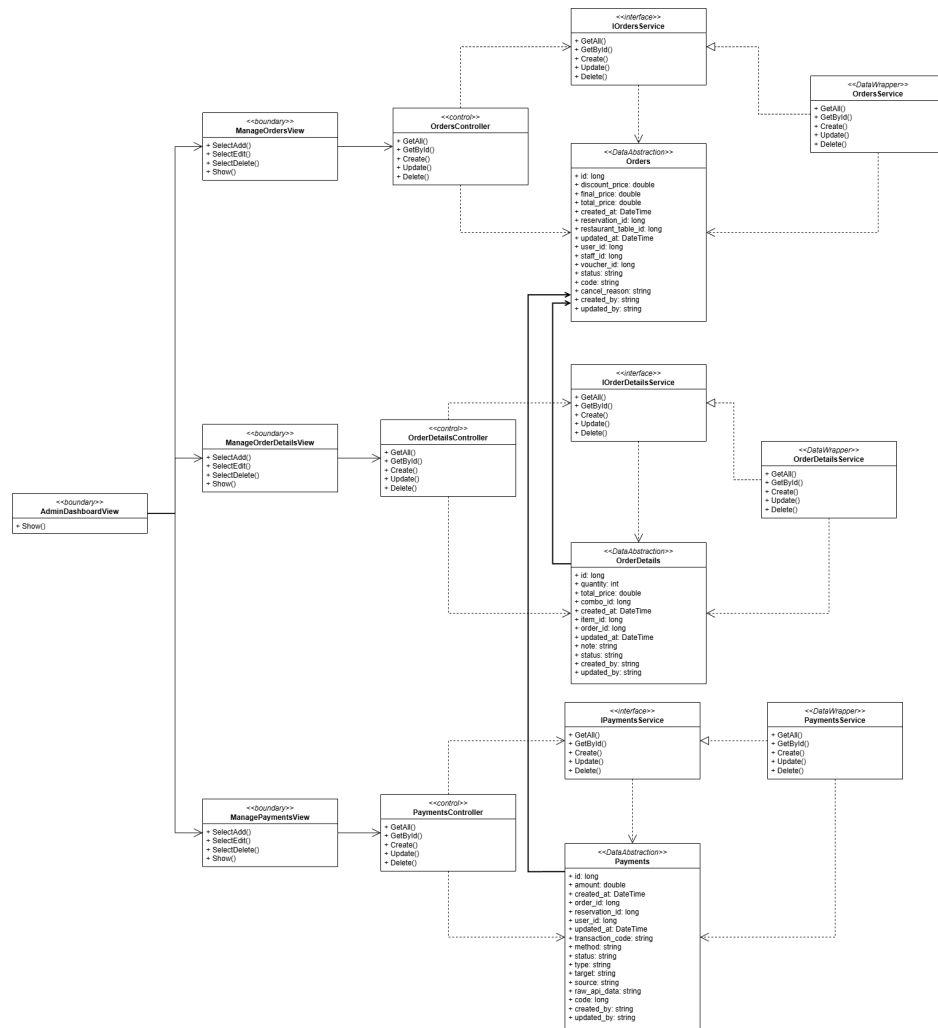


Figure 41: Order and Payment Management Detailed Design Diagram

Authentication and Users Detailed Design This module covers account registration, login, Google OAuth2, OTP verification, JWT token handling, profile management, staff accounts, roles, and authorization rules.

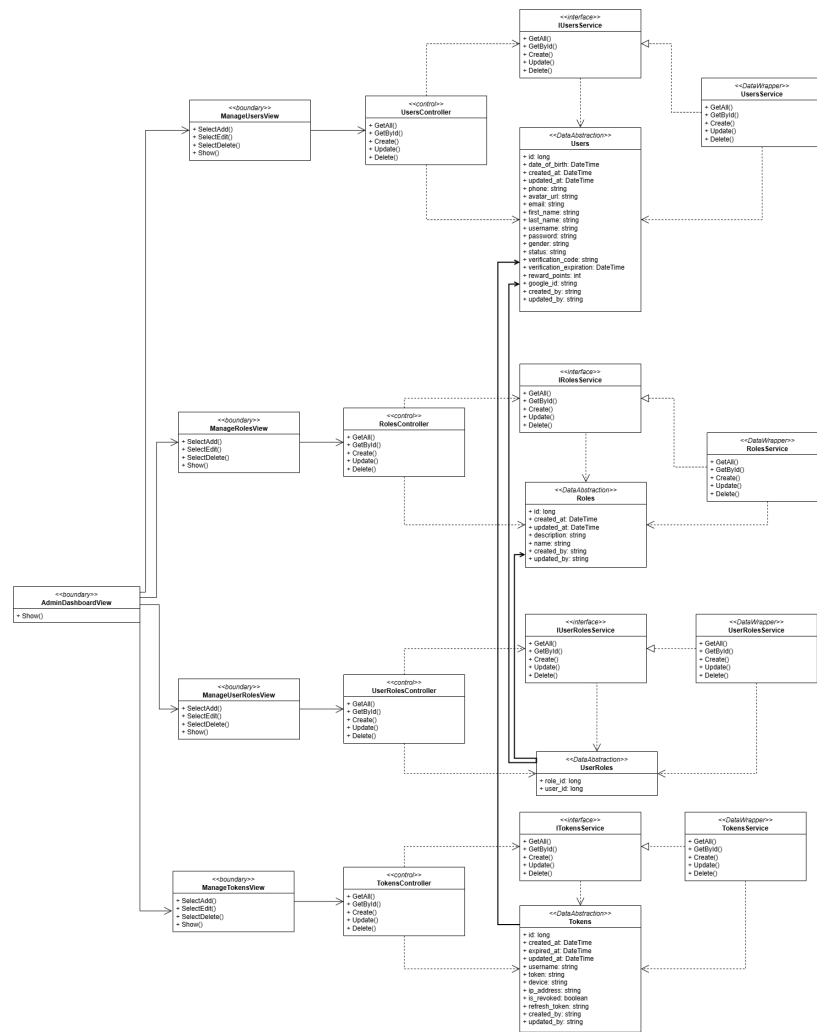


Figure 42: Authentication and Users Detailed Design Diagram

Staff Schedule Management Detailed Design This module defines staff shift templates, weekly work schedules, attendance records, check-in/check-out processing, and administrative attendance review.

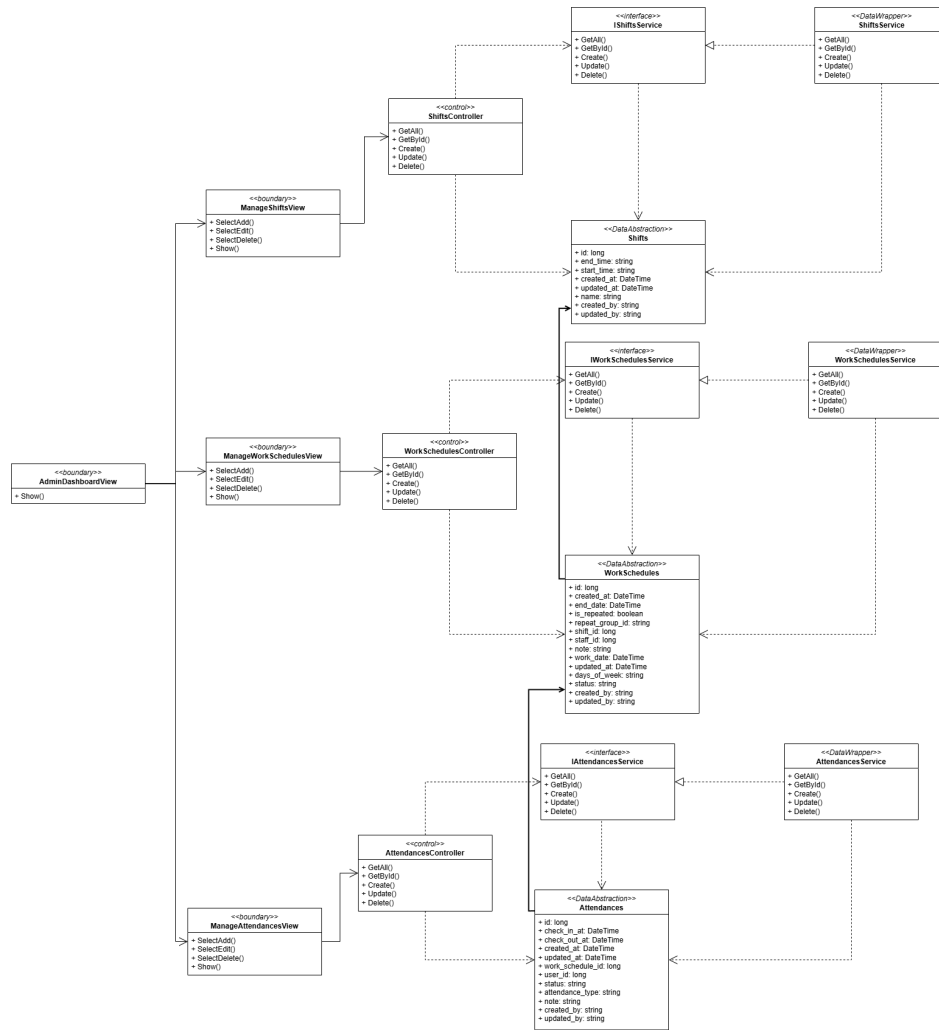


Figure 43: Staff Schedule Management Detailed Design Diagram

Marketing and Promotion Detailed Design This module covers voucher configuration, promotion rules, point exchange settings, user-specific voucher assignment, and discount validation during checkout.

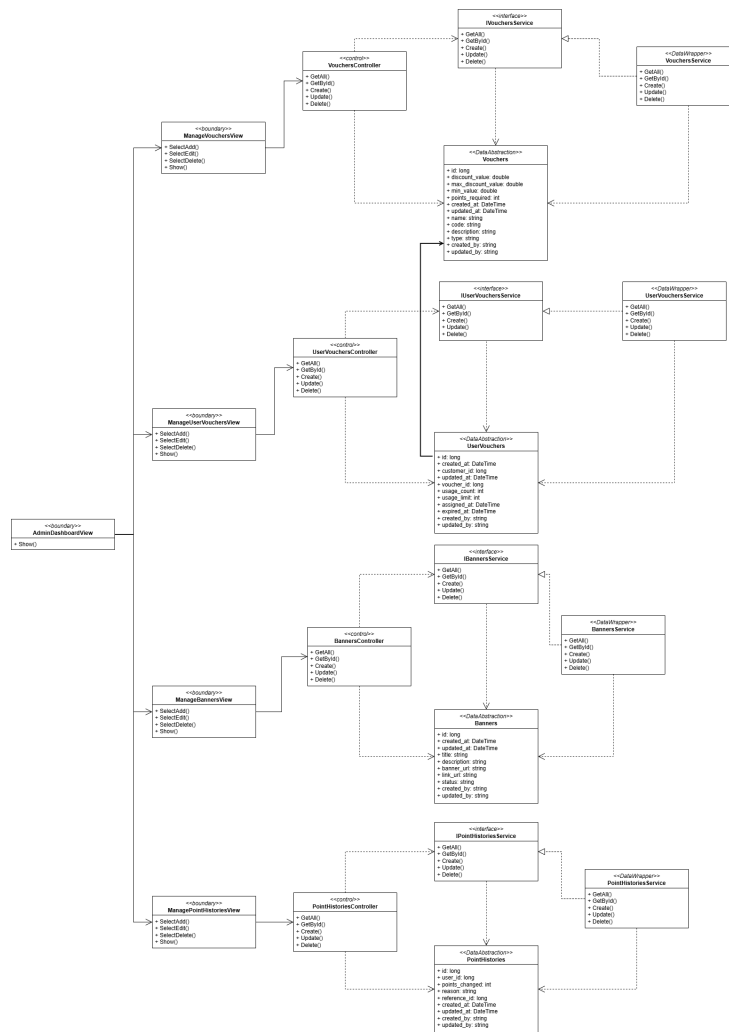


Figure 44: Marketing and Promotion Detailed Design Diagram

Customer Engagement Detailed Design This module describes customer-facing engagement features such as favorites, feedback, loyalty points, booking history, notification support, and AI-assisted booking interaction.

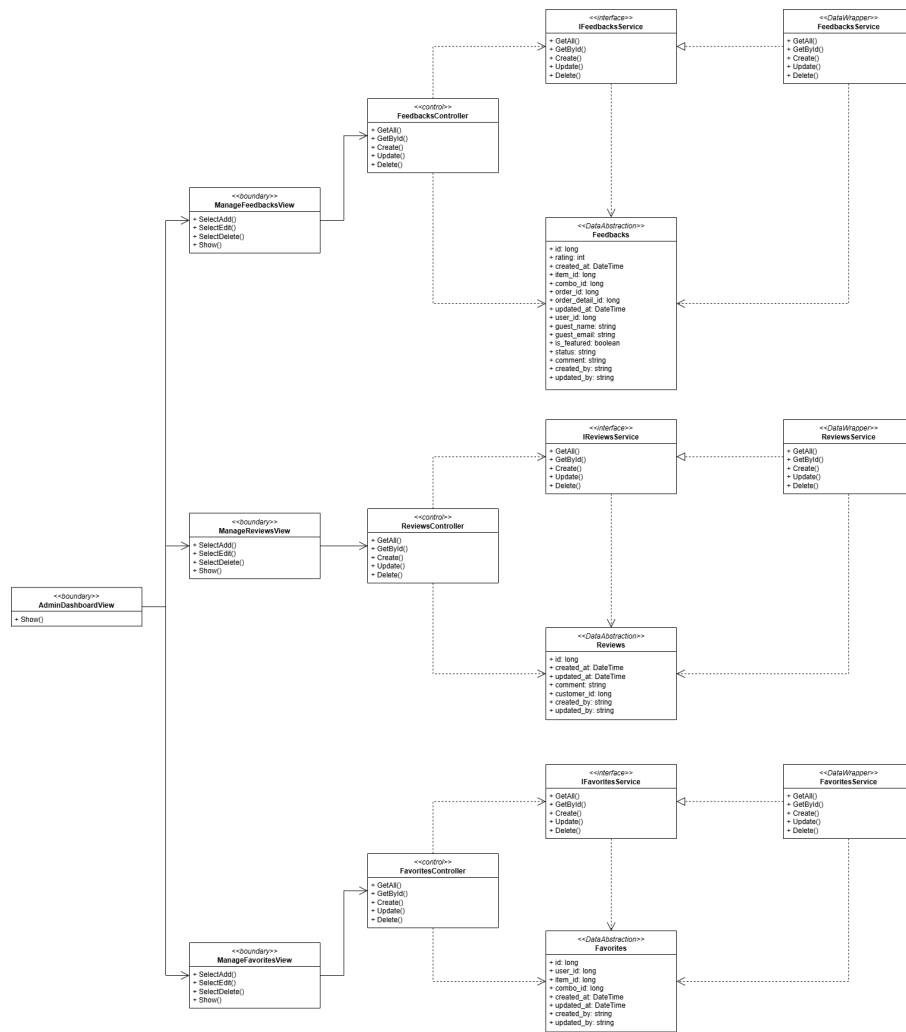


Figure 45: Customer Engagement Detailed Design Diagram

Communication Detailed Design This module defines communication mechanisms used by the system, including notification delivery, WebSocket updates, real-time kitchen coordination, and integration message flows.

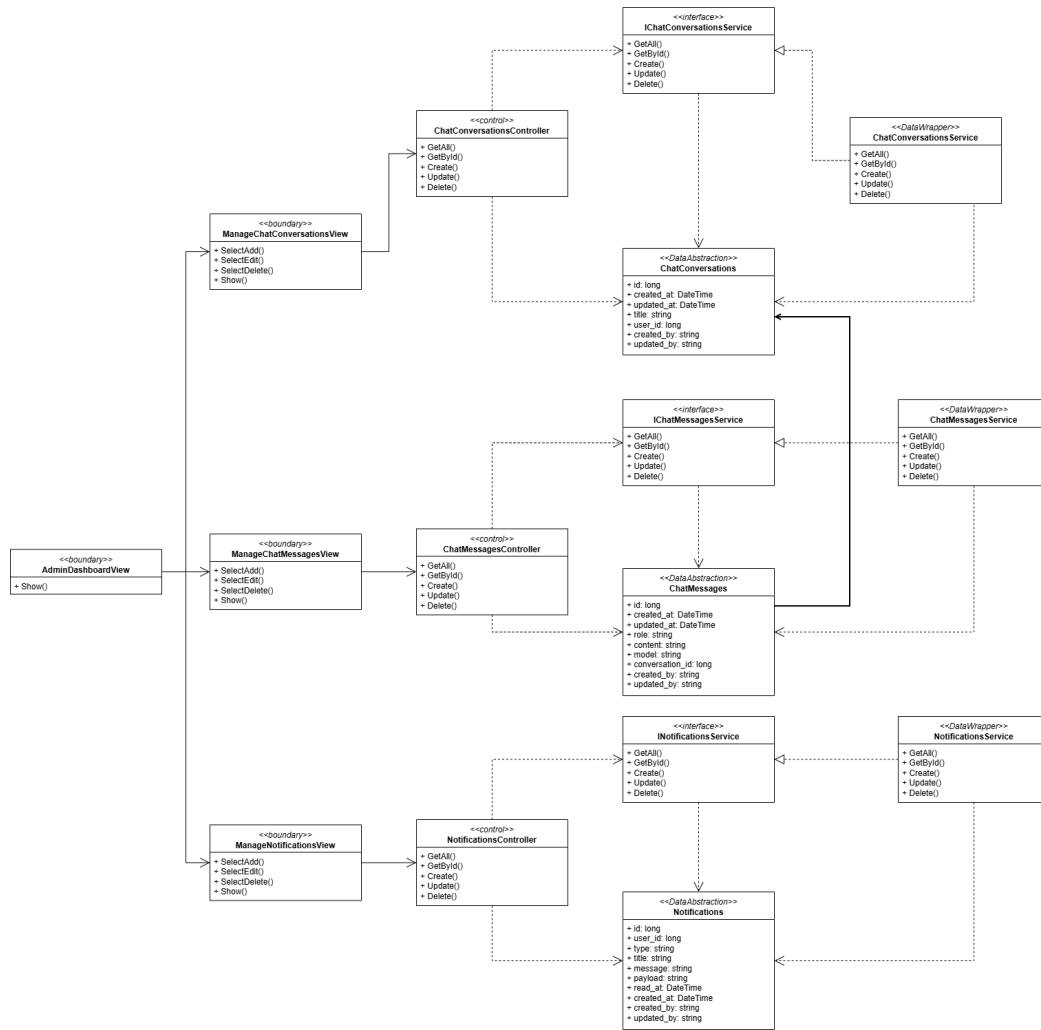


Figure 46: Communication Detailed Design Diagram

III.5 Database Design

This section describes the main database tables of the CrownDine Restaurant Management System. The schema is grouped by business domain to clarify table responsibilities, primary keys, foreign keys, and major attributes.

1. User & Role Management

Users(id [PK]: bigint, date_of_birth: date, created_at: datetime(6), updated_at: datetime(6), phone: varchar(11), avatar_url: varchar(200), email: varchar(255), first_name: varchar(50), last_name: varchar(50), username: varchar(50), password: varchar(255), gender: enum('FEMALE', 'MALE', 'OTHER'), status: enum('ACTIVE', 'INACTIVE'), verification_code: varchar(255), verification_expiration: datetime(6), reward_points: int, google_id: varchar(255), created_by: varchar(255), updated_by: varchar(255))

Roles(id [PK]: bigint, created_at: datetime(6), updated_at: datetime(6), description: varchar(500), name: enum('ADMIN', 'STAFF', 'USER'), created_by: varchar(255), updated_by: varchar(255))

User_Roles(role_id [PK, FK]: bigint, user_id [PK, FK]: bigint)

Tokens(id [PK]: bigint, created_at: datetime(6), expired_at: datetime(6), updated_at: datetime(6), username: varchar(255), token: text, device: varchar(1000), ip_address: varchar(45), is_revoked: bit(1), refresh_token: varchar(1000), created_by: varchar(255), updated_by: varchar(255))

2. Product & Combo Management

Categories(id [PK]: bigint, created_at: datetime(6), updated_at: datetime(6), description: varchar(255), name: varchar(255), slug: varchar(255), created_by: varchar(255), updated_by: varchar(255))

Items(id [PK]: bigint, price: decimal(38,2), price_after_discount: decimal(38,2), category_id [FK]: bigint, created_at: datetime(6), sold_count: bigint, updated_at: datetime(6), description: varchar(255), image_url: varchar(255), name: varchar(255), status: enum('AVAILABLE', 'UNAVAILABLE'), created_by: varchar(255), updated_by: varchar(255))

Combos(id [PK]: bigint, price: decimal(38,2), price_after_discount: decimal(38,2), created_at: datetime(6), sold_count: bigint, updated_at: datetime(6), description: varchar(255), image_url: varchar(255), name: varchar(255), slug: varchar(255), status: enum('AVAILABLE', 'UNAVAILABLE'), created_by: varchar(255), updated_by: varchar(255))

Combo_Items(id [PK]: bigint, quantity: int, combo_id [FK]: bigint, created_at: datetime(6), item_id [FK]: bigint, updated_at: datetime(6), created_by: varchar(255), updated_by: varchar(255))

Favorites(id [PK]: bigint, user_id [FK]: bigint, item_id [FK]: bigint, combo_id [FK]: bigint, created_at: datetime(6), updated_at: datetime(6), created_by: varchar(255), updated_by: varchar(255))

3. Table & Layout Management

Floors(id [PK]: bigint, name: varchar(255), floor_number: int, description: varchar(500), created_at: timestamp, updated_at: timestamp, created_by: varchar(255), updated_by: varchar(255))

Areas(id [PK]: bigint, name: varchar(255), description: varchar(500), floor_id [FK]: bigint, x: double, y: double, width: double, height: double, created_at: timestamp, updated_at: timestamp, created_by: varchar(255), updated_by: varchar(255))

Restaurant_Tables(id [PK]: bigint, capacity: int, created_at: datetime(6), updated_at: datetime(6), name: varchar(255), status: enum('AVAILABLE', 'OCCUPIED', 'RESERVED', 'UNAVAILABLE'), base_deposit: decimal(12,2), position_x: int, position_y: int, width: int, height: int, rotation: int, shape: varchar(50), area_id [FK]: bigint, image_url: varchar(500), description: text, created_by: varchar(255), updated_by: varchar(255))

4. Reservation & Order Management

Reservations(id [PK]: bigint, date/time fields, guest information, customer_id [FK], restaurant_table_id [FK], created_by_staff_id [FK], note, status enum, code, audit fields)

Orders(id [PK]: bigint, discount_price: decimal(38,2), final_price: decimal(38,2),

total_price: decimal(38,2), created_at: datetime(6), reservation_id [FK]: bigint, restaurant_table_id [FK]: bigint, updated_at: datetime(6), user_id [FK]: bigint, staff_id [FK]: bigint, voucher_id [FK]: bigint, status: enum('PRE_ORDER', 'CONFIRMED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED'), code: varchar(255), cancel_reason: varchar(255), created_by: varchar(255), updated_by: varchar(255))

Order_Details(id [PK]: bigint, quantity: int, total_price: decimal(38,2), combo_id [FK]: bigint, created_at: datetime(6), item_id [FK]: bigint, order_id [FK]: bigint, updated_at: datetime(6), note: varchar(255), status: enum('PENDING', 'COOKING', 'SERVED', 'CANCELLED'), created_by: varchar(255), updated_by: varchar(255))

Payments(id [PK]: bigint, amount: decimal(38,2), created_at: datetime(6), order_id [FK]: bigint, reservation_id [FK]: bigint, user_id [FK]: bigint, updated_at: datetime(6), transaction_code: varchar(255), method: enum('PAYOS', 'MOMO', 'CASH', 'ZALOPAY'), status: enum('PENDING', 'FAILED', 'SUCCESS'), type: enum('DEPOSIT', 'REFUND', 'SETTLEMENT'), target: enum('RESERVATION', 'ORDER'), source: enum('CLIENT_APP', 'POS_COUNTER'), raw_api_data: text, code: bigint, created_by: varchar(255), updated_by: varchar(255))

5. Attendance & Schedule

Shifts(id [PK]: bigint, end_time: time, start_time: time, created_at: datetime(6), updated_at: datetime(6), name: varchar(255), created_by: varchar(255), updated_by: varchar(255))

Work_Schedules(id [PK]: bigint, created_at: datetime(6), end_date: date, is_repeated: bit(1), repeat_group_id: varchar(255), shift_id [FK]: bigint, staff_id [FK]: bigint, note: varchar(200), work_date: date, updated_at: datetime(6), days_of_week: varchar(255), status: enum('APPROVED', 'CANCELLED', 'PENDING', 'REJECTED'), created_by: varchar(255), updated_by: varchar(255))

Attendances(id [PK]: bigint, check_in_at: datetime(6), check_out_at: datetime(6), created_at: datetime(6), updated_at: datetime(6), work_schedule_id [FK]: bigint, user_id [FK]: bigint, status: varchar(50), attendance_type: varchar(50), note: varchar(500), created_by: varchar(255), updated_by: varchar(255))

6. Voucher & Reward Points

Vouchers(id [PK]: bigint, discount_value: decimal(38,2), max_discount_value: decimal(38,2), min_value: decimal(38,2), points_required: int, created_at: datetime(6), updated_at: datetime(6), name: varchar(100), code: varchar(255), description: varchar(255), type: enum('FIXED_AMOUNT', 'PERCENTAGE'), created_by: varchar(255), updated_by: varchar(255))

User_Vouchers(id [PK]: bigint, created_at: datetime(6), customer_id [FK]: bigint, updated_at: datetime(6), voucher_id [FK]: bigint, usage_count: int, usage_limit: int, assigned_at: datetime(6), expired_at: datetime(6), created_by: varchar(255), updated_by: varchar(255))

Point_Histories(id [PK]: bigint, user_id [FK]: bigint, points_changed: int, reason: varchar(50), reference_id: bigint, created_at: timestamp, updated_at: timestamp, created_by: varchar(50), updated_by: varchar(50))

7. Feedback & Reviews

Feedbacks(id [PK]: bigint, rating: int, created_at: datetime(6), item_id [FK]: bigint, combo_id [FK]: bigint, order_id [FK]: bigint, order_detail_id [FK]: bigint, updated_at: datetime(6), user_id [FK]: bigint, guest_name: varchar(100), guest_email: varchar(255), is_featured: boolean, status: enum('PENDING', 'APPROVED', 'HIDDEN'), comment: varchar(200), created_by: varchar(255), updated_by: varchar(255))

Reviews(id [PK]: bigint, created_at: datetime(6), updated_at: datetime(6), comment: varchar(255), customer_id [FK]: bigint, created_by: varchar(255), updated_by: varchar(255))

8. Chat, Notification & Banner

Chat_Conversations(id [PK]: bigint, created_at: datetime(6), updated_at: datetime(6), title: varchar(255), user_id [FK]: bigint, created_by: varchar(255), updated_by: varchar(255))

Chat_Messages(id [PK]: bigint, created_at: datetime(6), updated_at: datetime(6), role: varchar(20), content: text, model: varchar(100), conversation_id [FK]: bigint, created_by: varchar(255), updated_by: varchar(255))

Notifications(id [PK]: bigint, user_id [FK]: bigint, notification type enum, title, message, payload, read_at, created_at, audit fields)

Banners(id [PK]: bigint, created_at: datetime(6), updated_at: datetime(6), title: varchar(100), description: varchar(500), banner_url: varchar(255), link_url: varchar(255), status: enum('ACTIVE', 'INACTIVE'), created_by: varchar(255), updated_by: varchar(255))

IV Implementation

IV.1 Map architecture to the structure of the project

This section maps the logical components in the Component Diagrams to the physical source code directory and file structure of the CrownDine project (covering both backend and frontend subsystems).

IV.1.1 Backend Subsystem (Spring Boot 3.5 Application)

The Backend architecture is designed following the Layered Architecture model and connects to external systems via defined boundary ports. Table 1 describes the detailed mapping of these components to the codebase:

Table 1: Architectural Mapping of the Backend Subsystem

UML Component Name	System Role	Project Directory & File Mapping
Backend Container	The outer container representing the entire server subsystem.	Directory <code>backend/</code>
HTTP Boundary Port	Boundary port receiving REST requests from the Client.	CORS/Filter configuration: <code>com.crowndine.config.WebMvcConfig</code>
WS Boundary Port	Boundary port receiving real-time WebSocket connections.	WebSocket endpoint configuration: <code>com.crowndine.config.WebSocketConfig</code>
Security & Auth Component	Handles JWT authentication and user authorization (Security Filter Chain).	Package <code>com.crowndine.security/</code> and security setup: <code>com.crowndine.config.SecurityConfig</code>
REST Controllers	API endpoints receiving REST requests from the client.	Package <code>com.crowndine.controller/</code> e.g., <code>ApiOrderController.java</code> , <code>ApiReservationController.java</code>
WebSocket Controllers	Endpoint receiving and dispatching WebSocket (STOMP) messages.	Package <code>com.crowndine.controller/</code> File: <code>WebSocketOrderController.java</code>
Application Services	Contains the core business logic of the restaurant.	Package <code>com.crowndine.service/</code> e.g., <code>OrderService.java</code> , <code>ReservationService.java</code>
Repositories	Data access layer executing SQL queries to the MySQL database.	Package <code>com.crowndine.repository/</code> e.g., <code>OrderRepository.java</code> , <code>ReservationRepository.java</code>
PayOS Adapter	Component encapsulating communication logic with the PayOS API.	Class <code>com.crowndine.service.PaymentService</code> and setup: <code>com.crowndine.config.PayOSConfig</code>
Cloudinary Adapter	Component managing image upload and storage on Cloudinary.	Upload helper configuration: <code>com.crowndine.config.CloudinaryConfig</code>
Spring AI Adapter	Component communicating with Google Gemini AI via the Spring AI framework.	Package <code>com.crowndine.ai/</code> Chat handler service: <code>AIService.java</code>

IV.1.2 Frontend Subsystem (React 19 SPA)

The Frontend architecture is built on the Single Page Application (SPA) model, component-driven, and utilizes centralized state management. Table 2 describes the detailed mapping of these components to the codebase:

Table 2: Architectural Mapping of the Frontend Subsystem

UML Component Name	System Role	Project Directory & File Mapping
Frontend Container	The outer container representing the entire client subsystem.	Directory <code>frontend/</code>
Event Port	Boundary port capturing user mouse/keyboard events.	App entry hook: <code>frontend/src/main.tsx</code>
React Router	Handles client-side routing and route authorization.	Main router setup: <code>frontend/src/router.tsx</code> and folder <code>frontend/src/routes/</code>
Views & Pages Component	Main UI screens partitioned by user roles and access rights.	Directory <code>frontend/src/pages/</code> • Customer Pages: <code>Home/</code>, <code>Menu/</code>, <code>Reservation/</code> • Kitchen Display: <code>Staffs/KitchenDisplay/</code> • Staff & Cashier: <code>Staffs/Cashier.tsx</code>, <code>Staffs/FloorPlan/</code> • Admin Pages: <code>Admin/Dashboard/</code>, <code>Admin/Staff/</code>
Shared UI Components	Library of shared reusable UI components (Buttons, Modals, Tables).	Folder <code>frontend/src/components/</code> (integrated with Tailwind CSS and Radix UI)
Zustand Stores	Manages global application state (such as shopping cart, auth status).	Directory <code>frontend/src/stores/</code> e.g., <code>authStore.ts</code> , <code>cartStore.ts</code>
React Query / Axios	Performs asynchronous REST API calls and manages data caching.	Folder <code>frontend/src/apis/</code> and custom API hooks at <code>frontend/src/hooks/</code>
WebSocket STOMP	Client maintaining the real-time WebSocket connection to the Server.	WebSocket context configuration: <code>frontend/src/contexts/</code> or socket helpers.

IV.1.3 Project Directory Structure Overview

To visually illustrate the source code organization, the project directory structure is laid out as follows:

```

CrownDine/
|-- backend/                                # BACKEND SUBSYSTEM
|   |-- src/main/java/com/crowndine/
|   |   |-- ai/                            # Spring AI / Gemini Chat
|   |   |-- common/                        # Shared utilities
|   |   |-- config/                       # Integration and security configs
|   |   |-- controller/                   # REST and WebSocket controllers
|   |   |-- dto/                          # Data Transfer Objects

```



```

| | | -- exception/          # Global exception handler
| | | -- model/              # JPA entities and DB mapping
| | | -- repository/         # Data access layer
| | | -- security/           # JWT filters and authorization
| | | -- service/            # Application services
| | -- pom.xml               # Maven dependencies
|
|-- frontend/                # FRONTEND SUBSYSTEM
|   |-- src/
|     |-- apis/              # Axios API clients
|     |-- components/        # Shared UI components
|     |-- hooks/             # React Query hooks
|     |-- layouts/           # Layout templates
|     |-- pages/             # UI views and screens
|     |-- router.tsx          # Client routing and guards
|     |-- stores/            # Zustand global stores
|     |-- types/             # TypeScript definitions
|     -- main.tsx            # Application entry point
|-- package.json             # NPM scripts

```

IV.2 Mapping Class Diagrams & Interaction Diagrams to Code

Based on the Detailed Design classes and the Entity–Relationship Diagram (ERD) of the Crown-Dine project, the mapping of the core Item Management, Reservation Management, and Order Management modules into Java Spring Boot code is presented as follows.

1. Item Management Module

Service Interface

```

package com.crowndine.service;

import com.crowndine.model.Item;
import java.util.List;

public interface ItemService {
    List<Item> getItems();
    Item getItem(Long id);
    Item addItem(Item item);
    Item editItem(Item item);
    boolean deleteItem(Long id);
}

```

Service Implementation

```

package com.crowndine.service.impl;

import com.crowndine.model.Item;
import com.crowndine.repository.ItemRepository;
import com.crowndine.repository.ComboItemRepository;
import com.crowndine.repository.OrderDetailRepository;
import com.crowndine.service.ItemService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

```

```
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.Optional;

@Service
public class ItemServiceImpl implements ItemService {
    @Autowired
    private ItemRepository itemRepository;

    @Autowired
    private ComboItemRepository comboItemRepository;

    @Autowired
    private OrderDetailRepository orderDetailRepository;

    @Override
    public List<Item> getItems() {
        return itemRepository.findAll();
    }

    @Override
    public Item getItem(Long id) {
        return itemRepository.findById(id).orElse(null);
    }

    @Override
    @Transactional
    public Item addItem(Item item) {
        return itemRepository.save(item);
    }

    @Override
    @Transactional
    public Item editItem(Item item) {
        return itemRepository.save(item);
    }

    @Override
    @Transactional
    public boolean deleteItem(Long id) {
        boolean isReferenced = comboItemRepository.existsByItemId(id)
            || orderDetailRepository.existsByItemId(id);

        if (isReferenced) {
            return false;
        }

        Optional<Item> item = itemRepository.findById(id);
        if (item.isPresent()) {
            itemRepository.delete(item.get());
            return true;
        }
        return false;
    }
}
```

Controller Implementation

```
package com.crowndine.controller;

import com.crowndine.model.Item;
import com.crowndine.service.ItemService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@Controller
@RequestMapping("/item")
public class ItemController {
    @Autowired
    private ItemService itemService;

    @GetMapping("/manage")
    public String manage(Model model) {
        List<Item> items = itemService.getItems();
        model.addAttribute("items", items);
        return "item/manage";
    }

    @GetMapping("/add")
    public String add() {
        return "item/_addItemModal";
    }

    @PostMapping("/add")
    public String add(@ModelAttribute Item item, Model model) {
        try {
            itemService.addItem(item);
            return "redirect:/item/manage";
        } catch (Exception e) {
            model.addAttribute("item", item);
            return "item/_addItemModal";
        }
    }
}
```

2. Reservation Management Module

Service Interface

```
package com.crowndine.service;

import com.crowndine.model.Reservation;
import java.util.List;

public interface ReservationService {
    List<Reservation> getReservations();
    Reservation getReservation(Long id);
    Reservation createReservation(Reservation reservation);
    Reservation editReservation(Reservation reservation);
    boolean cancelReservation(Long id, String cancelReason);
}
```

Service Implementation

```
package com.crowndine.service.impl;

import com.crowndine.model.Reservation;
import com.crowndine.repository.ReservationRepository;
import com.crowndine.service.ReservationService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@Service
public class ReservationServiceImpl implements ReservationService {
    @Autowired
    private ReservationRepository reservationRepository;

    @Override
    public List<Reservation> getReservations() {
        return reservationRepository.findAll();
    }

    @Override
    public Reservation getReservation(Long id) {
        return reservationRepository.findById(id).orElse(null);
    }

    @Override
    @Transactional
    public Reservation createReservation(Reservation reservation) {
        reservation.setStatus("Pending");
        reservation.setCreatedAt(LocalDateTime.now());
        return reservationRepository.save(reservation);
    }

    @Override
    @Transactional
    public Reservation editReservation(Reservation reservation) {
        return reservationRepository.save(reservation);
    }

    @Override
    @Transactional
    public boolean cancelReservation(Long id, String cancelReason) {
        Optional<Reservation> optionalReservation =
            reservationRepository.findById(id);

        if (optionalReservation.isPresent()) {
            Reservation reservation = optionalReservation.get();
            if (!"Cancelled".equals(reservation.getStatus())
                && !"Completed".equals(reservation.getStatus())) {
                reservation.setStatus("Cancelled");
                reservation.setNote(cancelReason);
                reservationRepository.save(reservation);
                return true;
            }
        }
    }
}
```

```
        return false;
    }
}
```

Controller Implementation

```
package com.crowndine.controller;

import com.crowndine.model.Reservation;
import com.crowndine.service.ReservationService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;

@Controller
@RequestMapping("/reservation")
public class ReservationController {
    @Autowired
    private ReservationService reservationService;

    @GetMapping("/manage")
    public String manage(Model model) {
        List<Reservation> reservations = reservationService.getReservations();
        model.addAttribute("reservations", reservations);
        return "reservation/manage";
    }

    @GetMapping("/create")
    public String create() {
        return "reservation/_createReservationModal";
    }

    @PostMapping("/create")
    public String create(@ModelAttribute Reservation reservation, Model model) {
        try {
            reservationService.createReservation(reservation);
            return "redirect:/reservation/manage";
        } catch (Exception e) {
            model.addAttribute("reservation", reservation);
            return "reservation/_createReservationModal";
        }
    }

    @PostMapping("/cancel/{id}")
    @ResponseBody
    public ResponseEntity<?> cancel(@PathVariable Long id,
                                    @RequestParam String reason) {
        boolean success = reservationService.cancelReservation(id, reason);
        if (success) {
            return ResponseEntity.ok(Map.of(
                "message", "Reservation cancelled successfully."));
        }
        return ResponseEntity.badRequest().body(Map.of(
            "message", "Failed to cancel this reservation."));
    }
}
```

```
}  
}
```

3. Order Management Module

Entity Mapping

```
package com.crowndine.model;  
  
import jakarta.persistence.*;  
import java.math.BigDecimal;  
import java.time.LocalDateTime;  
  
@Entity  
@Table(name = "Orders")  
public class Order {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(name = "total_price")  
    private BigDecimal totalPrice;  
  
    @Column(name = "final_price")  
    private BigDecimal finalPrice;  
  
    private String status;  
    private String code;  
  
    @Column(name = "created_at")  
    private LocalDateTime createdAt;  
  
    // Getters and setters  
}
```

Repository Mapping

```
package com.crowndine.repository;  
  
import com.crowndine.model.Order;  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
  
@Repository  
public interface OrderRepository extends JpaRepository<Order, Long> {  
}
```

Service Interface

```
package com.crowndine.service;  
  
import com.crowndine.model.Order;  
import java.util.List;  
  
public interface OrderService {  
    List<Order> getOrders();  
    Order getOrder(Long id);  
}
```

```
    Order createOrder(Order order);
    Order processPayment(Long id);
}
```

Service Implementation

```
package com.crowndine.service.impl;

import com.crowndine.model.Order;
import com.crowndine.repository.OrderRepository;
import com.crowndine.service.OrderService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@Service
public class OrderServiceImpl implements OrderService {
    @Autowired
    private OrderRepository orderRepository;

    @Override
    public List<Order> getOrders() {
        return orderRepository.findAll();
    }

    @Override
    public Order getOrder(Long id) {
        return orderRepository.findById(id).orElse(null);
    }

    @Override
    @Transactional
    public Order createOrder(Order order) {
        order.setStatus("Pending");
        order.setCreatedAt(LocalDateTime.now());
        return orderRepository.save(order);
    }

    @Override
    @Transactional
    public Order processPayment(Long id) {
        Optional<Order> orderOpt = orderRepository.findById(id);
        if (orderOpt.isPresent()) {
            Order order = orderOpt.get();
            if (!"Paid".equals(order.getStatus())) {
                order.setStatus("Paid");
                return orderRepository.save(order);
            }
        }
        return null;
    }
}
```

Controller Implementation

```
package com.crowndine.controller;

import com.crowndine.model.Order;
import com.crowndine.service.OrderService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;

@Controller
@RequestMapping("/order")
public class OrderController {
    @Autowired
    private OrderService orderService;

    @GetMapping("/manage")
    public String manage(Model model) {
        List<Order> orders = orderService.getOrders();
        model.addAttribute("orders", orders);
        return "order/manage";
    }

    @PostMapping("/create")
    @ResponseBody
    public ResponseEntity<?> createOrder(@RequestBody Order order) {
        try {
            Order createdOrder = orderService.createOrder(order);
            return ResponseEntity.ok(createdOrder);
        } catch (Exception e) {
            return ResponseEntity.badRequest().body(Map.of(
                "message", "Failed to create order."));
        }
    }

    @PostMapping("/checkout/{id}")
    @ResponseBody
    public ResponseEntity<?> checkout(@PathVariable Long id) {
        Order processedOrder = orderService.processPayment(id);
        if (processedOrder != null) {
            return ResponseEntity.ok(Map.of(
                "message", "Payment processed successfully.",
                "order", processedOrder));
        }
        return ResponseEntity.badRequest().body(Map.of(
            "message", "Failed to process payment. "
                + "Order might not exist or is already paid."));
    }
}
```


IV.3 Applying Alternative Architecture Patterns

During the design process of the **CrownDine** system, several alternative architectural patterns were analyzed and evaluated to compare their advantages and disadvantages against the current architecture (Monolithic 3-Tier integrated with WebSockets and Spring AI).

The following details the alternative patterns considered and the rationale behind selecting the current optimal architecture:

IV.3.1 1. Microservices Architecture

Description: Deconstruct the current Spring Boot Monolith application into self-contained, deployable services running in individual containers such as: *Reservation Service*, *Ordering Service*, *Kitchen (KDS) Service*, *Billing Service*, and *AI Analytics Service*, communicating via an API Gateway and REST/gRPC protocols.

- **Advantages (Pros):**

- **Scalability:** Offers independent scaling capabilities for high-load services (e.g., scaling up the Ordering Service instances during peak hours without redeploying the entire system).
- **Fault Isolation:** A failure in one service (e.g., AI Service timeout) will not disrupt core functionalities like customer ordering or cashier billing.
- **Technology Flexibility:** Enables different technology stacks for different services (e.g., using Python for the AI Service).

- **Disadvantages (Cons):**

- **Complexity:** Increases distributed management overhead (Service Discovery, API Gateway, Distributed Transactions, Eventual Consistency).
- **Network Latency:** Introduces additional network overhead due to continuous inter-service remote communication.
- **Operation Cost:** Requires complex CI/CD pipelines, container orchestration (Kubernetes), and centralized logging.

Decision: *Rejected for the current phase.* Since CrownDine is designed to serve small-to-medium-sized restaurant branches, a Monolithic architecture is the most cost-effective and optimal approach. It ensures database ACID transaction integrity and simplifies operational complexity.

IV.3.2 2. Event-Driven Architecture (EDA)

Description: Instead of synchronous REST API calls or local WebSocket message dispatching, the system integrates an external Message Broker such as **Apache Kafka** or **RabbitMQ**. When a customer places a reservation or order, an event (e.g., **OrderCreatedEvent**) is published to the broker for the KDS and Notification Service to consume asynchronously.

- **Advantages (Pros):**

- **High Throughput:** Exceptional asynchronous processing capacity, dampening instant load spikes during peak hours.
 - **Loose Coupling:** Eliminates tight compile-time dependencies between services, allowing new services (e.g., Loyalty/Voucher Service) to plug in seamlessly by subscribing to events.
 - **Message Durability:** Messages are persisted in queue topics and support robust retry mechanisms if a client momentarily disconnects.
- **Disadvantages (Cons):**
 - **Eventual Consistency:** Developers must handle eventual consistency instead of immediate consistency, complicating workflows like order cancellations or item updates.
 - **Debugging Difficulty:** Tracing and debugging messages across distributed queues is highly complex.

Decision: *Postponed for future multi-branch phase.* The local WebSocket STOMP Broker embedded in Spring Boot currently meets all real-time requirements with low latency (< 100ms). We will pivot to Apache Kafka/RabbitMQ once the system scales to manage franchise-level chains.

IV.3.3 3. Serverless (Function-as-a-Service - FaaS)

Description: Offload heavy analytics computations or AI prompt construction logic to on-demand Serverless Cloud Functions (e.g., AWS Lambda or Google Cloud Functions).

- **Advantages (Pros):**
 - **Cost Efficiency:** Scale-to-zero model where costs are only incurred per chatbot execution without running an idle server 24/7.
 - **Auto-scaling:** Instantly scales up when hundreds of managers query the chatbot simultaneously.
- **Disadvantages (Cons):**
 - **Cold Start Latency:** Initial requests may experience start-up delays, slowing down the chatbot's response time.
 - **Data Connection Complexity:** High complexity in managing secure, persistent database connections from ephemeral functions back to the MySQL Database.

Decision: *Rejected.* Utilizing the **Spring AI** framework directly inside Spring Boot allows the system to fetch database reports and construct context prompts locally, ensuring high security and data privacy without external cloud orchestration.

IV.3.4 Trade-off Analysis Matrix

Table 3 summarizes the trade-offs of the analyzed architecture patterns for the CrownDine project:

Table 3: Architectural Trade-off Comparison Matrix

Evaluation Criteria	Monolith + WS	Microservices	Event-Driven	Serverless
Development Simplicity	Very High	Low	Medium	Medium
Real-time Performance	High	Medium	Very High	Low
Maintainability & Debugging	Easy	Complex	Very Complex	Medium
Operating Cost	Most Optimal	Very High	High	Medium
Scalability	Medium	Very High	Very High	Very High

V Service-Oriented Architecture (SOA)

V.1 Applying the Service-Oriented Architecture

NF-05 Reusability: The system should be built so that the same code and features can be reused across different platforms, allowing the same backend functionality to serve both the Web and Mobile applications without duplicating business logic. This saves time and effort when adding new platforms or updating existing ones. To satisfy this requirement, the architecture is redesigned using a service-oriented approach.

The current CrownDine architecture is a single monolithic Spring Boot backend, which does not fully satisfy NF-05. Any new client platform, such as the upcoming mobile application, would have to depend on the same tightly-coupled backend with no clear service boundary. To address this, the backend is restructured into six coarse-grained service components: Auth & User Service, Menu Service, Reservation & Table Service, Order & Payment Service, Staff & Workforce Service and Notification & AI Service. Each service is deployed as an independent Spring Boot application artifact within its own execution environment.

The system exposes a single REST API contract through the API Gateway Server, implemented using Spring Cloud Gateway. Both the Web Browser, implemented as a React single-page application, and the Mobile App, implemented using React Native, consume this contract over HTTPS, while the gateway forwards each request over HTTP to the appropriate backend service. Because both platforms rely on the exact same contract and the same set of services, no backend logic needs to be duplicated or re-implemented when a new client platform is introduced. This directly fulfills NF-05 Reusability.

All six services continue to share a single MySQL database using the `crowndine` schema, accessed through JPA/Hibernate over JDBC. This design intentionally adopts a shared-database approach instead of the database-per-service pattern commonly adopted in microservice architectures in order to simplify data consistency and reduce migration complexity. Therefore, the proposed architecture reflects a service-oriented style rather than a fully decentralized microservices style.

In addition, three services integrate with external cloud providers. The Order & Payment Service connects to PayOS for payment processing, the Menu Service connects to Cloudinary for media storage, and the Notification & AI Service connects to both Gemini AI for the chatbot feature and an external Email/SMS provider for notification delivery. Each external service is reached over HTTPS.

This restructuring transforms the original monolithic architecture into a distributed service-oriented architecture. By exposing a common REST API contract through the API Gateway,

the backend services can be reused across multiple client platforms without duplicating business logic, thereby satisfying NF-05 Reusability. In addition, the architecture provides a clearer separation of business capabilities, improved modularity, easier maintenance, and greater flexibility for future system evolution. The resulting software architecture is illustrated in Figure V-1.

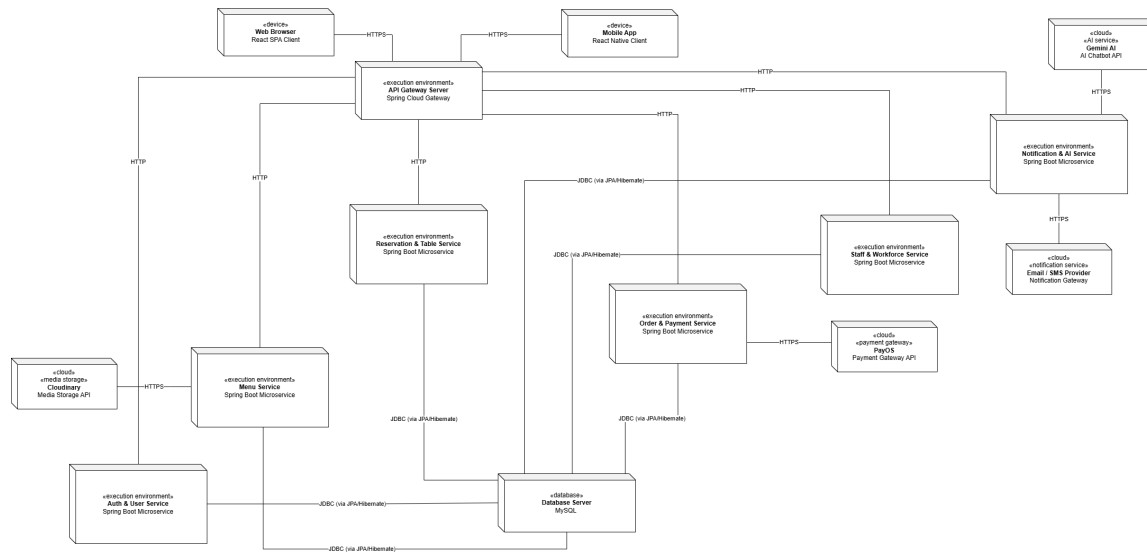


Figure 47: Deployment Diagram for the CrownDine System Architecture Based on Service-Oriented Architecture

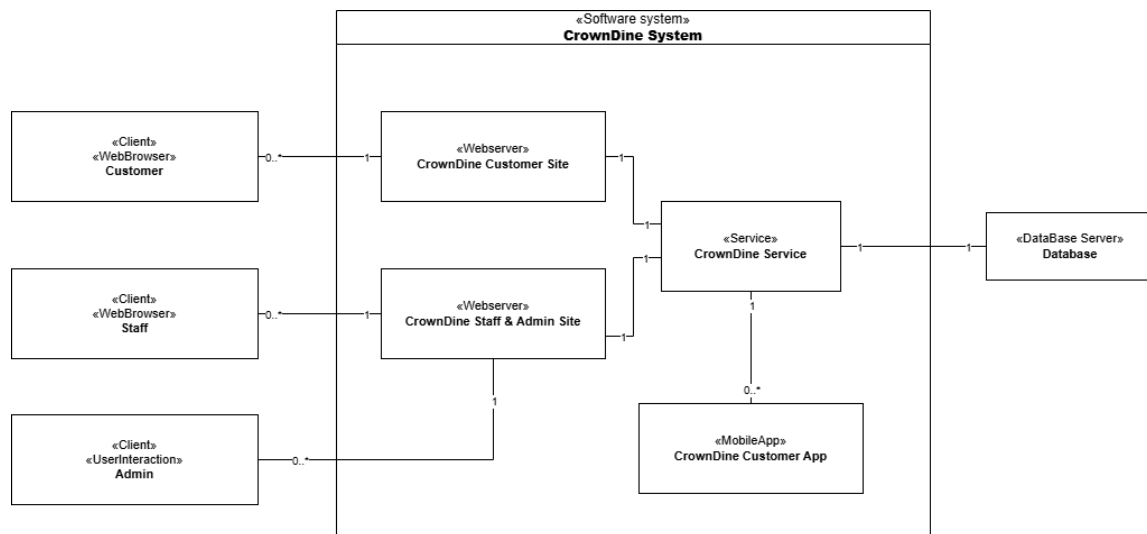


Figure 48: Class Diagram for the System Architecture Based on Service-Oriented Architecture

The Service Discovery Pattern enables services to locate each other dynamically through a service registry. In the Crowndine system, this pattern may be applied if the project evolves into independent services such as order service, menu service, payment service, and notification service.

V.2 Applying Service Discovery Pattern in the Service-Oriented Architecture

Assuming that the Crowndine restaurant system expands into multiple branches and serves an increasing number of customers, staff members, and concurrent transactions, the system must satisfy an additional non-functional requirement related to scalability, availability, and performance.

NF-06: Scalability and High Availability.

The system shall be designed to accommodate an increasing number of restaurant branches, users, reservations, menu requests, and customer orders while maintaining acceptable performance and reliability. The architecture shall support both horizontal and vertical scaling. New service instances must be added or removed without requiring changes to client applications or hard-coded service addresses.

To satisfy this requirement, the Crowndine RMS applies the Service Discovery Pattern in a microservice-oriented architecture. An API Gateway acts as the single entry point for customer, staff, administrator, and mobile applications. Each microservice registers its current network location with the Service Registry when it starts and periodically sends heartbeat signals to indicate that it remains available.

When the API Gateway needs to process a request, it queries the Service Registry to locate an available instance of the required service. The request is then routed to one of the available service instances. This mechanism supports dynamic service location, load balancing, fault isolation, and independent service deployment.

The main services of the Crowndine RMS include:

- **User Service:** manages customer, staff, administrator accounts, authentication, authorization, and user roles.
- **Menu Service:** manages menu categories, menu items, prices, descriptions, images, and availability.
- **Reservation Service:** manages table reservations, guest information, time slots, table assignment, updates, and cancellations.
- **Order Service:** manages dine-in and online orders, order items, special requests, and order status.
- **Payment Service:** handles payment requests, payment status, transaction references, and communication with external payment providers.
- **Notification Service:** sends reservation confirmations, order status updates, payment notifications, email messages, SMS messages, and push notifications.
- **Reporting Service:** generates reports related to sales, orders, reservations, menu performance, and restaurant operations.

The Service Registry stores the locations and health status of all available service instances. The API Gateway discovers the required service dynamically instead of using a fixed address. If

one service instance becomes unavailable, the request may be redirected to another healthy instance. This design improves system reliability and supports future expansion of the Crowndine restaurant chain.

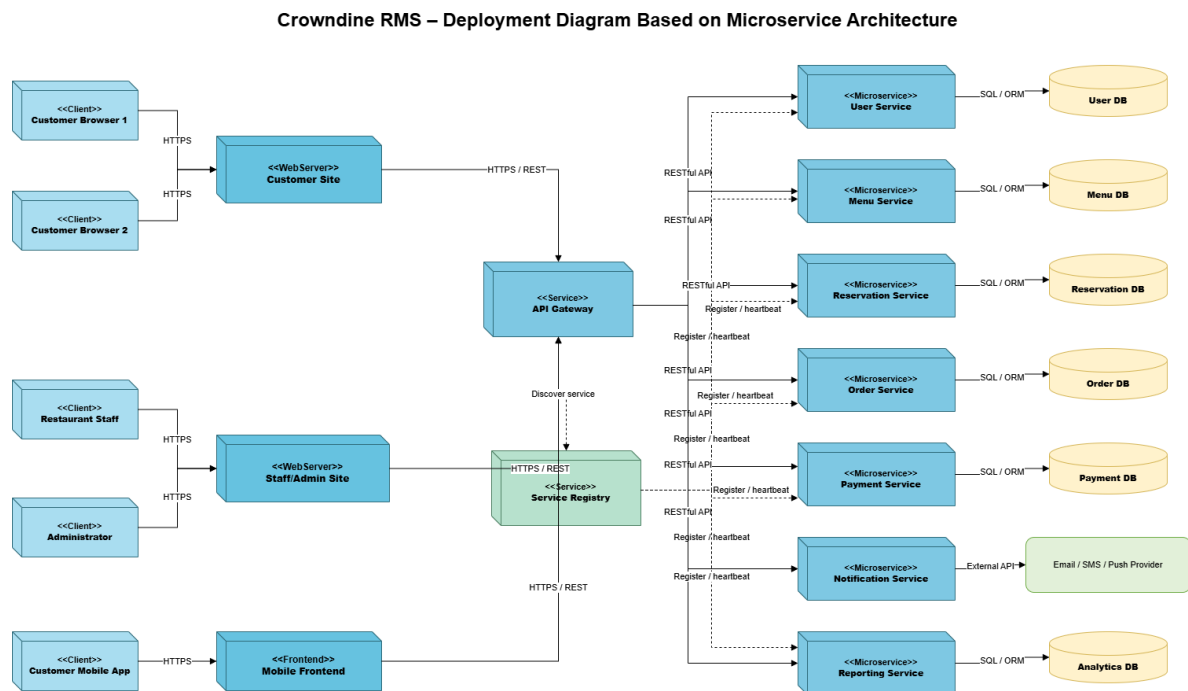


Figure V-4. Deployment Diagram for Crowndine RMS Based on Microservice Architecture

Figure 49: Deployment Diagram for the Crowndine RMS Architecture Based on Microservice Architecture

Figure 49 illustrates the physical deployment of the Crowndine RMS. Customers access the Customer Site or Mobile Application, while restaurant staff and administrators access the Staff/Admin Site. All frontend applications communicate with the API Gateway through HTTPS and RESTful APIs.

The API Gateway communicates with the Service Registry to identify available service instances. It then routes requests to User Service, Menu Service, Reservation Service, Order Service, Payment Service, Notification Service, or Reporting Service. Each major service may use an independent database to reduce coupling and allow separate scaling.

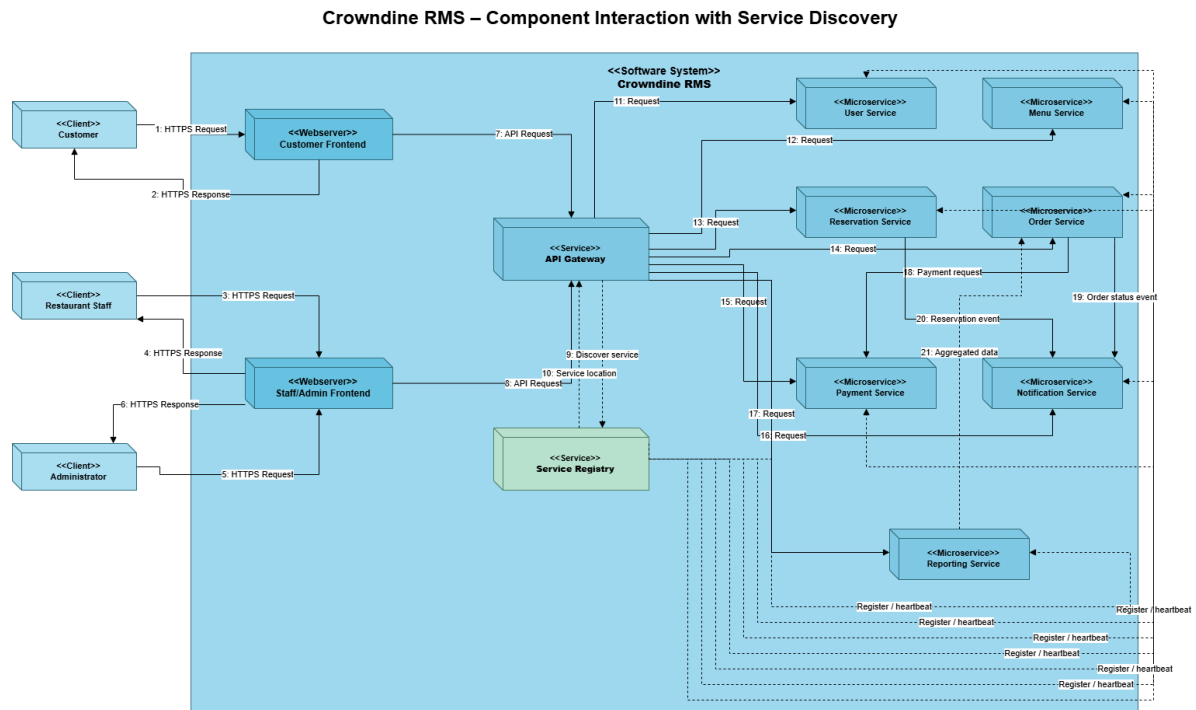


Figure V-6. Component Interaction Diagram for Crowndine RMS Based on Microservice Architecture

Figure 50: Architecture Class Diagram for the Crowndine RMS Based on Microservice Architecture

Figure 50 presents the main architectural components and their relationships. Customer Web, Customer Mobile, Restaurant Staff, and Administrator clients communicate with their corresponding frontend applications. These frontend applications send requests to the API Gateway.

The API Gateway is associated with the Service Registry and the system microservices. Each microservice is independently deployable and is responsible for a specific business capability. This separation improves maintainability, testability, and scalability.

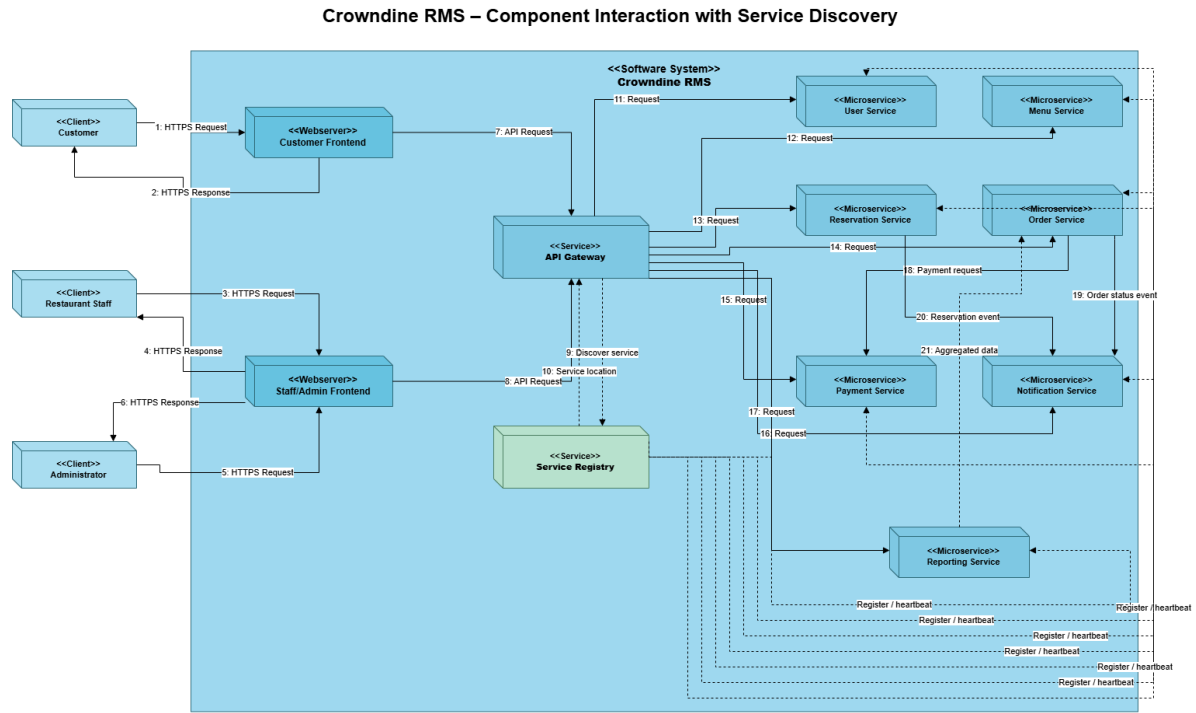


Figure V-6. Component Interaction Diagram for Crowndine RMS Based on Microservice Architecture

Figure 51: Component Interaction Diagram for the Crowndine RMS Based on Microservice Architecture

Figure 51 describes the interaction flow among the major components. The typical processing sequence is as follows:

1. A customer, staff member, or administrator sends an HTTPS request to a frontend application.
2. The frontend application forwards the request to the API Gateway.
3. The API Gateway sends a discovery request to the Service Registry.
4. The Service Registry returns the location of a healthy service instance.
5. The API Gateway forwards the request to the selected microservice.
6. The microservice performs the required business operation and accesses its database where necessary.
7. The microservice returns the processing result to the API Gateway.
8. The API Gateway returns the response to the frontend application and then to the user.

For example, when a customer places an order, the API Gateway discovers the Order Service and forwards the order request. The Order Service may communicate with the Menu Service to validate item availability, with the Payment Service to process payment, and with the Notification Service to send order confirmation or status updates.

When a customer creates a reservation, the Reservation Service checks table availability and stores the reservation information. After the reservation is confirmed, the Reservation Service publishes an event or sends a request to the Notification Service so that a confirmation message can be delivered to the customer.

The Service Discovery Pattern provides the following advantages for the Crowndine RMS:

- Services can be deployed, restarted, scaled, or moved without changing client applications.
- Multiple instances of frequently used services can be created to support load balancing.
- Failed service instances can be removed automatically from the Service Registry.
- Each business capability can be maintained and deployed independently.
- The architecture can support future restaurant branches and a larger number of concurrent users.

However, this architecture also introduces several disadvantages:

- The Service Registry becomes an important infrastructure component and must be highly available.
- Network communication between services increases system complexity.
- Distributed tracing, logging, monitoring, and error handling are required.
- Data consistency across separate service databases is more difficult to manage.
- Deployment and testing are more complex than in a monolithic architecture.

To reduce these risks, the Service Registry should be deployed with multiple instances, health checks, and automatic failover. The API Gateway should apply load balancing, request timeout, retry, circuit breaker, authentication, and rate-limiting policies. Centralized logging and monitoring should also be used to observe service health and request performance.

Based on the scalability requirement NF-06, the Service Discovery Pattern is appropriate for the future development of the Crowndine RMS. It allows the system to expand from a single restaurant into a multi-branch restaurant platform while preserving modularity, maintainability, availability, and performance.